
Phân tích và Quản lý đầu tư nâng cao

Kinh tế lượng và trí tuệ nhân tạo trong tài chính

TS. Đỗ Như Tài
Đại Học Sài Gòn
dntai@sgu.edu.vn

Kinh tế lượng và Trí tuệ nhân tạo trong tài chính

Mục lục

- **Kinh tế lượng tài chính**
 - Lý thuyết tài chính, Phân tích hồi quy dùng OLS
- **Máy học**
 - Quá trình học, Đánh giá, Bias and variance
 - Cross-validation
- **Trí tuệ nhân tạo trong tài chính**
 - Thị trường hiệu quả
 - Dự đoán thị trường dựa trên dữ liệu lợi nhuận
 - Dự đoán thị trường với nhiều tính năng hơn

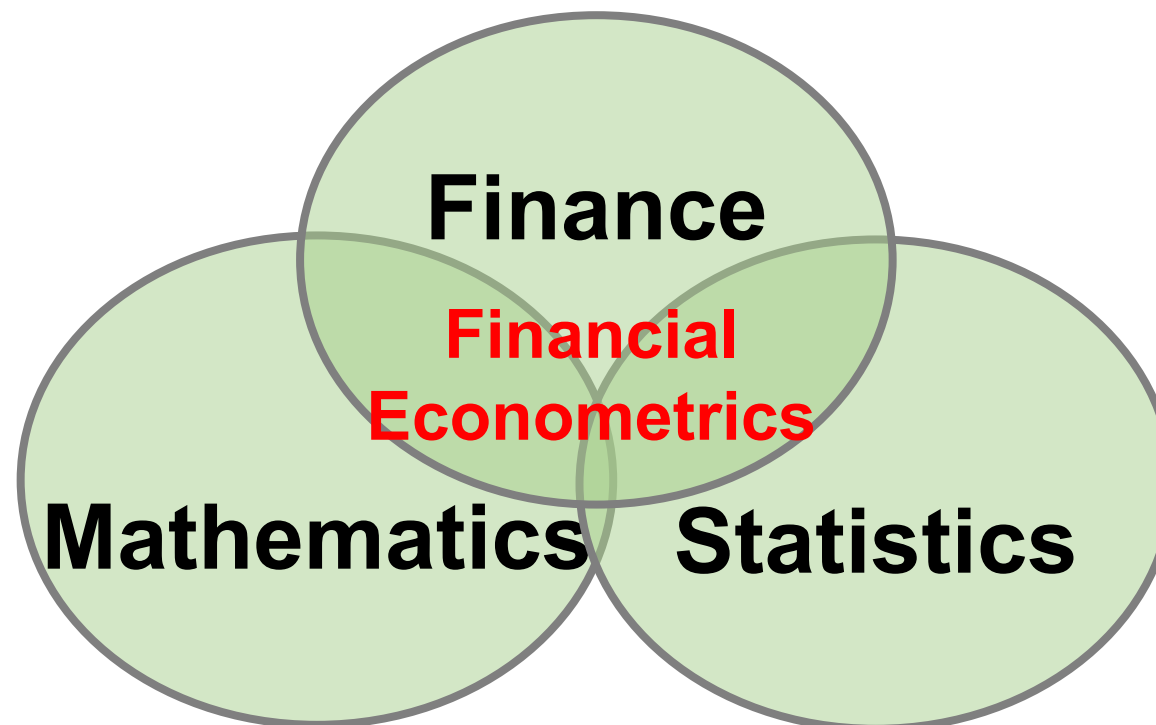
Kinh tế lượng tài chính

Kinh tế lượng tài chính

- Lý thuyết tài chính
- Phân tích hồi quy dùng OLS

Kinh tế lượng tài chính

- Môn học giao thoa giữa toán học, thống kê và tài chính áp dụng các phương pháp đó vào dữ liệu thị trường tài chính thường được gọi là kinh tế lượng tài chính.



Kinh tế lượng tài chính

(Chris Brooks, 2019)

- **Kinh tế lượng tài chính**
 - việc áp dụng các kỹ thuật thống kê vào các vấn đề tài chính
- Kinh tế lượng tài chính có thể hữu ích để kiểm tra các lý thuyết về tài chính,
 - xác định giá hoặc lợi nhuận tài sản,
 - kiểm tra các giả thuyết liên quan đến mối quan hệ giữa các biến số,
 - kiểm tra tác động lên thị trường tài chính của những thay đổi trong điều kiện kinh tế,
 - dự báo giá trị tương lai của các biến tài chính và ra quyết định tài chính.

Kinh tế lượng tài chính

- Kinh tế lượng [tài chính] là ứng dụng định lượng của các mô hình thống kê và toán học sử dụng dữ liệu [tài chính] để phát triển các lý thuyết tài chính hoặc kiểm tra các giả thuyết hiện có về tài chính và dự báo xu hướng trong tương lai từ dữ liệu lịch sử.
- Nó đưa dữ liệu [tài chính] trong thế giới thực vào các thử nghiệm thống kê, sau đó so sánh và đối chiếu các kết quả với lý thuyết [tài chính] hoặc các lý thuyết đang được thử nghiệm.

Các chủ đề trong Kinh tế lượng tài chính

(Oliver Linton, 2019)

1. Kinh tế lượng
2. Khả năng dự đoán lợi nhuận và giả thuyết thị trường hiệu quả
3. Các thử nghiệm và thử nghiệm mạnh mẽ về khả năng dự đoán lợi nhuận phi tuyến tính
4. Cấu trúc vi mô thị trường thực nghiệm
5. Phân tích nghiên cứu sự kiện
6. Lựa chọn danh mục đầu tư và thử nghiệm mô hình định giá tài sản vốn
7. Mô hình định giá đa yếu tố

Các chủ đề trong Kinh tế lượng tài chính

(Oliver Linton, 2019)

- 8. Quan hệ giá trị hiện tại
- 9. Định giá cân bằng giữa các thời gian
- 10. Biến động
- 11. Quy trình thời gian liên tục
- 12. Đường cong lợi suất
- 13. Quản lý rủi ro và ước tính đuôi

Các ứng dụng của Kinh tế lượng tài chính

(Chris Brooks, 2019)

1. Kiểm tra xem thị trường tài chính có hiệu quả về mặt thông tin ở dạng yếu hay không
2. Kiểm tra xem mô hình định giá tài sản vốn (CAPM) hay lý thuyết định giá chênh lệch giá (APT) có phải là các mô hình ưu việt để xác định lợi nhuận trên tài sản rủi ro hay không
3. Đo lường và dự báo sự biến động của lợi nhuận trái phiếu
4. Giải thích các yếu tố quyết định xếp hạng tín dụng trái phiếu được các cơ quan xếp hạng sử dụng
5. Mô hình hóa mối quan hệ lâu dài giữa giá cả và tỷ giá hối đoái

Các ứng dụng của Kinh tế lượng tài chính

(Chris Brooks, 2019)

6. Xác định tỷ lệ phòng ngừa rủi ro tối ưu cho vị thế giao ngay trong dầu
7. Kiểm tra các quy tắc giao dịch kỹ thuật để xác định quy tắc nào kiếm được nhiều tiền nhất
8. Kiểm tra giả thuyết rằng thu nhập hoặc thông báo cổ tức không ảnh hưởng đến giá cổ phiếu
9. Kiểm tra xem thị trường giao ngay hay thị trường tương lai phản ứng nhanh hơn với tin tức
10. Dự báo mối tương quan giữa chỉ số chứng khoán hai nước

Học máy và Kinh tế lượng tài chính

- Các phương pháp ML và DL có thể phát hiện ra những điểm kém hiệu quả về mặt thống kê và thậm chí cả những điểm kém hiệu quả về mặt kinh tế mà các phương pháp kinh tế lượng truyền thống không thể phát hiện được, chẳng hạn như hồi quy OLS đa biến.

Các lý thuyết tài chính chuẩn mực

- Các lý thuyết tài chính chuẩn mực chủ yếu dựa vào các giả định và tiên đề kết hợp với suy luận là phương pháp phân tích chính để đi đến kết quả trung tâm của chúng.
 - Lý thuyết hữu dụng kỳ vọng (EUT) giả định rằng các tác nhân có cùng chức năng hữu dụng bất kể trạng thái thế giới diễn ra như thế nào và họ tối đa hóa hữu dụng kỳ vọng trong những điều kiện không chắc chắn.
 - Lý thuyết danh mục đầu tư phương sai trung bình (MVP) mô tả cách các nhà đầu tư nên đầu tư trong điều kiện không chắc chắn với giả định rằng chỉ có lợi nhuận kỳ vọng và độ biến động kỳ vọng của danh mục đầu tư trong một khoảng thời gian mới được tính.

Các lý thuyết tài chính chuẩn mực

- Mô hình định giá tài sản vốn (CAPM) giả định rằng chỉ có rủi ro thị trường không thể đa dạng hóa mới giải thích được lợi nhuận kỳ vọng và sự biến động kỳ vọng của một cổ phiếu trong một thời kỳ.
- Lý thuyết định giá chênh lệch giá (APT) giả định rằng một số yếu tố rủi ro có thể xác định được sẽ giải thích cho lợi nhuận kỳ vọng và sự biến động kỳ vọng của một cổ phiếu theo thời gian; phải thừa nhận rằng, so với các lý thuyết khác, việc xây dựng APT khá rộng và cho phép diễn giải trên phạm vi rộng.

Kinh tế lượng tài chính và hồi quy

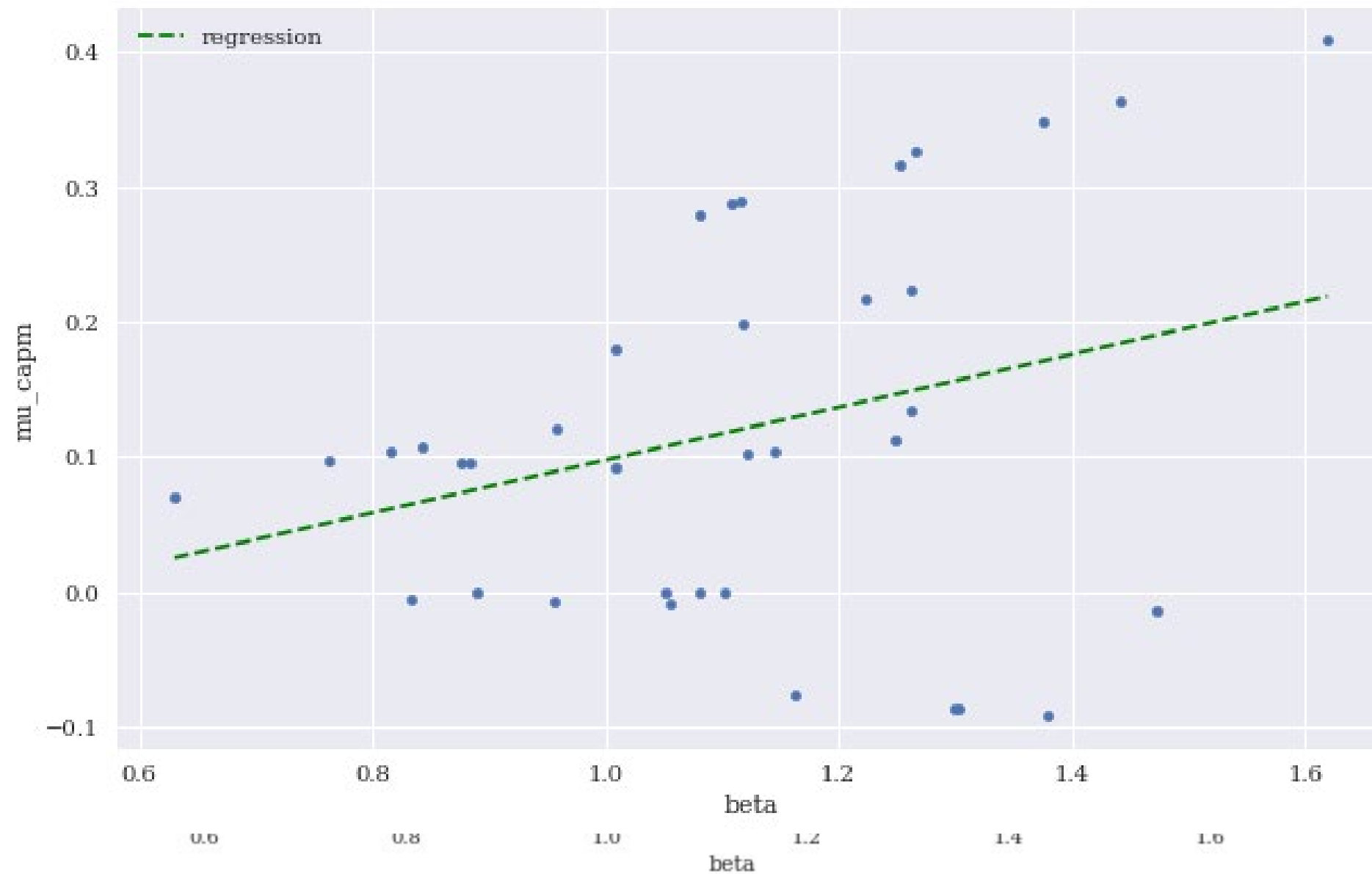
- Một trong những công cụ chính trong kinh tế lượng tài chính là hồi quy, ở cả dạng đơn biến và đa biến.
 - $y = \alpha + \beta x$
 - $y = \alpha + \beta_1 x_1 + \beta_2 x_2$
 - $y = \alpha + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3$
- Hồi quy cũng là một công cụ trung tâm trong học thống kê nói chung

CAPM và APT

Hồi quy OLS

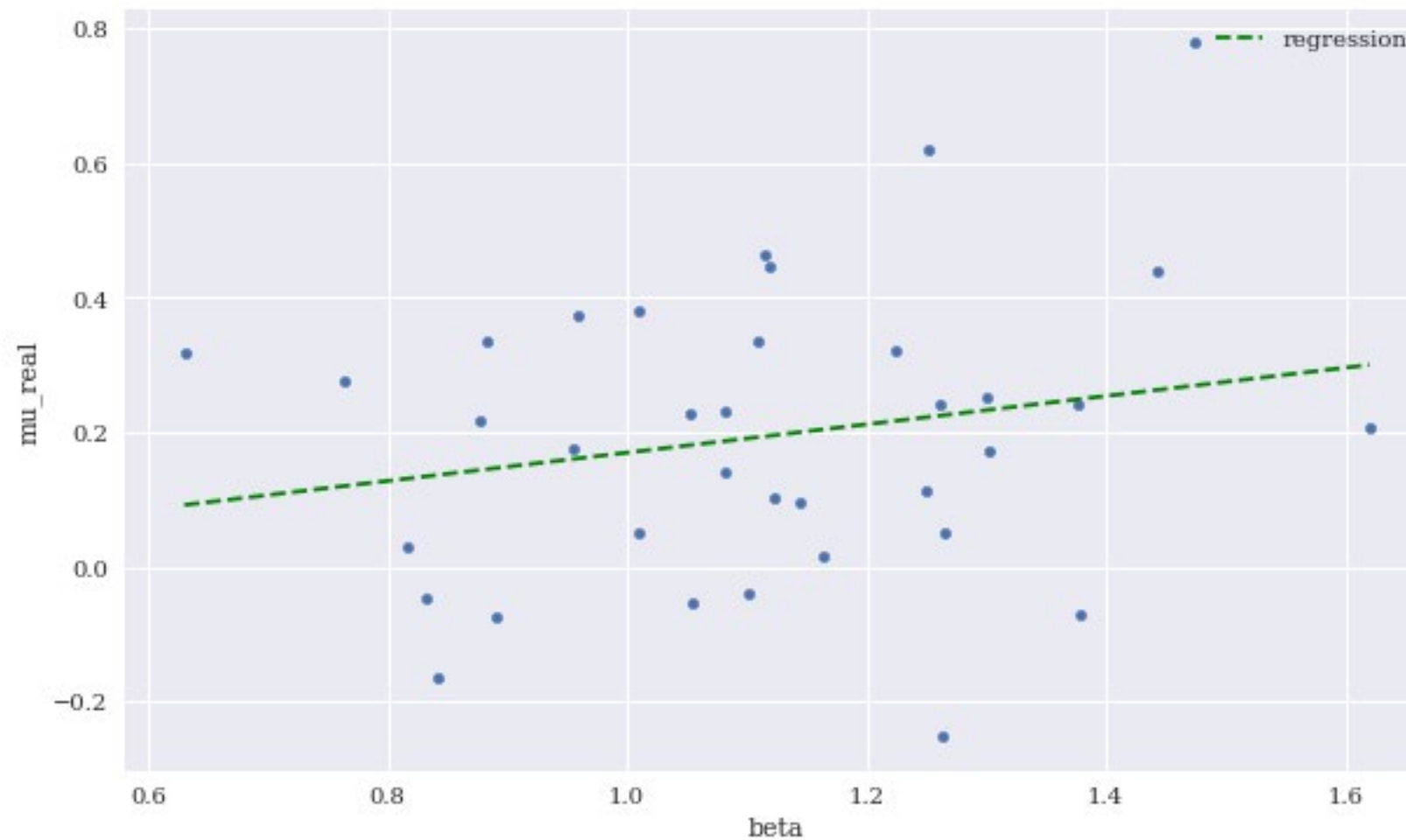
- Cả CAPM và APT đều liên hệ các biến đầu ra với các yếu tố đầu vào có liên quan theo kiểu tuyến tính.
- Từ quan điểm kinh tế lượng, cả hai mô hình đều được triển khai dựa trên hồi quy tuyến tính bình phương nhỏ nhất (OLS).
- CAPM: hồi quy OLS tuyến tính đơn biến
- APT: hồi quy OLS đa biến

Lợi nhuận CAPM dự kiến so với beta (bao gồm cả hồi quy tuyến tính)



Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

Lợi nhuận CAPM dự kiến so với beta (bao gồm cả hồi quy tuyến tính)



Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

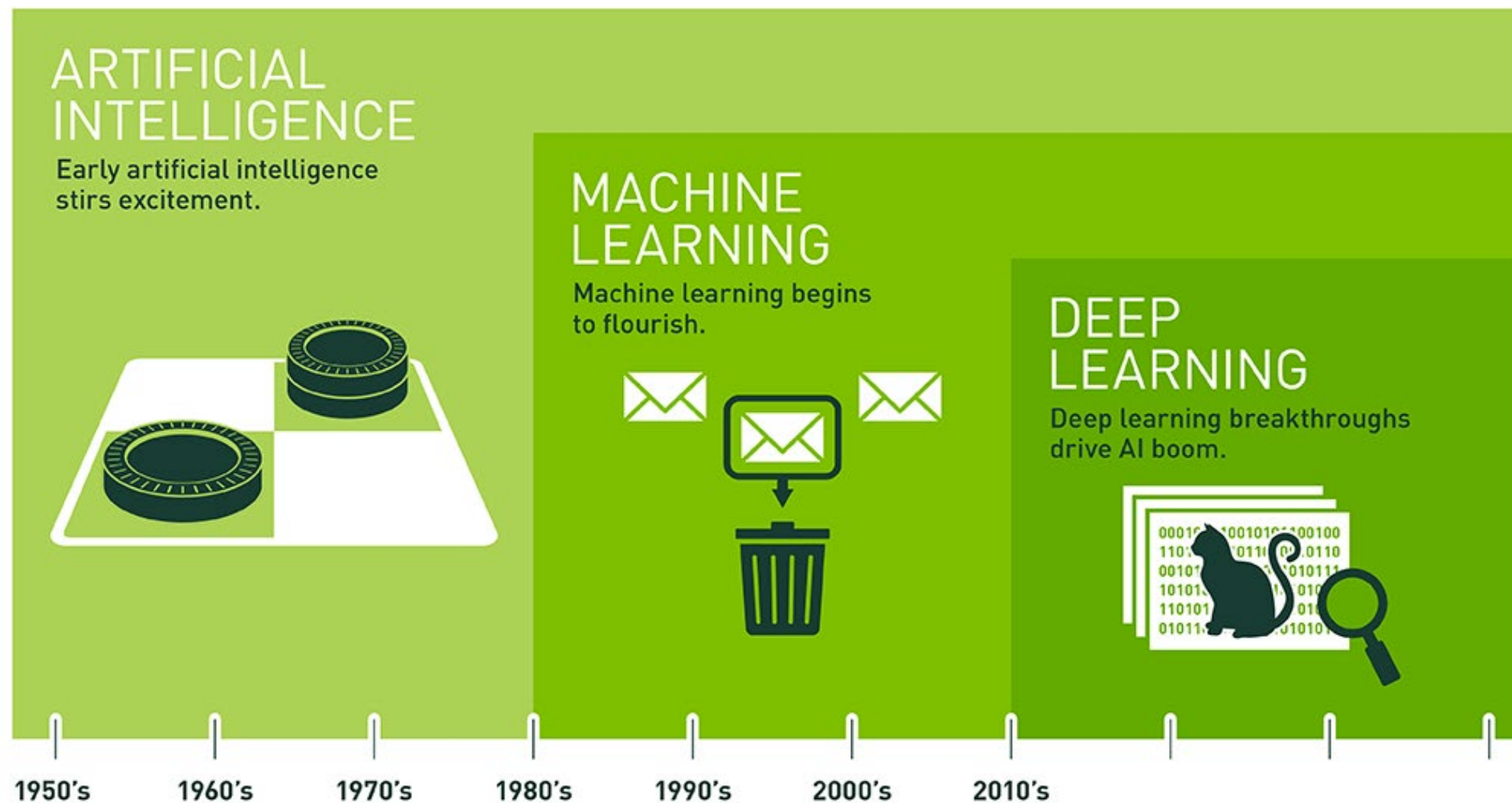
Học Máy

Học Máy

- Quá trình học
- Lượng giá
- Bias and variance
- Cross-validation

Artificial Intelligence

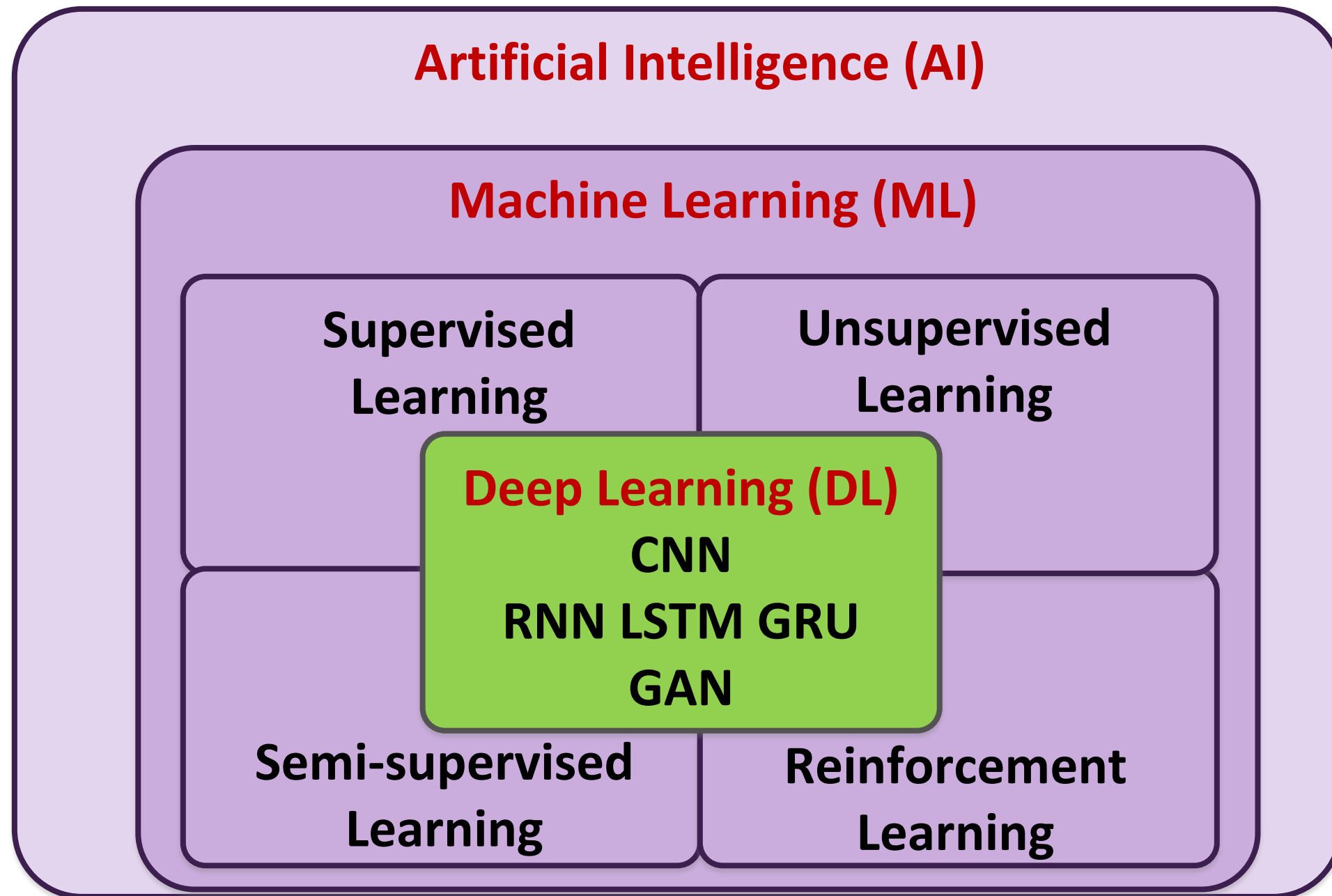
Machine Learning & Deep Learning



Since an early flush of optimism in the 1950s, smaller subsets of artificial intelligence – first machine learning, then deep learning, a subset of machine learning – have created ever larger disruptions.

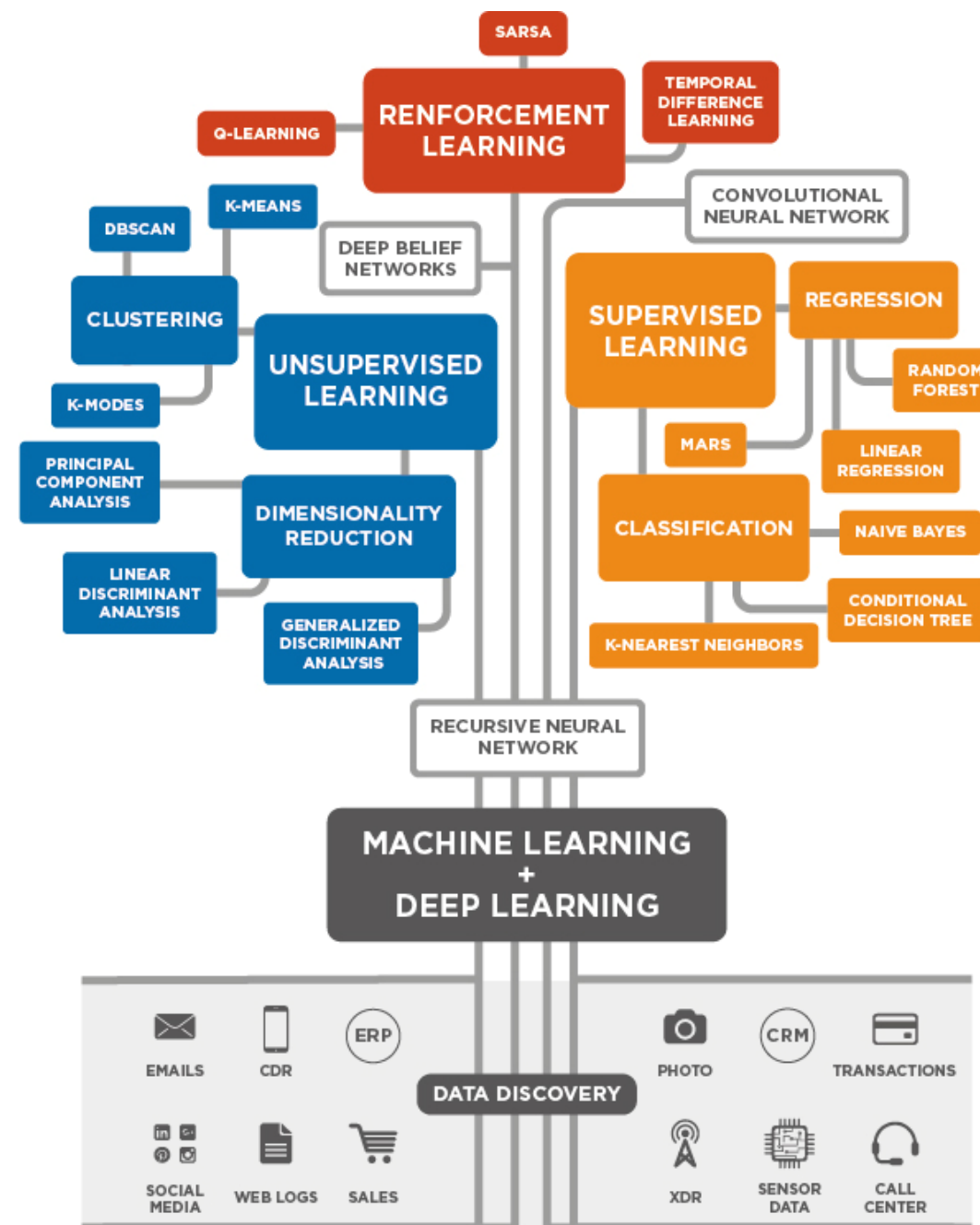
Source: <https://blogs.nvidia.com/blog/2016/07/29/whats-difference-artificial-intelligence-machine-learning-deep-learning-ai/>

AI, ML, DL



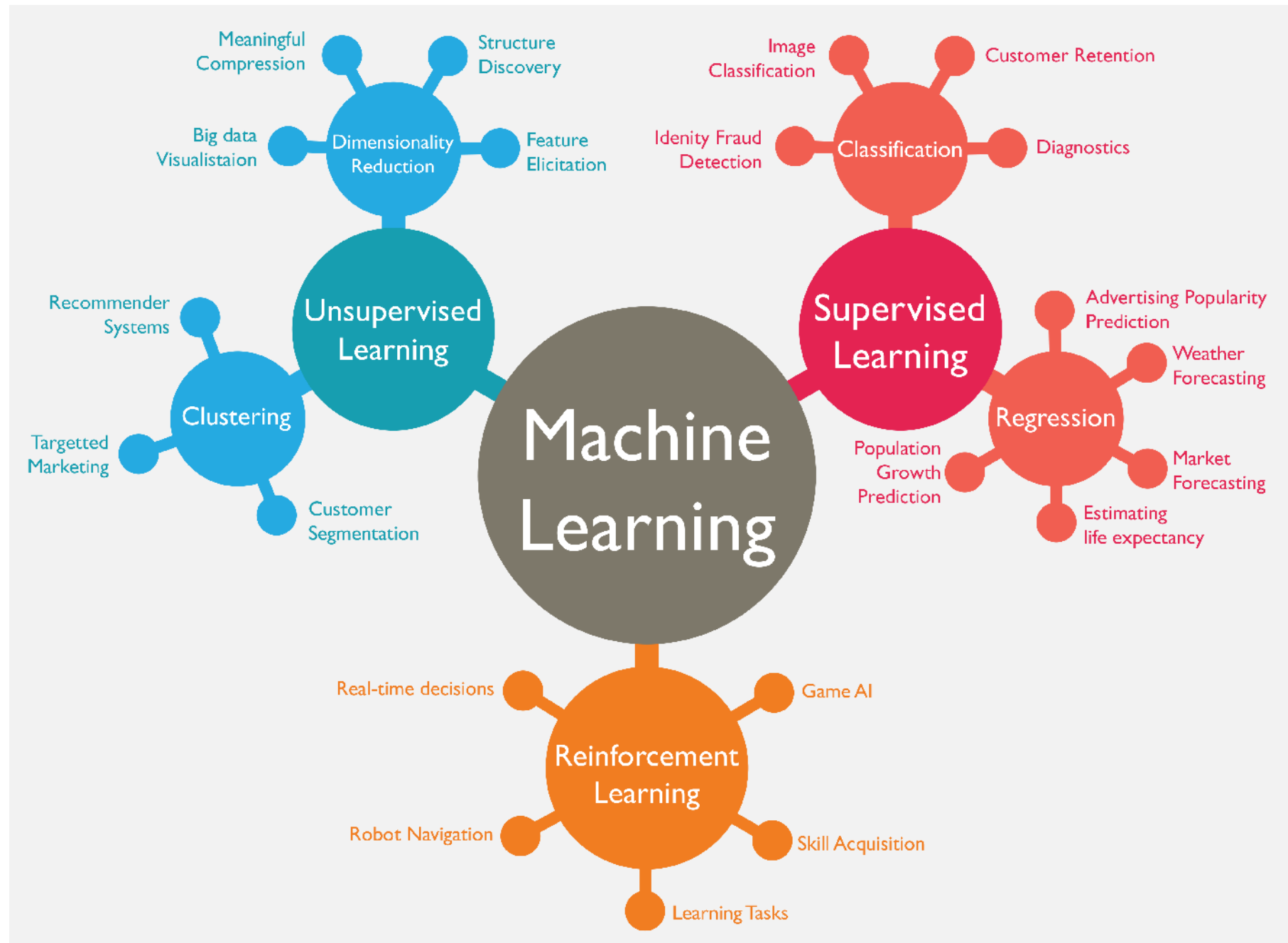
Source: https://leonardoaraujosantos.gitbooks.io/artificial-intelligence/content/deep_learning.html

3 Machine Learning Algorithms



Source: Enrico Galimberti, <http://blogs.teradata.com/data-points/tree-machine-learning-algorithms/>

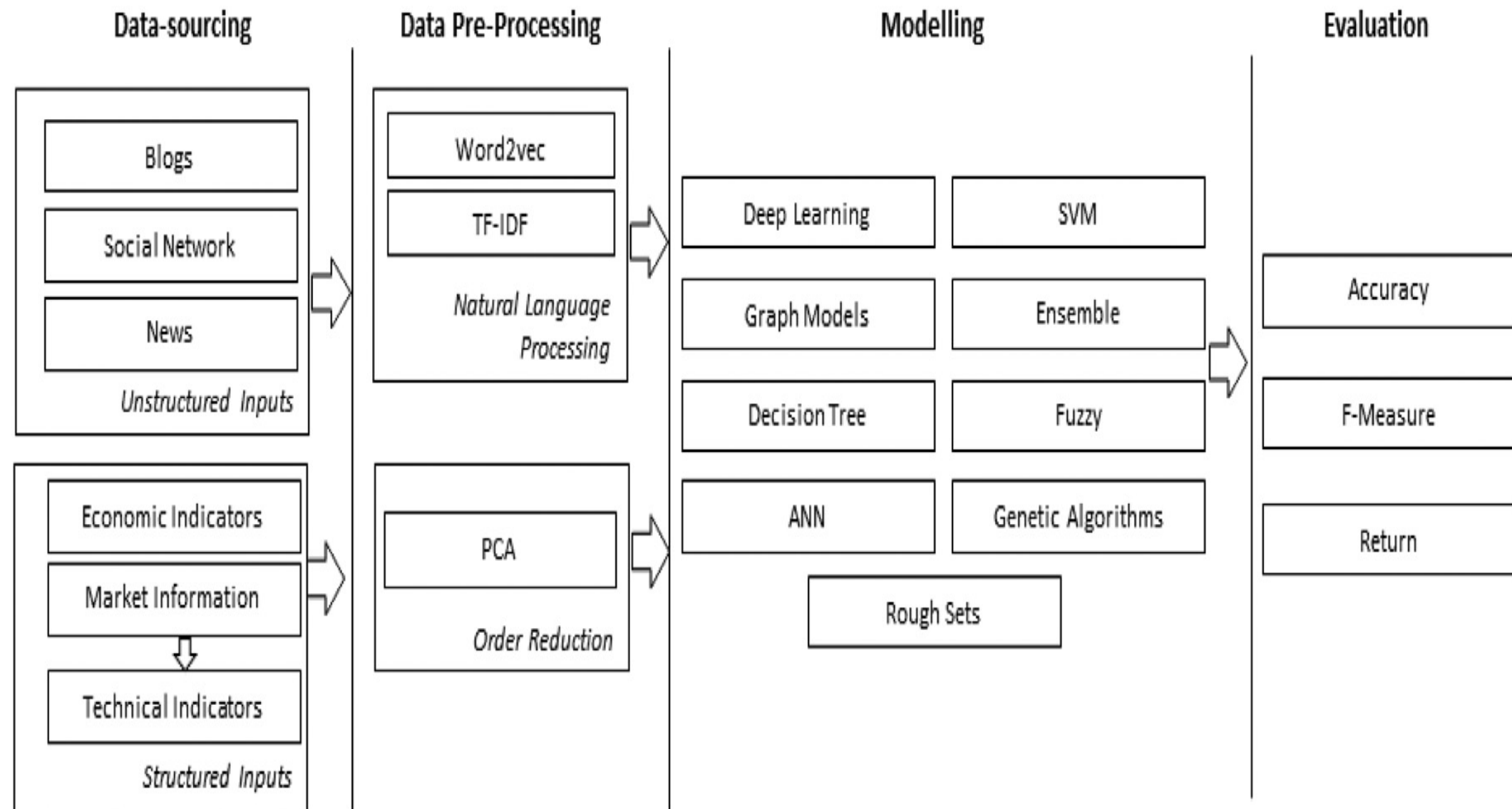
Machine Learning (ML)



Source: <https://www.mactores.com/services/aws-big-data-machine-learning-cognitive-services/>

Dự báo diễn biến thị trường chứng khoán:

ML Các giai đoạn của mô hình thị trường chứng khoán



Source: O. Bustos and A. Pomares-Quimbaya (2020), "Stock Market Movement Forecast: A Systematic Review." Expert Systems with Applications (2020): 113464.

Machine Learning

- Learning
- Data: Features, Labels
- Success (Loss Function): MSE
- Capacity (Model Fit)
- Evaluation
- Bias and variance
- Cross-validation

Learning (Mitchell, 1997)

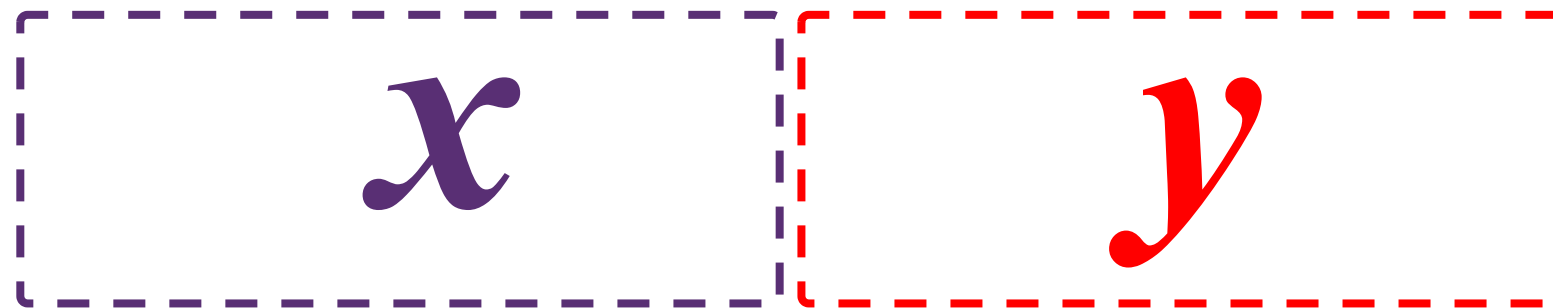
- A computer program is said to **learn** from **experience E** with respect to some class of tasks **T** and **performance measure P**, if its performance at tasks in **T**, as measured by **P**, improves with experience **E**.

Machine Learning

Supervised Learning (Classification)

Learning from Examples

$$y = f(x)$$

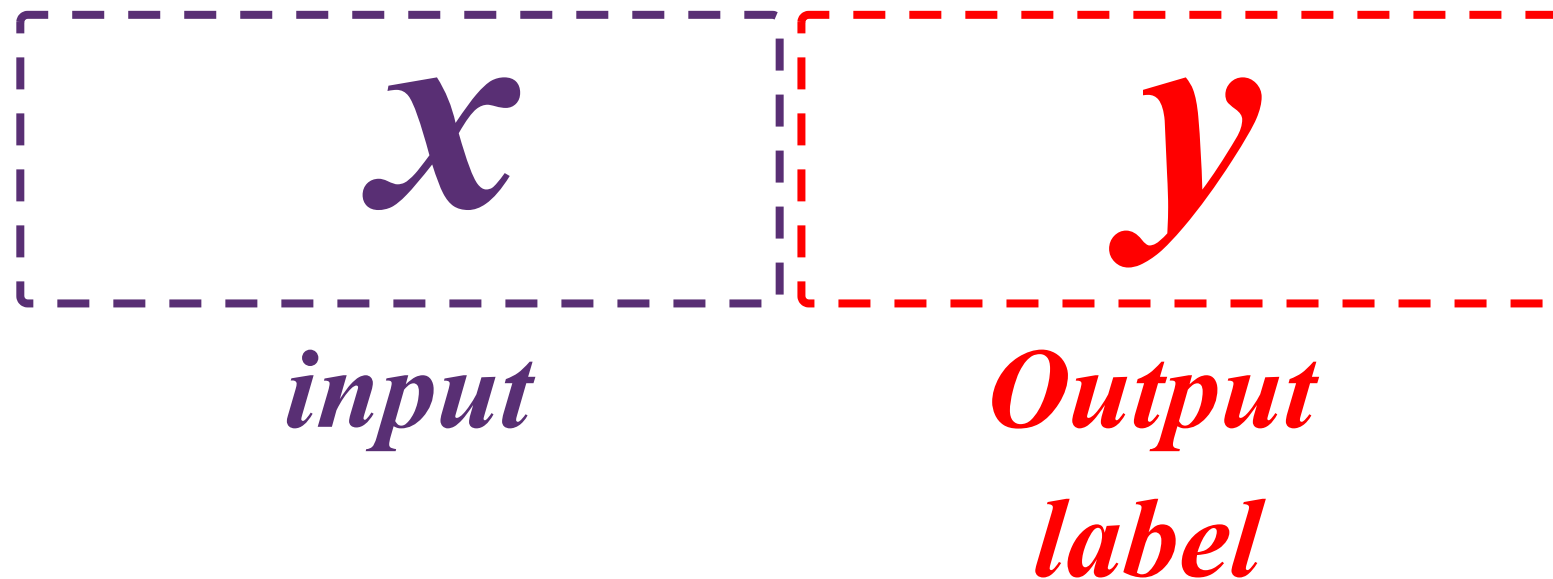


Machine Learning

Supervised Learning (Classification)

Learning from Examples

$$y = f(x)$$



Machine Learning

Supervised Learning (Classification)

Learning from Examples

$$y = f(x)$$

```
5.1, 3.5, 1.4, 0.2, Iris-setosa
4.9, 3.0, 1.4, 0.2, Iris-setosa
4.7, 3.2, 1.3, 0.2, Iris-setosa
7.0, 3.2, 4.7, 1.4, Iris-versicolor
6.4, 3.2, 4.5, 1.5, Iris-versicolor
6.9, 3.1, 4.9, 1.5, Iris-versicolor
6.3, 3.3, 6.0, 2.5, Iris-virginica
5.8, 2.7, 5.1, 1.9, Iris-virginica
7.1, 3.0, 5.9, 2.1, Iris-virginica
```

Machine Learning

Supervised Learning (Classification)

Learning from Examples

$$y = f(x)$$

<i>Example</i>	5.1, 3.5, 1.4, 0.2, Iris-setosa
	4.9, 3.0, 1.4, 0.2, Iris-setosa
	4.7, 3.2, 1.3, 0.2, Iris-setosa
	7.0, 3.2, 4.7, 1.4, Iris-versicolor
	6.4, 3.2, 4.5, 1.5, Iris-versicolor
	6.9, 3.1, 4.9, 1.5, Iris-versicolor
	6.3, 3.3, 6.0, 2.5, Iris-virginica
	5.8, 2.7, 5.1, 1.9, Iris-virginica
	7.1, 3.0, 5.9, 2.1, Iris-virginica

Machine Learning

Supervised Learning (Classification)

Learning from Examples

$$y = f(x)$$

Example

x

5.1, 3.5, 1.4, 0.2,	Iris-setosa
4.9, 3.0, 1.4, 0.2,	Iris-setosa
4.7, 3.2, 1.3, 0.2,	Iris-setosa
7.0, 3.2, 4.7, 1.4,	Iris-versicolor
6.4, 3.2, 4.5, 1.5,	Iris-versicolor
6.9, 3.1, 4.9, 1.5,	Iris-versicolor
6.3, 3.3, 6.0, 2.5,	Iris-virginica
5.8, 2.7, 5.1, 1.9,	Iris-virginica
7.1, 3.0, 5.9, 2.1,	Iris-virginica

y

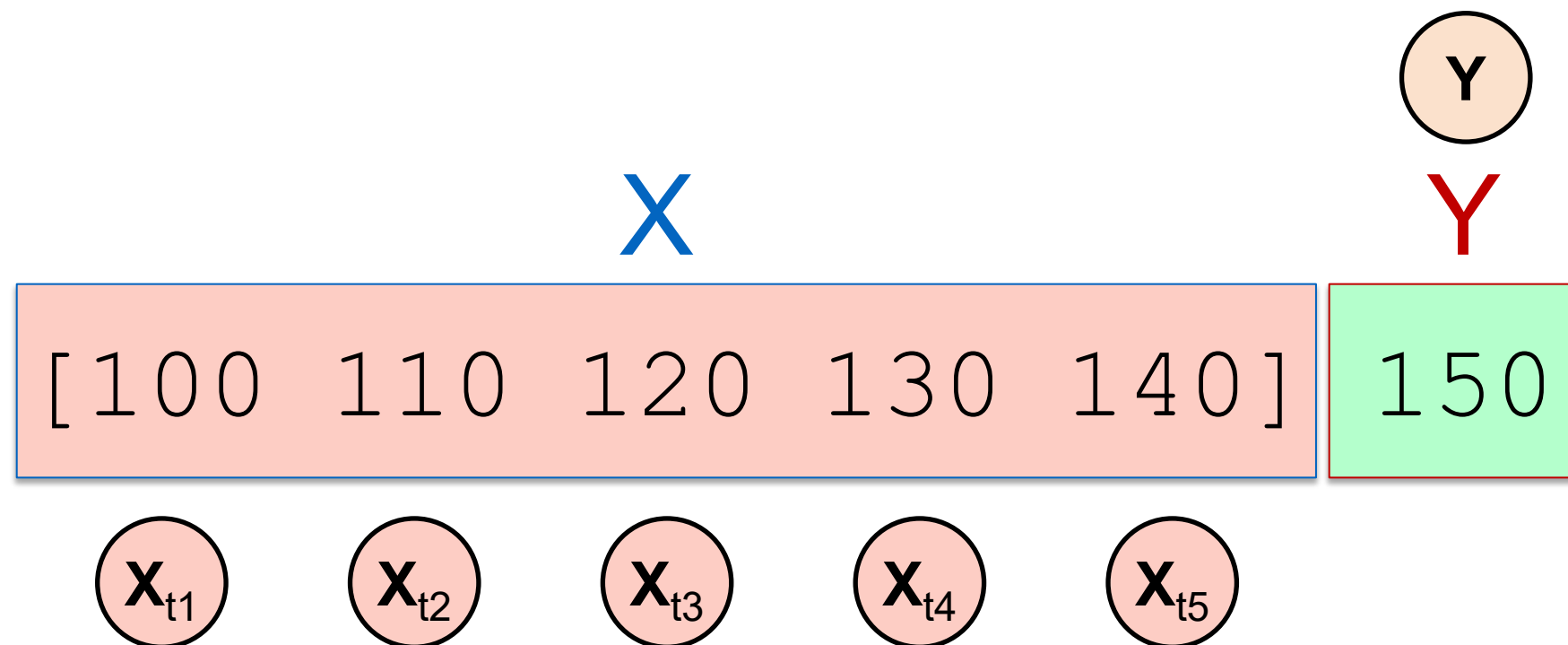
Time Series Data

[10, 20, 30, 40, 50, 60, 70, 80, 90]

X			Y
[10	20	30]	40
[20	30	40]	50
[30	40	50]	60
[40	50	60]	70
[50	60	70]	80
[60	70	80]	90

Time Series Data

[100, 110, 120, 130, 140, 150]



The Theory of Learning

- How can we be sure that our **learned hypothesis** will **predict** well for previously unseen inputs?
 - How do we know that the **hypothesis h** is close to the **target function f** if we don't know what is?
- How many **examples** do we need to get a good **h** ?
- What **hypothesis space** should we use?
- If the hypothesis space is very complex, can we even find the best **h** or do we have to settle for a **local maximum**?
- How **complex** should **h** be?
- How do we avoid **overfitting**?

Linear function

$$y = f(x)$$

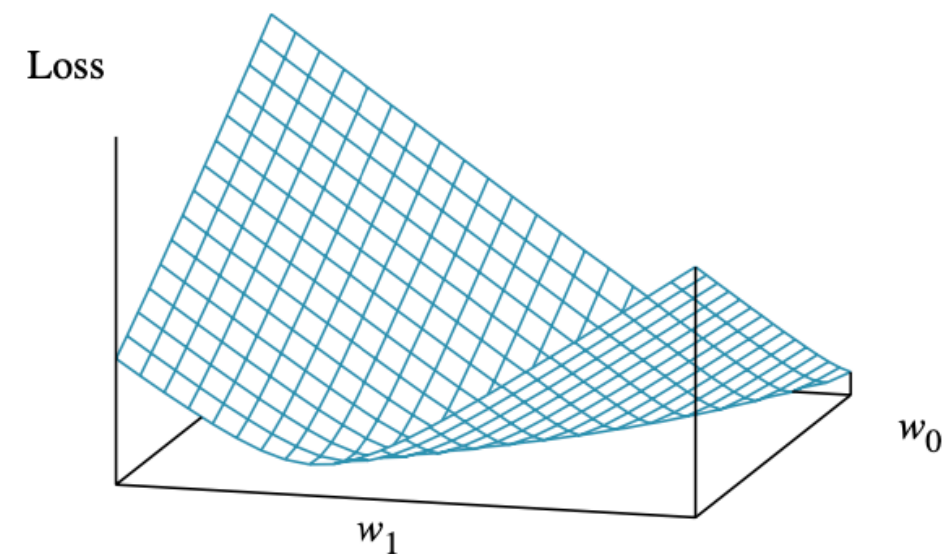
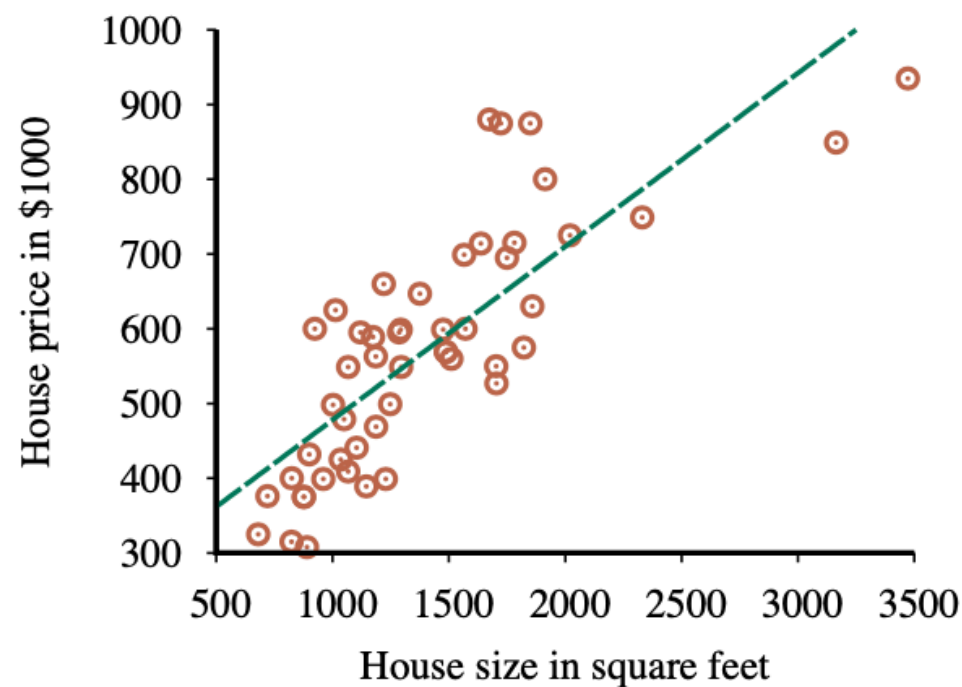
$$y = w_1 x + w_0$$

$$h_w(x) = w_1 x + w_0$$

Linear Regression Weight Space

$$h_w(x) = w_1 x + w_0$$

$$w^* = \operatorname{argmin}_w \operatorname{Loss}(h_w)$$



$$y = 0.232x + 246$$

Loss function for Weights (w_1, w_0)

Performance Measure

- The measure of success for estimation problems
 - mean-squared error (MSE)
- Classification problems
 - accuracy ratio

Evaluation

(Accuracy of Classification Model)

Assessing the Classification Model

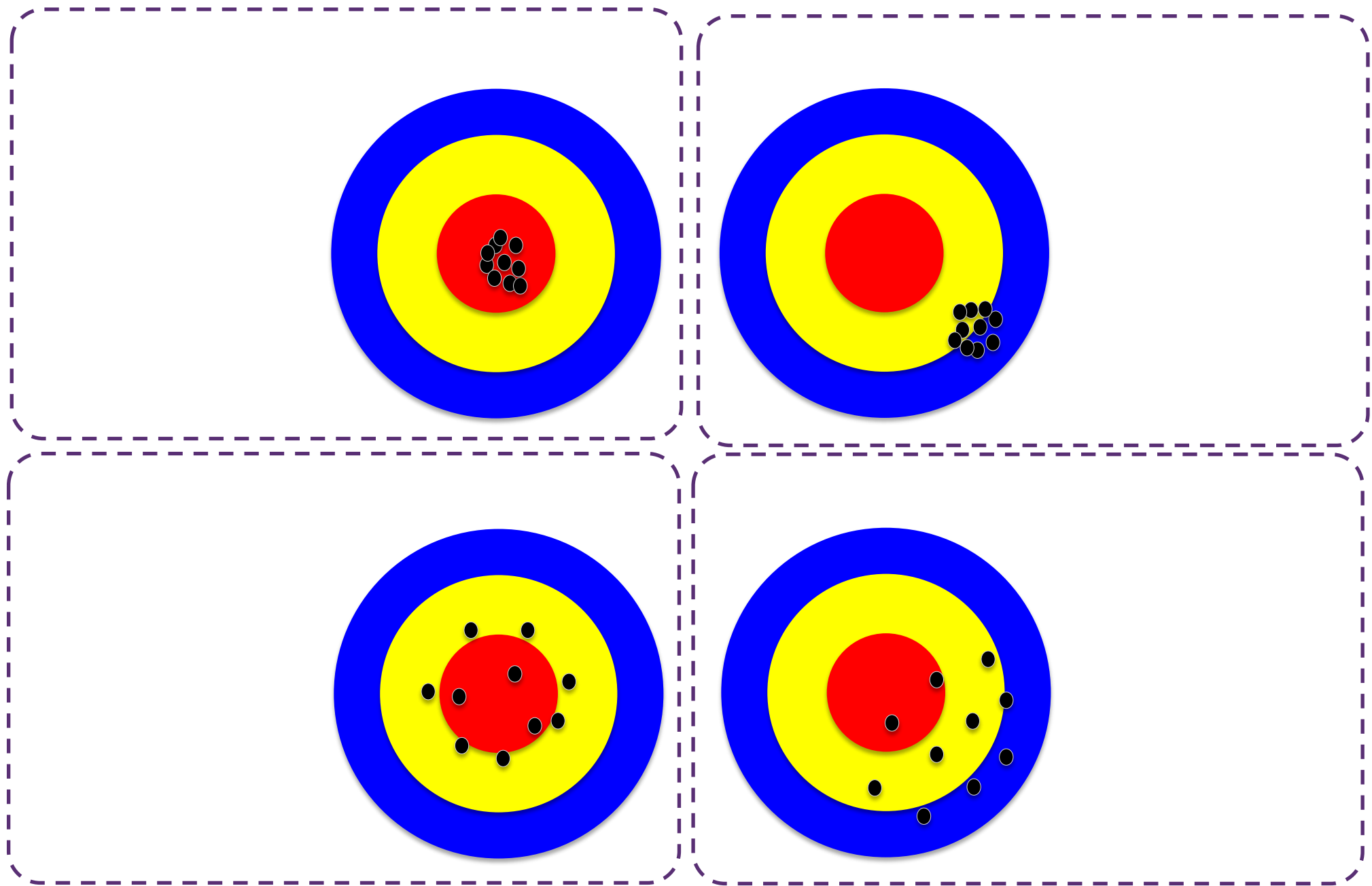
- Predictive accuracy
 - Hit rate
- Speed
 - Model building; predicting
- Robustness
- Scalability
- Interpretability
 - Transparency, explainability

Accuracy

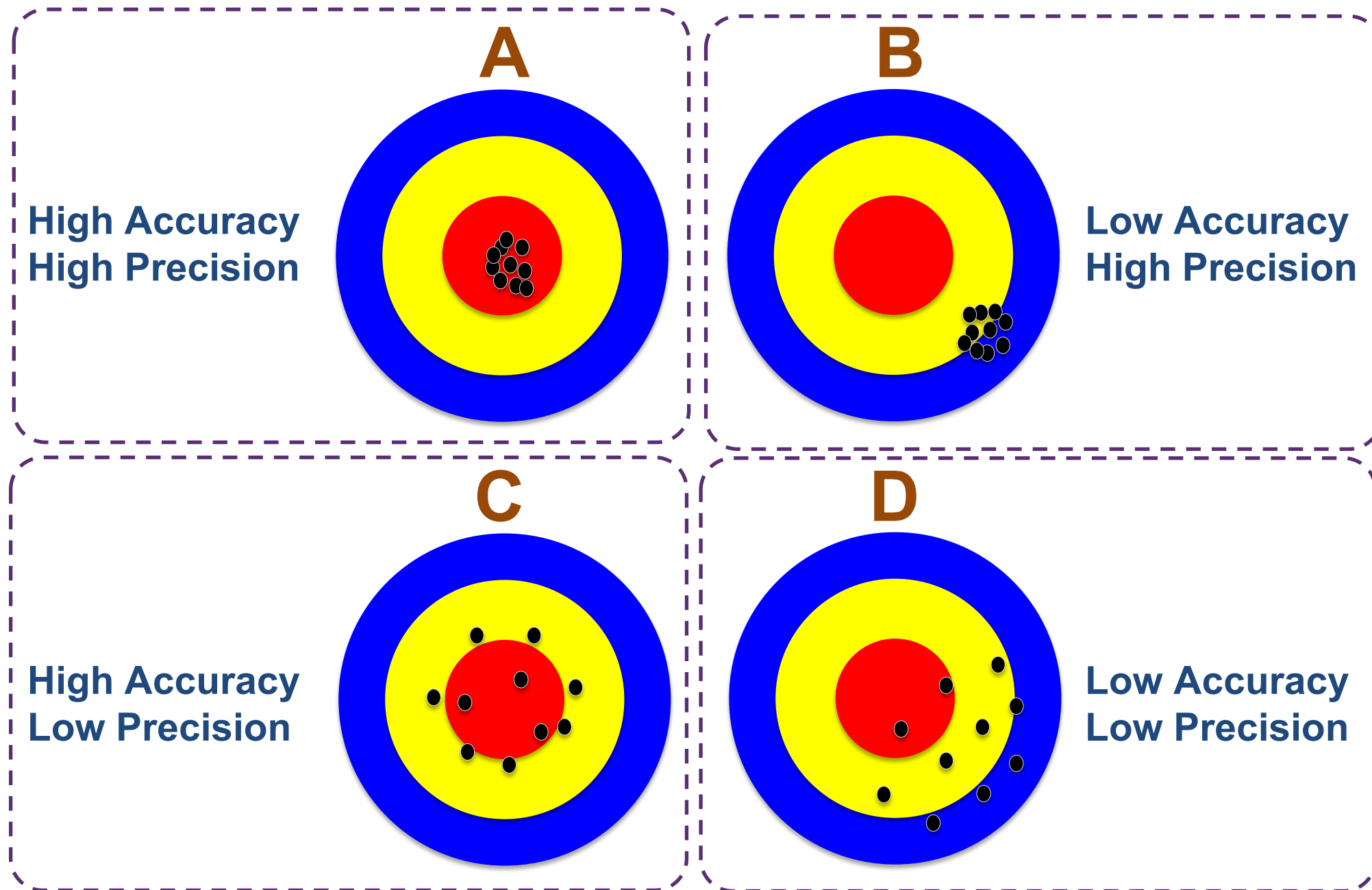
Validity

Precision

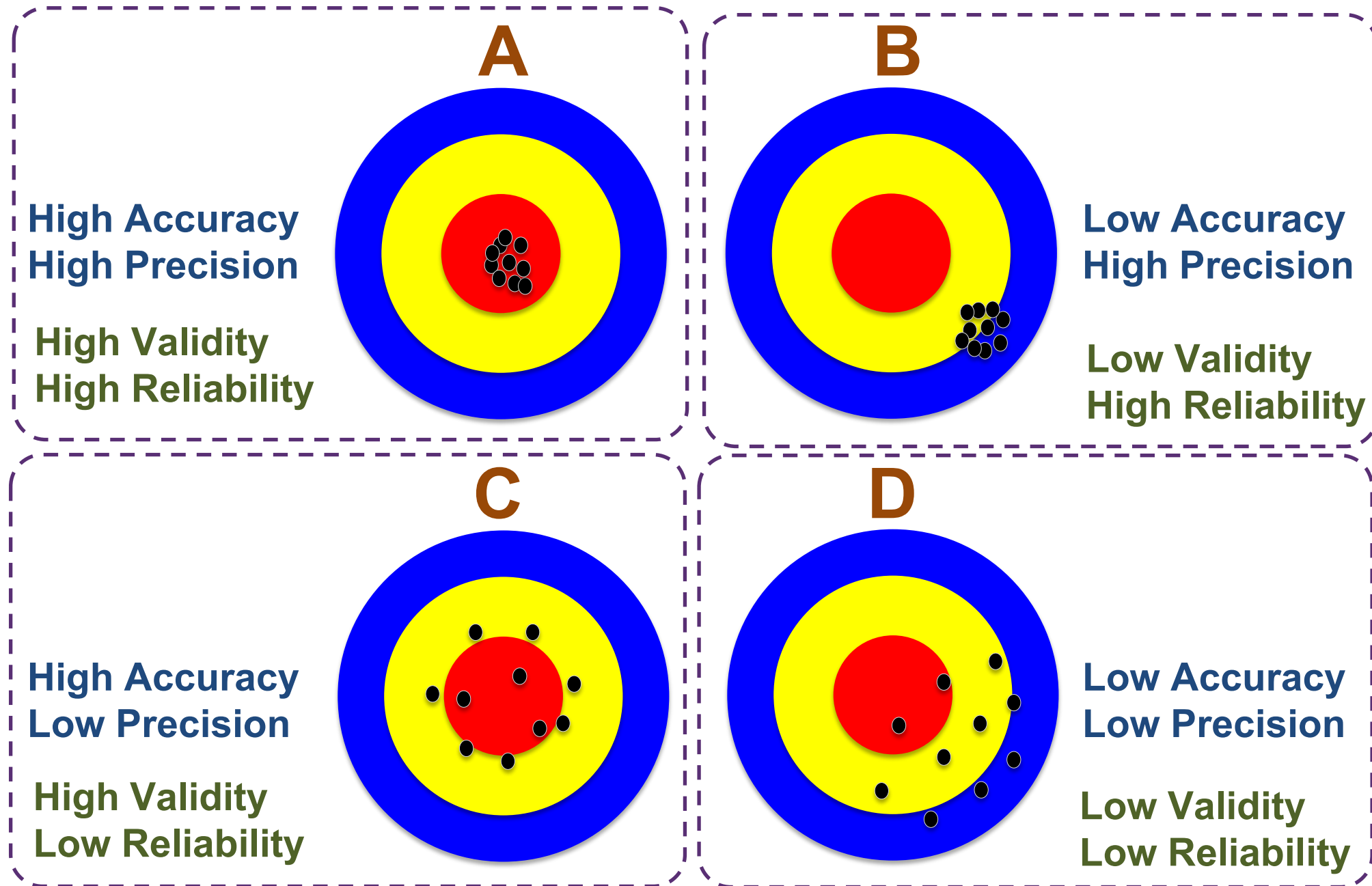
Reliability



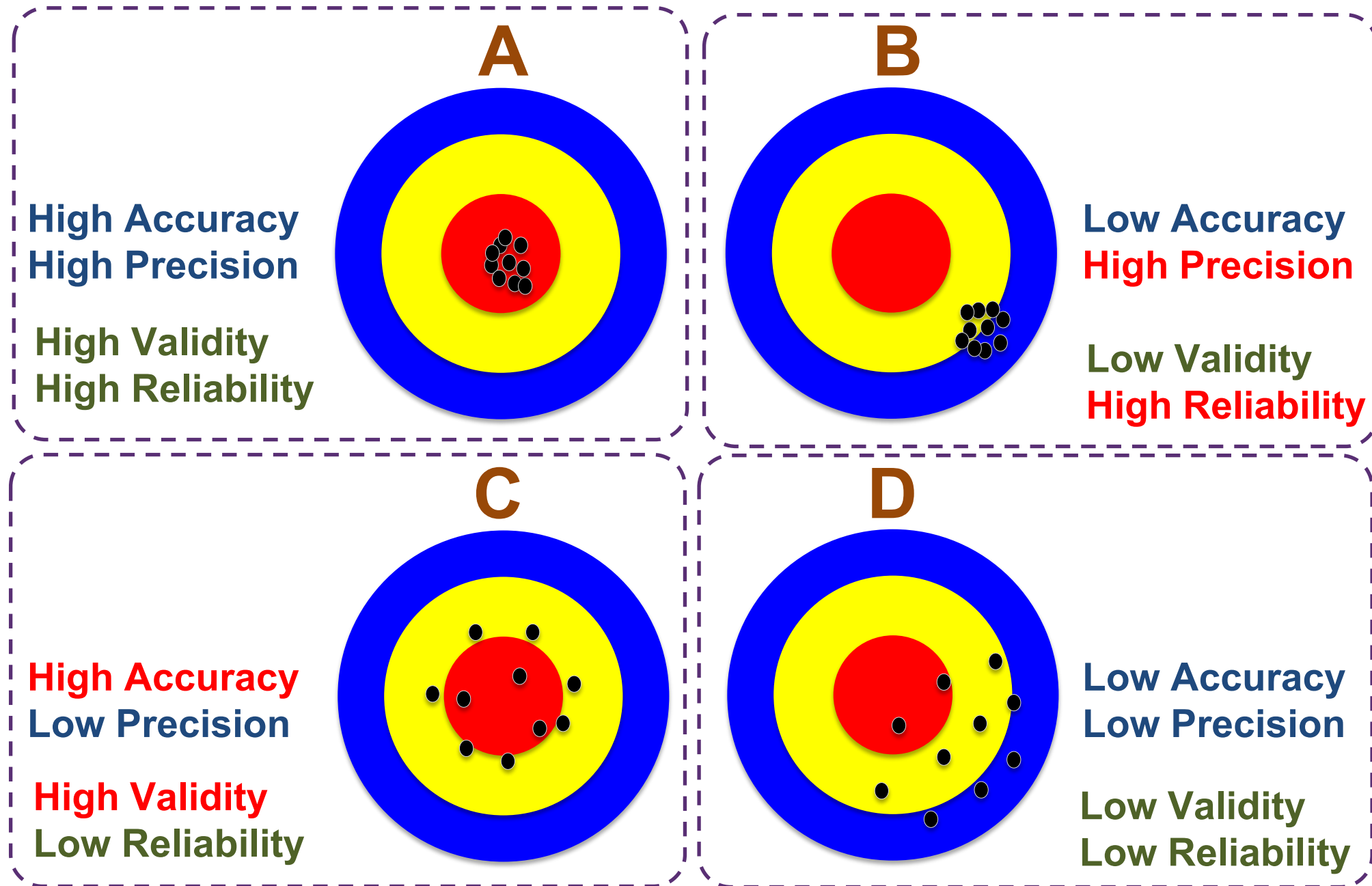
Accuracy vs. Precision



Accuracy vs. Precision



Accuracy vs. Precision



Confusion Matrix

for Tabulation of Two-Class Classification Results

		True/Observed Class			
		Positive	Negative		
Predicted Class	Positive	True Positive Count (TP)	False Positive Count (FP)	$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$	
	Negative	False Negative Count (FN)	True Negative Count (TN)	$True\ Positive\ Rate = \frac{TP}{TP + FN}$ $True\ Negative\ Rate = \frac{TN}{TN + FP}$ $Precision = \frac{TP}{TP + FP}$ $Recall = \frac{TP}{TP + FN}$	

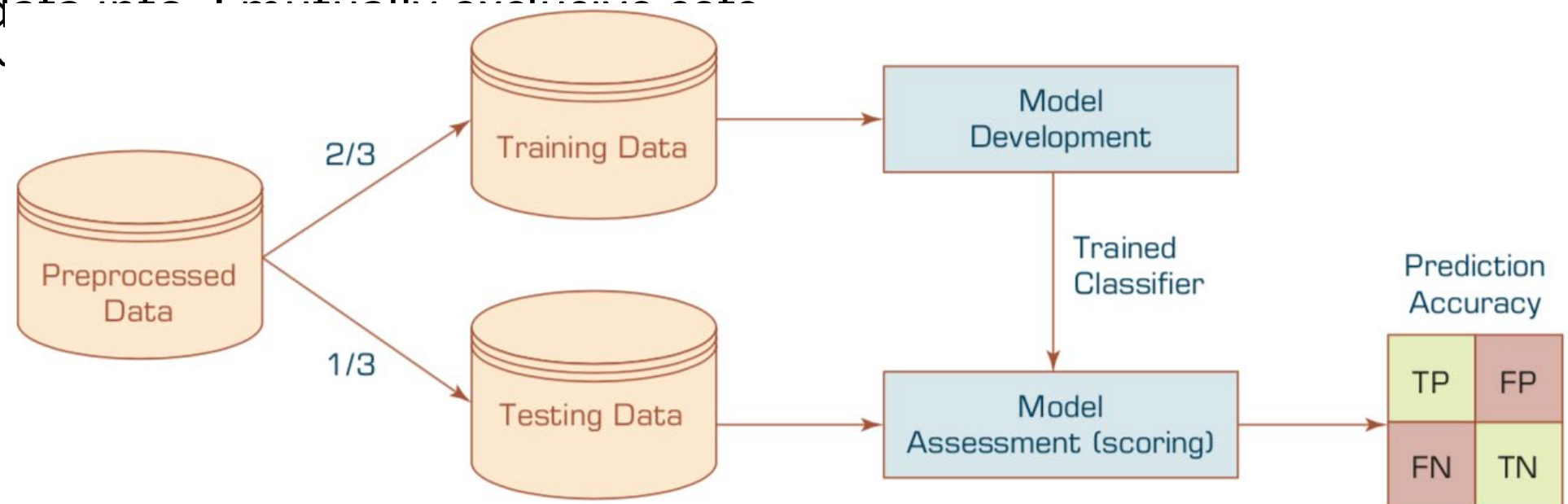
Sensitivity = True Positive Rate

Specificity = True Negative Rate

Estimation Methodologies for Classification

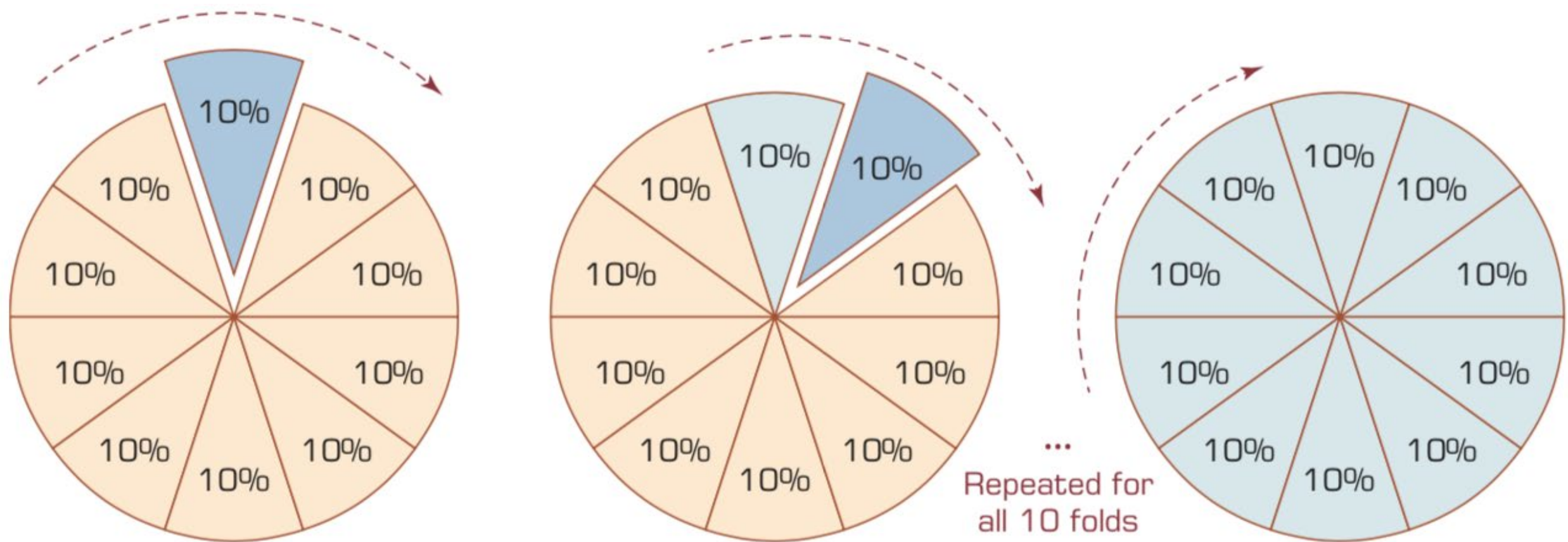
- **Simple split** (or holdout or test sample estimation)

- Split the data into 2 mutually exclusive sets:
 - training (~2/3)
 - testing (~1/3)

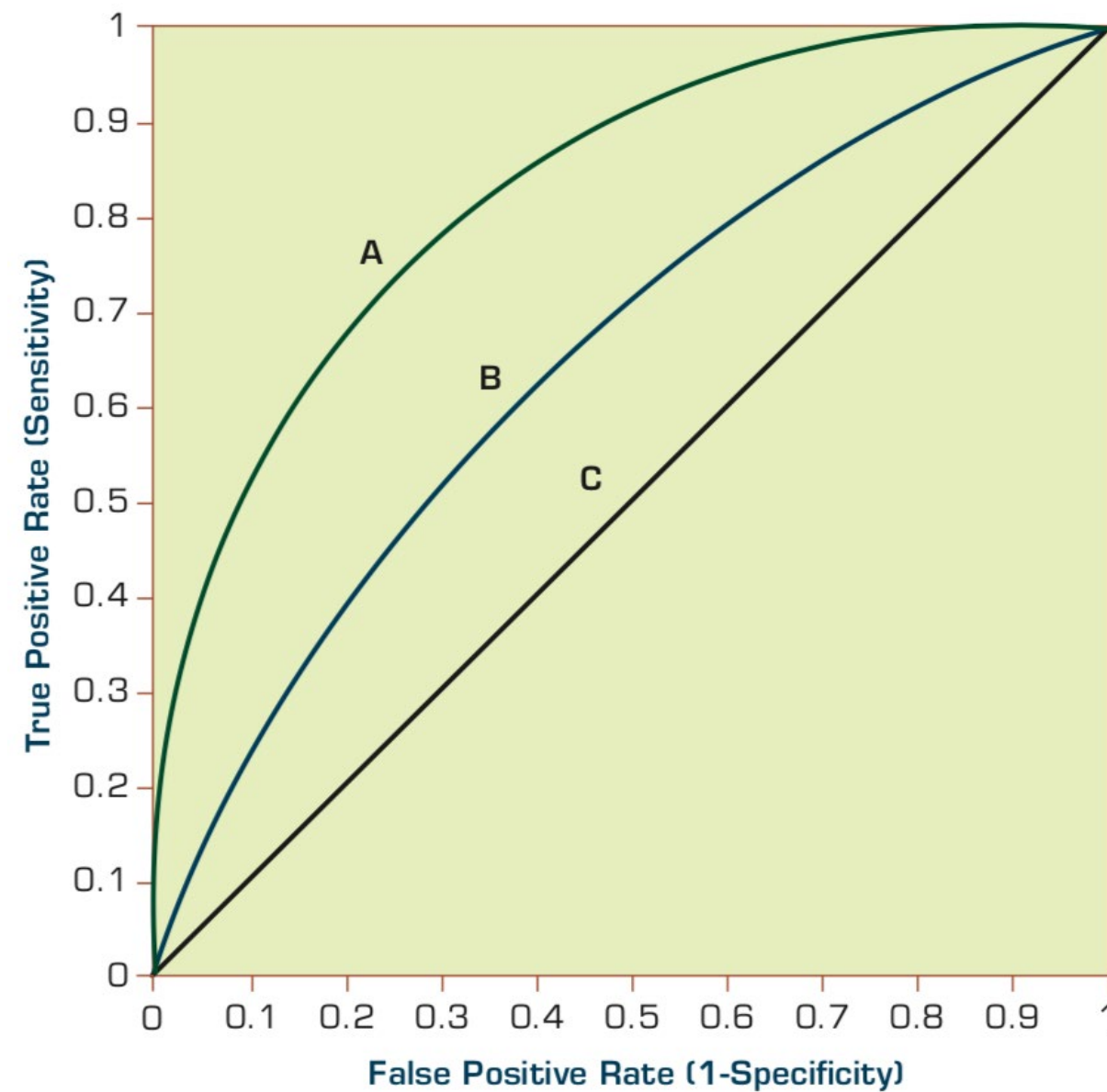


- For ANN, the data is split into three sub-sets (training [~60%], validation [~20%], testing [~20%])

k -Fold Cross-Validation



Estimation Methodologies for Classification Area under the ROC curve



Source: Ramesh Sharda, Dursun Delen, and Efraim Turban (2017), Business Intelligence, Analytics, and Data Science: A Managerial Perspective, 4th Edition, Pearson

		True Class (actual value)		total
		Positive	Negative	
Predictive Class (prediction outcome)	Positive	True Positive (TP)	False Positive (FP)	P'
	Negative	False Negative (FN)	True Negative (TN)	N'
total		P	N	

$$\text{True Positive Rate (Sensitivity)} = \frac{TP}{TP + FN}$$

$$\text{True Negative Rate (Specificity)} = \frac{TN}{TN + FP}$$

$$\text{False Positive Rate} = \frac{FP}{FP + TN}$$

$$\text{False Positive Rate (1 - Specificity)} = \frac{FP}{FP + TN}$$

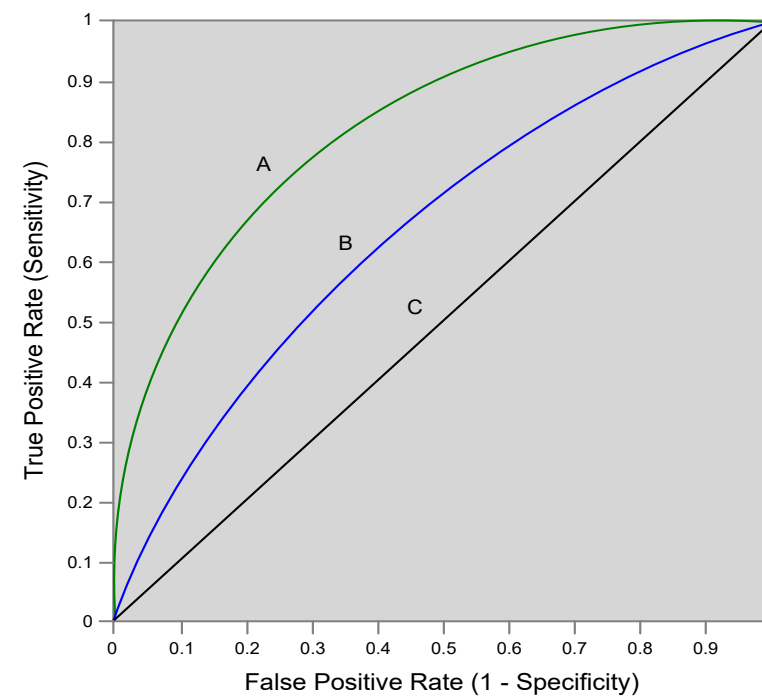
$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{True Positive Rate} = \frac{TP}{TP + FN}$$

$$\text{True Negative Rate} = \frac{TN}{TN + FP}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$



Source: http://en.wikipedia.org/wiki/Receiver_operating_characteristic

		True Class (actual value)		total
		Positive	Negative	
Predictive Class (prediction outcome)	Positive	True Positive (TP)	False Positive (FP)	P'
	Negative	False Negative (FN)	True Negative (TN)	N'
total		P	N	

$$\text{True Positive Rate (Sensitivity)} = \frac{TP}{TP + FN}$$

Sensitivity

= True Positive Rate

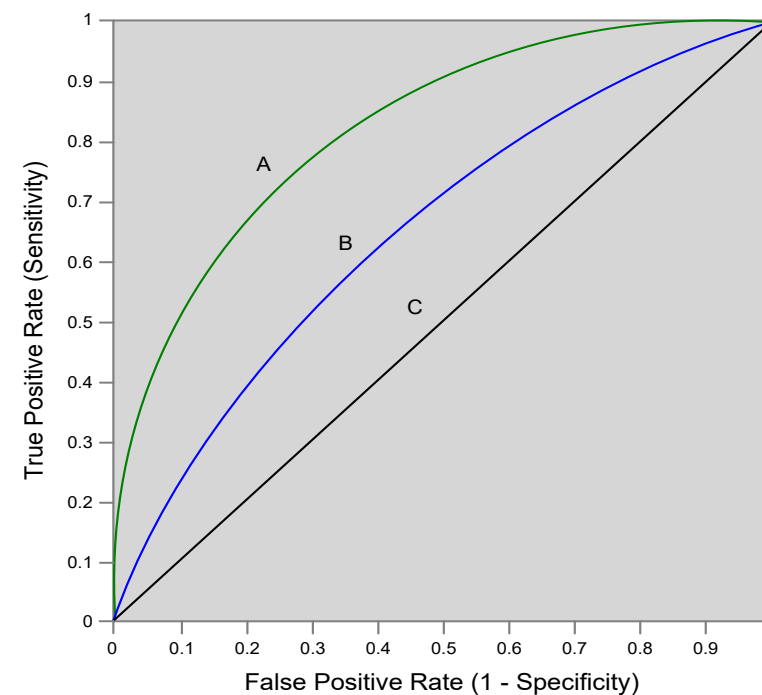
= Recall

= Hit rate

= $TP / (TP + FN)$

$$\text{True Positive Rate} = \frac{TP}{TP + FN}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$



Source: http://en.wikipedia.org/wiki/Receiver_operating_characteristic

		True Class (actual value)		total
		Positive	Negative	
Predictive Class (prediction outcome)	Positive	True Positive (TP)	False Positive (FP)	P'
	Negative	False Negative (FN)	True Negative (TN)	N'
total		P	N	

Specificity

= True Negative Rate

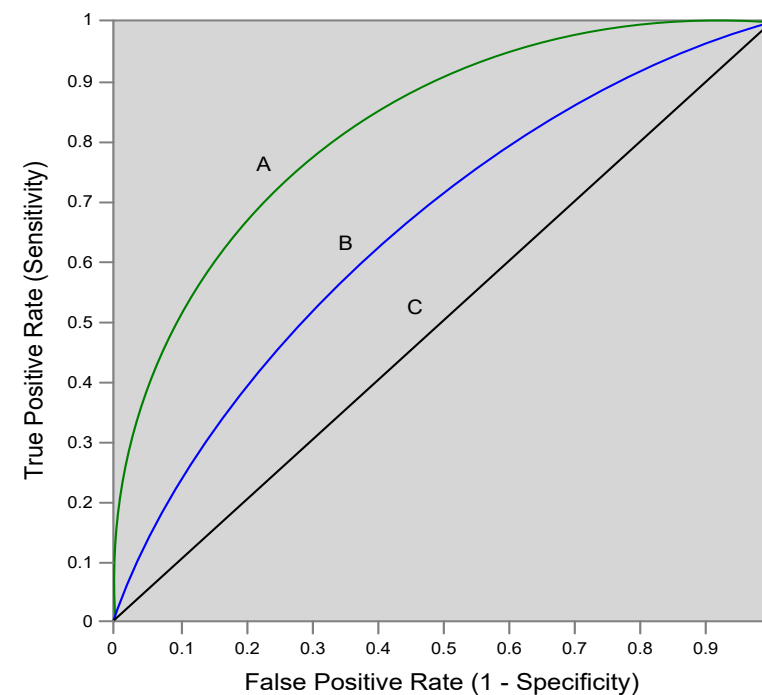
= TN / N

= $TN / (TN + FP)$

$$\text{True Negative Rate (Specificity)} = \frac{TN}{TN + FP}$$

$$\text{False Positive Rate (1 - Specificity)} = \frac{FP}{FP + TN}$$

$$\text{True Negative Rate} = \frac{TN}{TN + FP}$$



Source: http://en.wikipedia.org/wiki/Receiver_operating_characteristic

		True Class (actual value)		total
		Positive	Negative	
Predictive Class (prediction outcome)	Positive	True Positive (TP)	False Positive (FP)	P'
	Negative	False Negative (FN)	True Negative (TN)	N'
total		P	N	

Precision

= Positive Predictive Value (PPV)

$$Precision = \frac{TP}{TP + FP}$$

Recall

= True Positive Rate (TPR)

= Sensitivity

= Hit Rate

$$Recall = \frac{TP}{TP + FN}$$

F1 score (F-score)(F-measure)

is the harmonic mean of precision and recall

$$= 2TP / (P + P')$$

$$= 2TP / (2TP + FP + FN)$$

$$F = 2 * \frac{precision * recall}{precision + recall}$$

Source: http://en.wikipedia.org/wiki/Receiver_operating_characteristic

A

63 (TP)	28 (FP)	91
37 (FN)	72 (TN)	109
100	100	200

Recall

= True Positive Rate (TPR)
 = Sensitivity
 = Hit Rate
 = $TP / (TP + FN)$

Specificity

= True Negative Rate
 = TN / N
 = $TN / (TN + FP)$

$$TPR = 0.63$$

$$Recall = \frac{TP}{TP + FN}$$

$$True\ Negative\ Rate\ (Specificity) = \frac{TN}{TN + FP}$$

$$FPR = 0.28$$

$$False\ Positive\ Rate\ (1 - Specificity) = \frac{FP}{FP + TN}$$

$$PPV = 0.69$$

$$= 63 / (63 + 28)$$

$$= 63 / 91$$

$$Precision = \frac{TP}{TP + FP}$$

Precision

= Positive Predictive Value (PPV)

$$F1 = 0.66$$

$$= 2 * (0.63 * 0.69) / (0.63 + 0.69)$$

$$= (2 * 63) / (100 + 91)$$

$$= (0.63 + 0.69) / 2 = 1.32 / 2 = 0.66$$

$$F = 2 * \frac{precision * recall}{precision + recall}$$

F1 score (F-score) (F-measure)

is the harmonic mean of precision and recall

$$= 2TP / (P + P')$$

$$= 2TP / (2TP + FP + FN)$$

$$ACC = 0.68$$

$$= (63 + 72) / 200$$

$$= 135 / 200 = 67.5$$

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Source: http://en.wikipedia.org/wiki/Receiver_operating_characteristic

A

63 (TP)	28 (FP)	91
37 (FN)	72 (TN)	109
100	100	200

$$\text{TPR} = 0.63$$

$$\text{FPR} = 0.28$$

$$\text{PPV} = 0.69$$

$$= 63 / (63 + 28)$$

$$= 63 / 91$$

$$\text{F1} = 0.66$$

$$= 2 * (0.63 * 0.69) / (0.63 + 0.69)$$

$$= (2 * 63) / (100 + 91)$$

$$= (0.63 + 0.69) / 2 = 1.32 / 2 = 0.66$$

$$\text{ACC} = 0.68$$

$$= (63 + 72) / 200$$

$$= 135 / 200 = 67.5$$

B

77 (TP)	77 (FP)	154
23 (FN)	23 (TN)	46
100	100	200

$$\text{TPR} = 0.77$$

$$\text{FPR} = 0.77$$

$$\text{PPV} = 0.50$$

$$\text{F1} = 0.61$$

$$\text{ACC} = 0.50$$

Recall

= True Positive Rate (TPR)

= Sensitivity

= Hit Rate

$$\text{Recall} = \frac{TP}{TP + FN}$$

Precision

= Positive Predictive Value (PPV)

$$\text{Precision} = \frac{TP}{TP + FP}$$

C

24 (TP)	88 (FP)	112
76 (FN)	12 (TN)	88
100	100	200

TPR = 0.24
 FPR = 0.88
 PPV = 0.21
 F1 = 0.22
 ACC = 0.18

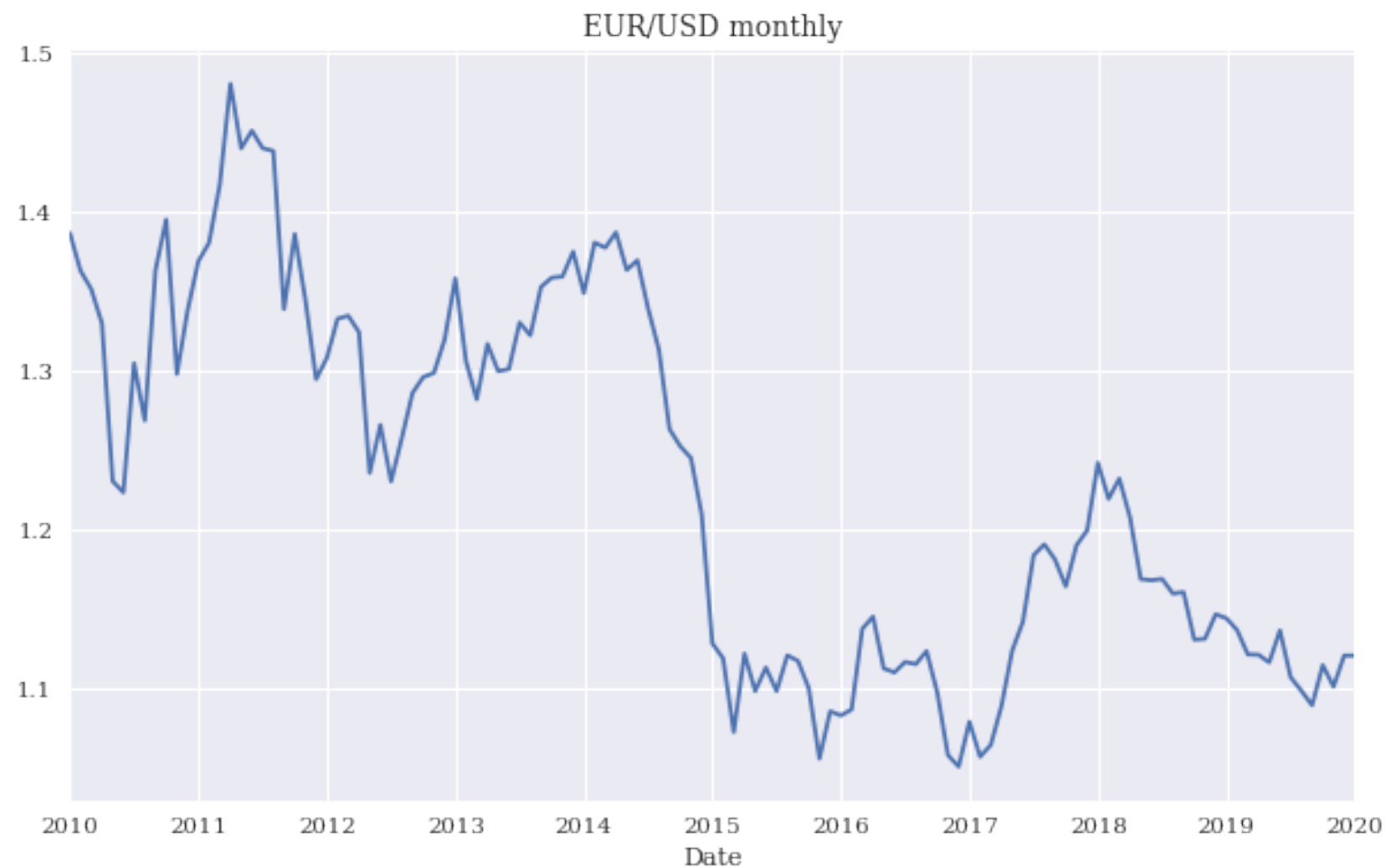
C'

76 (TP)	12 (FP)	88
24 (FN)	88 (TN)	112
100	100	200

TPR = 0.76
 FPR = 0.12
 PPV = 0.86
 F1 = 0.81
 ACC = 0.82

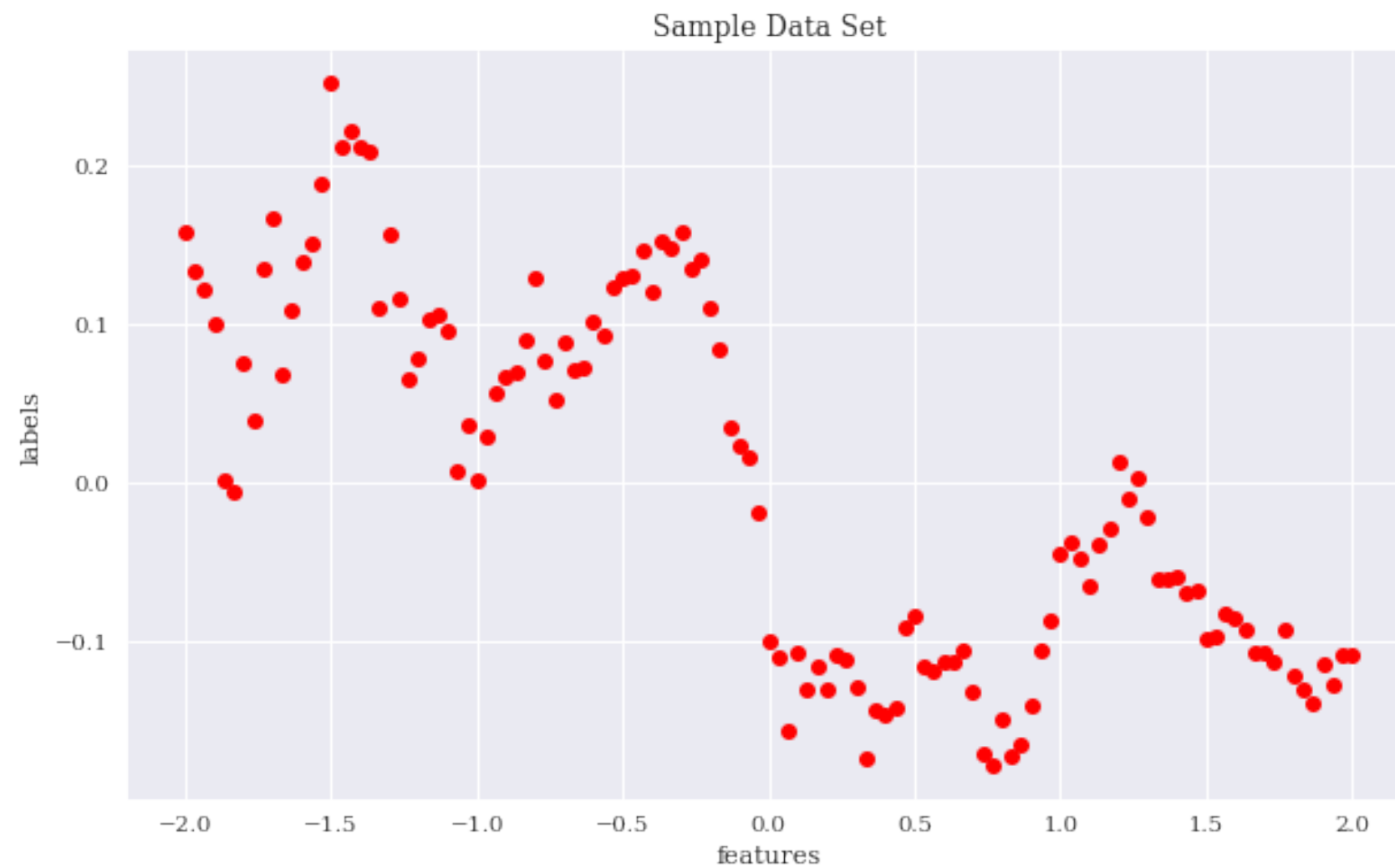
Recall	$Recall = \frac{TP}{TP + FN}$
= True Positive Rate (TPR)	
= Sensitivity	
= Hit Rate	
Precision	$Precision = \frac{TP}{TP + FP}$
= Positive Predictive Value (PPV)	

EUR/USD exchange rate as time series (monthly)



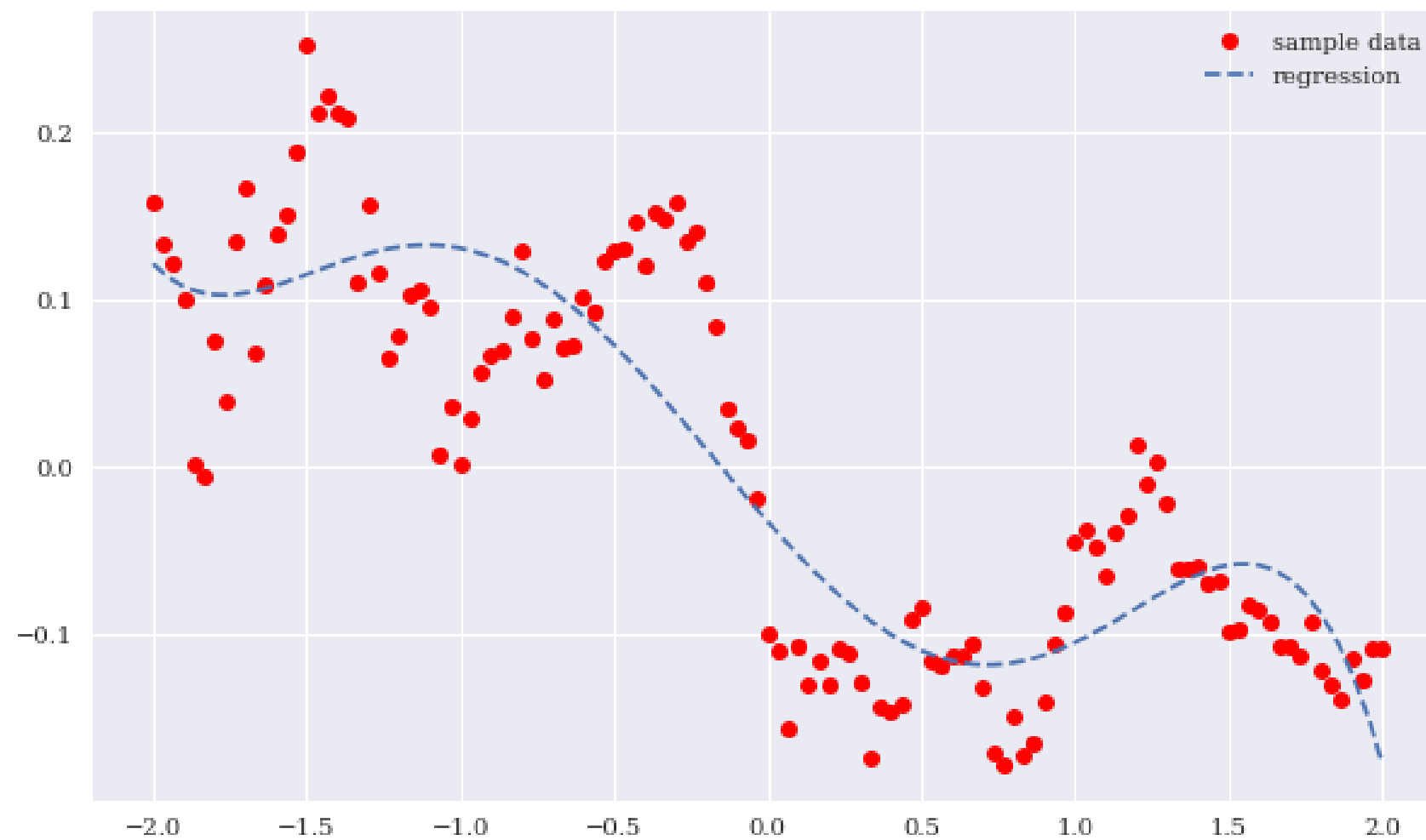
Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

Sample data set



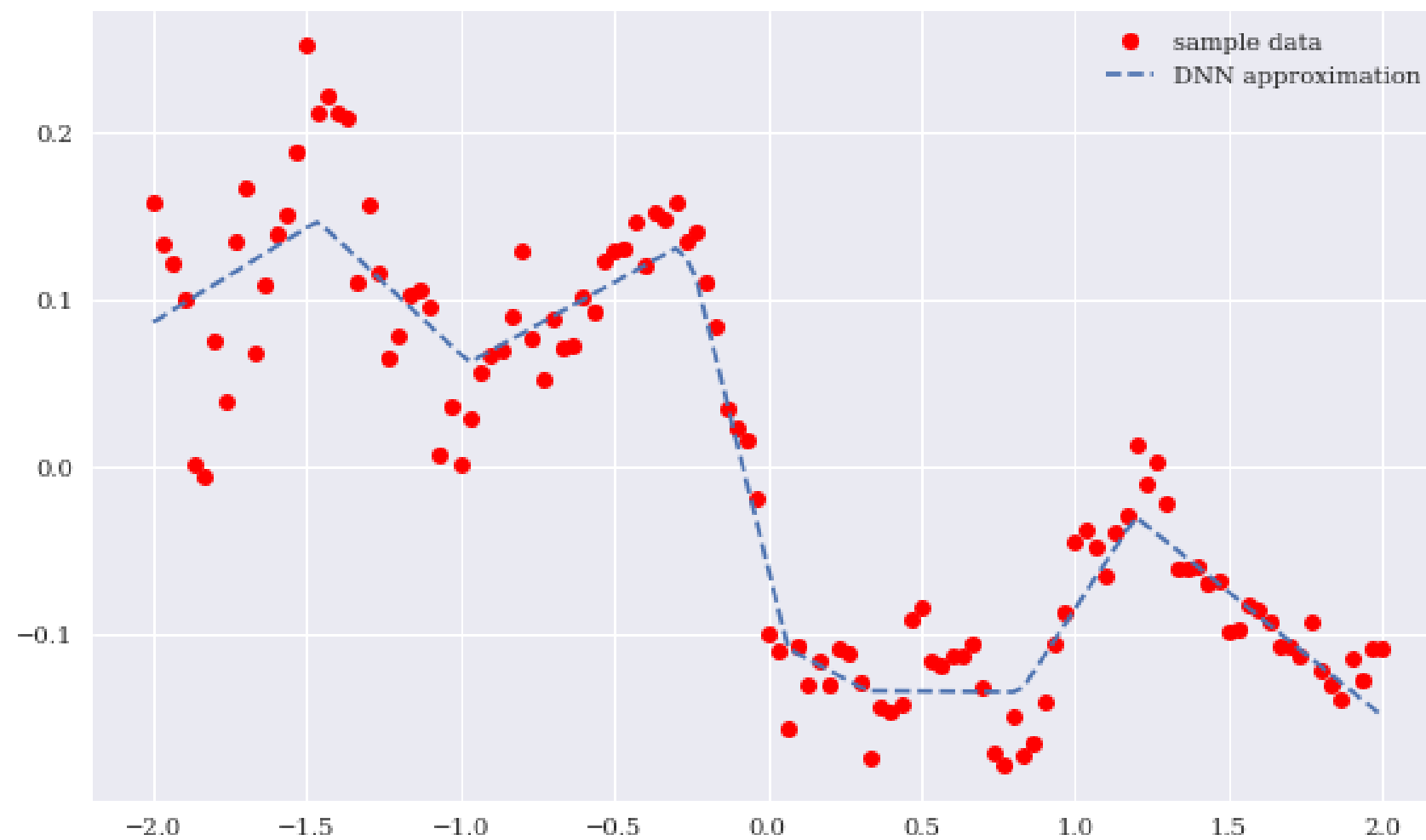
Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

Sample data and cubic regression line



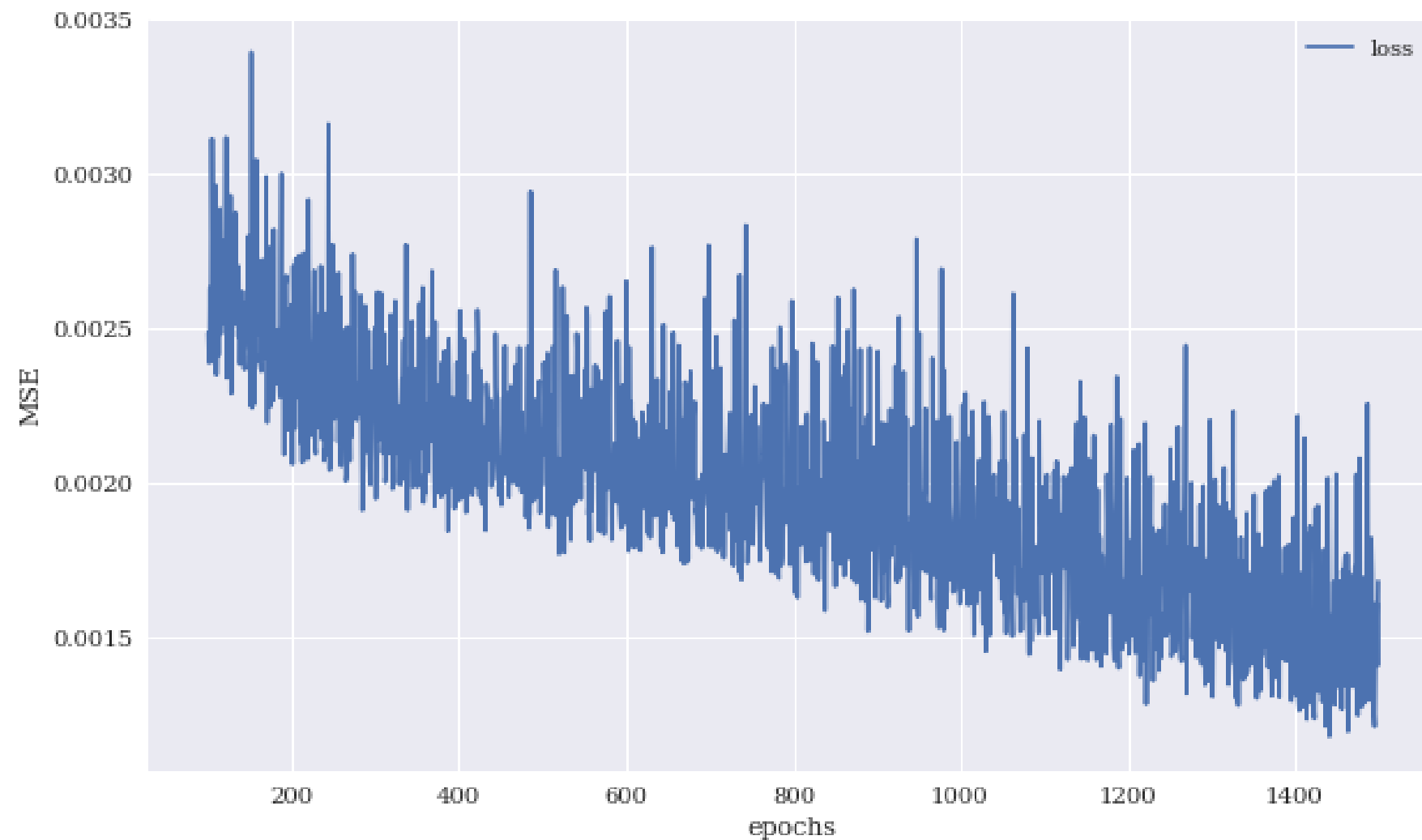
Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

Sample data and neural network approximation



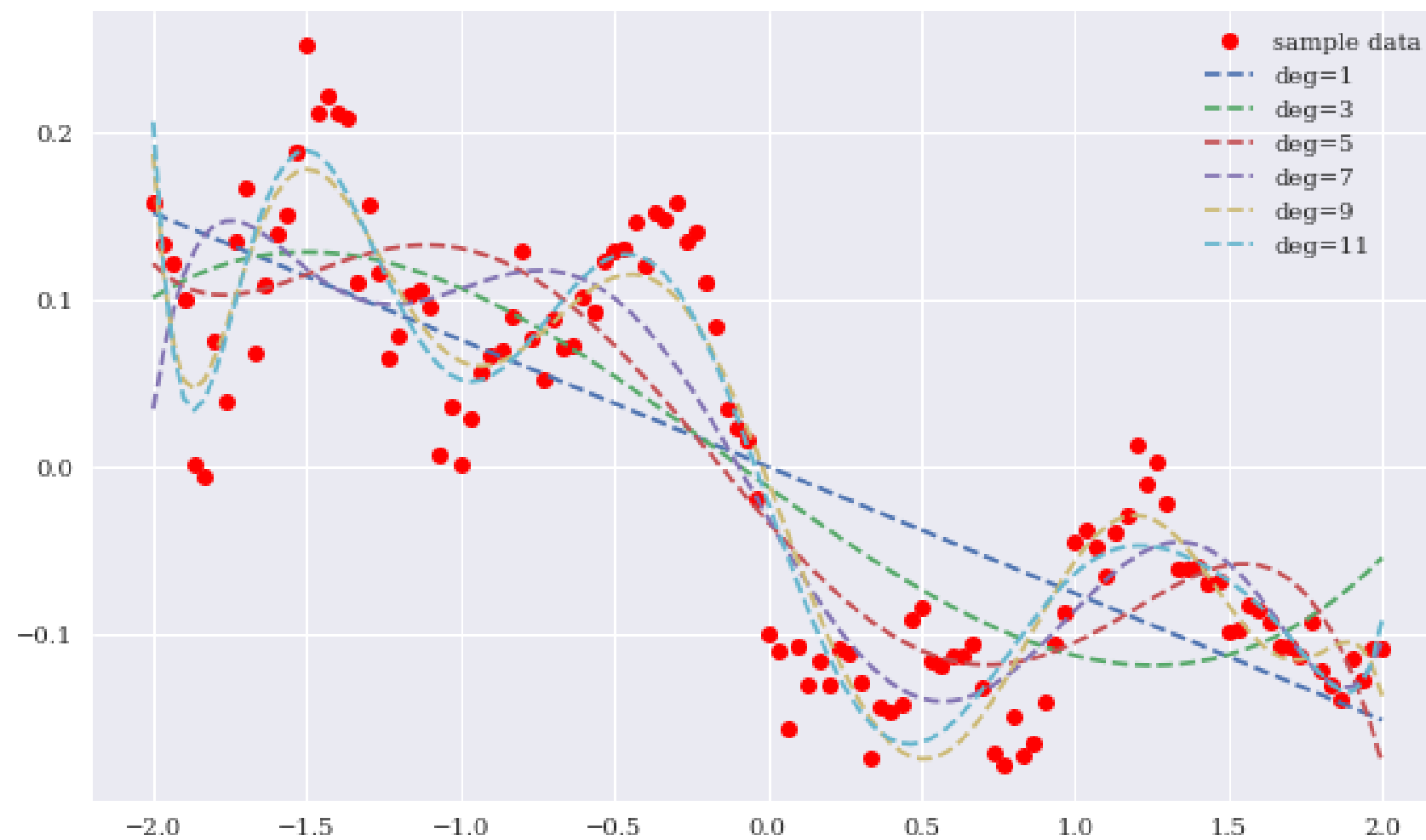
Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

MSE values against number of training epochs



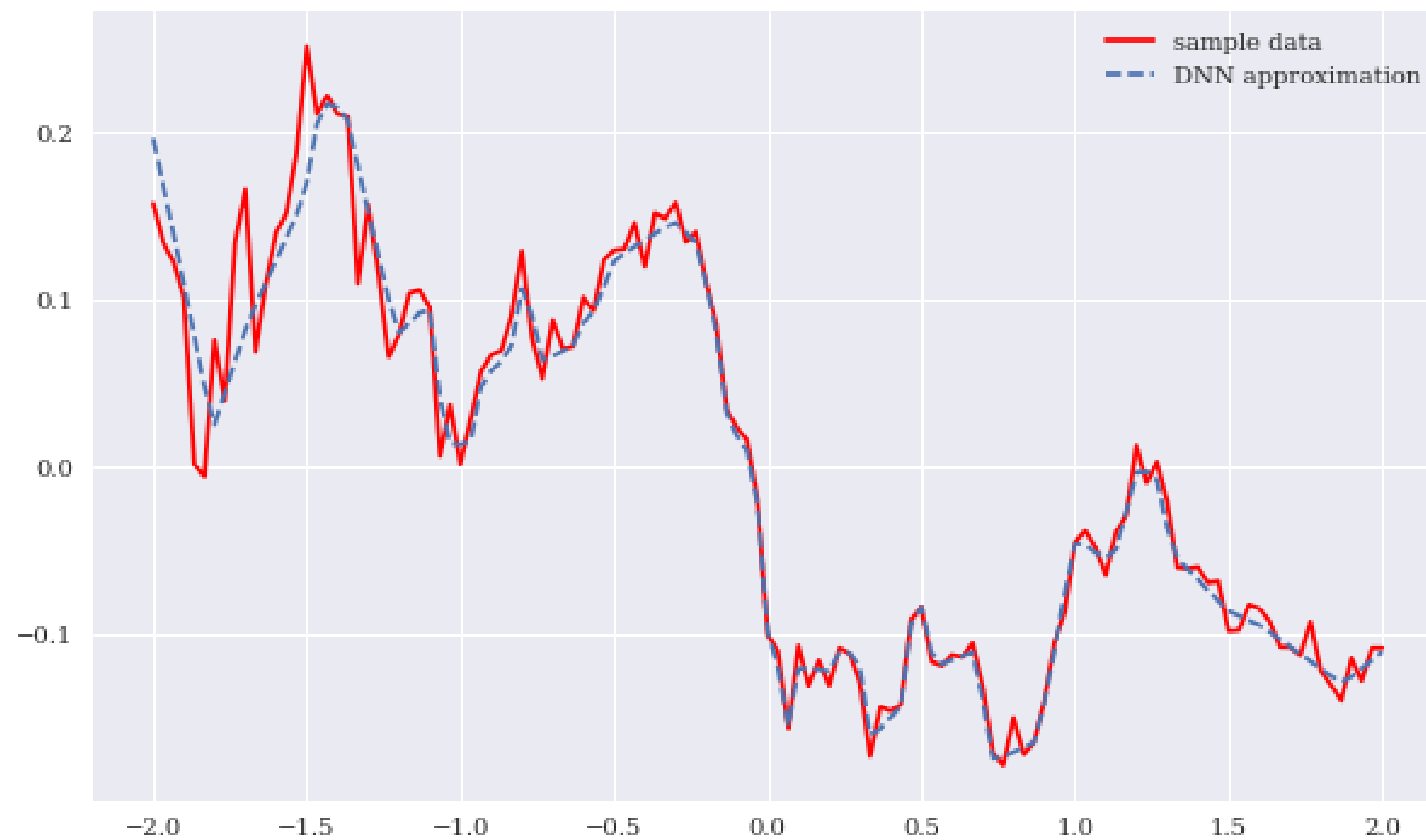
Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

Regression lines for different highest degrees



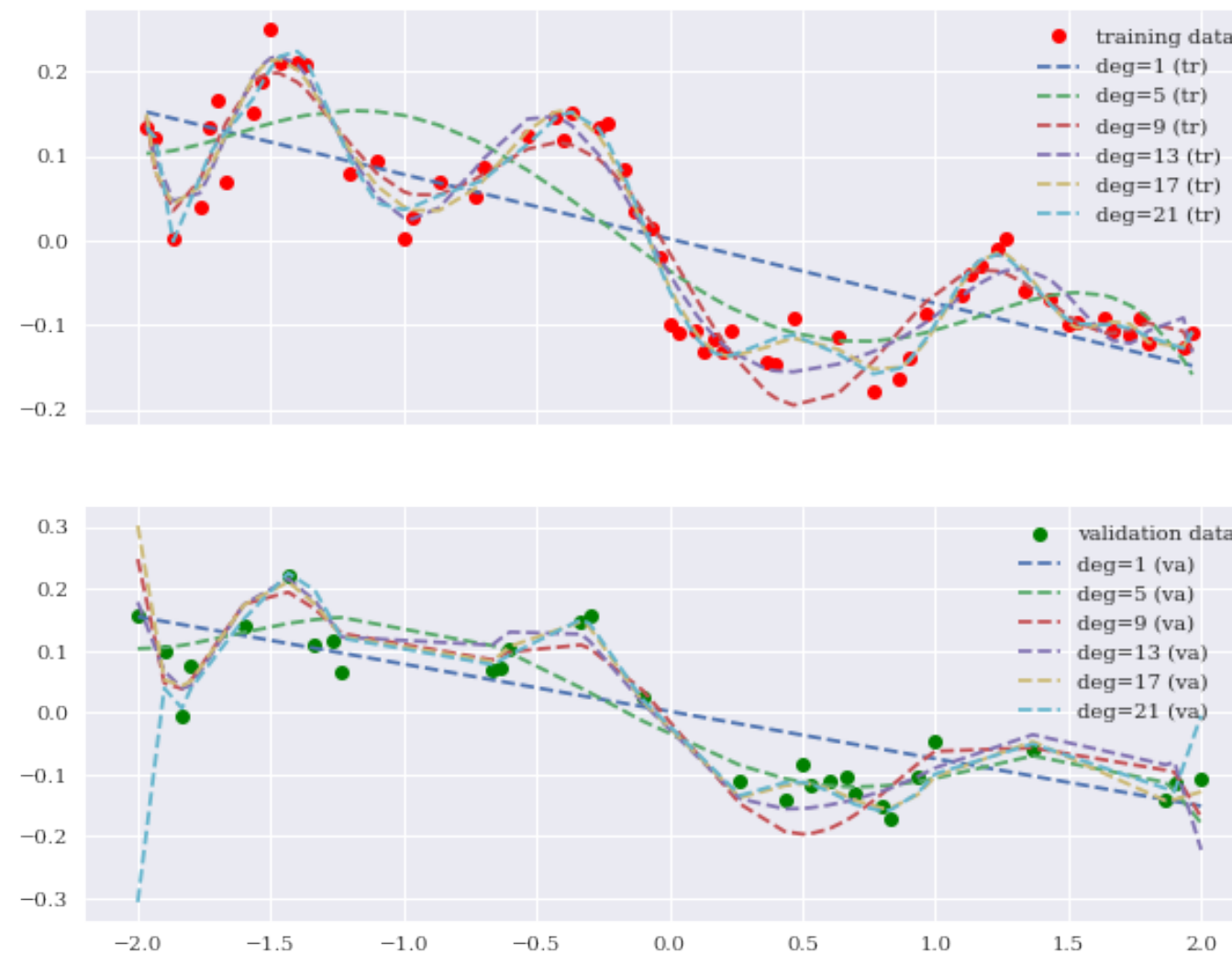
Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

Sample data and DNN approximation (higher capacity)



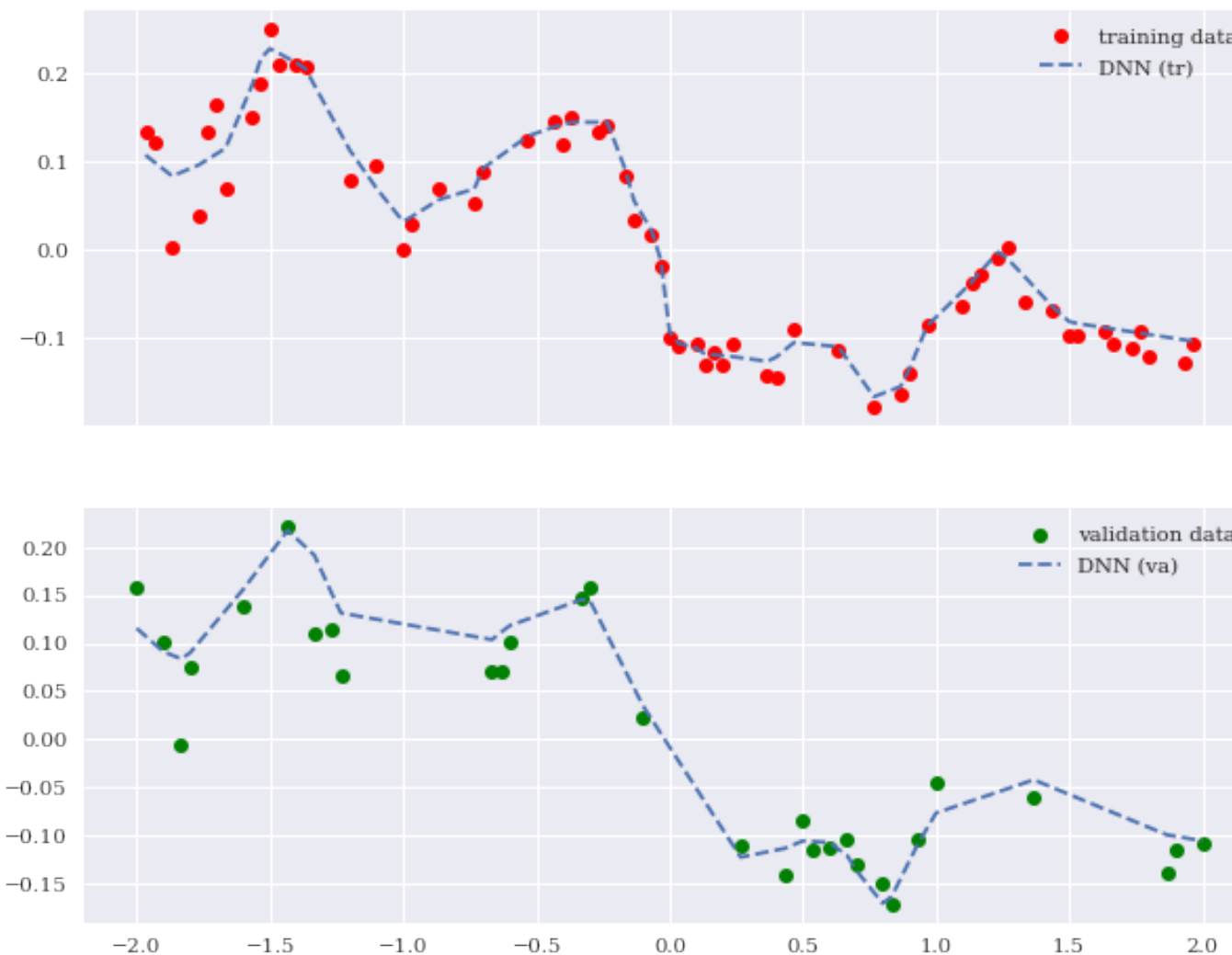
Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

Training and validation data including regression fits



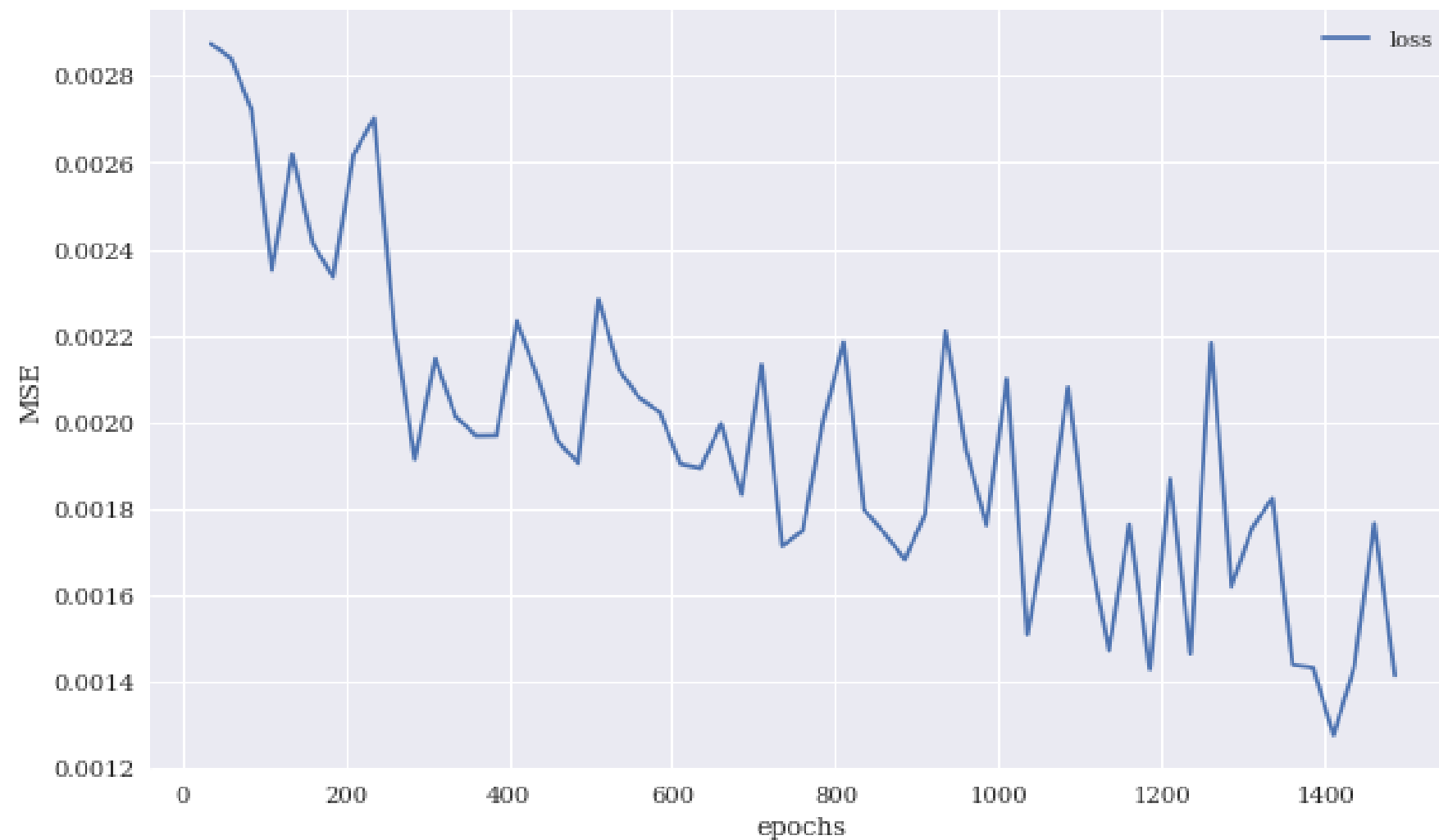
Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

Training and validation data including DNN predictions



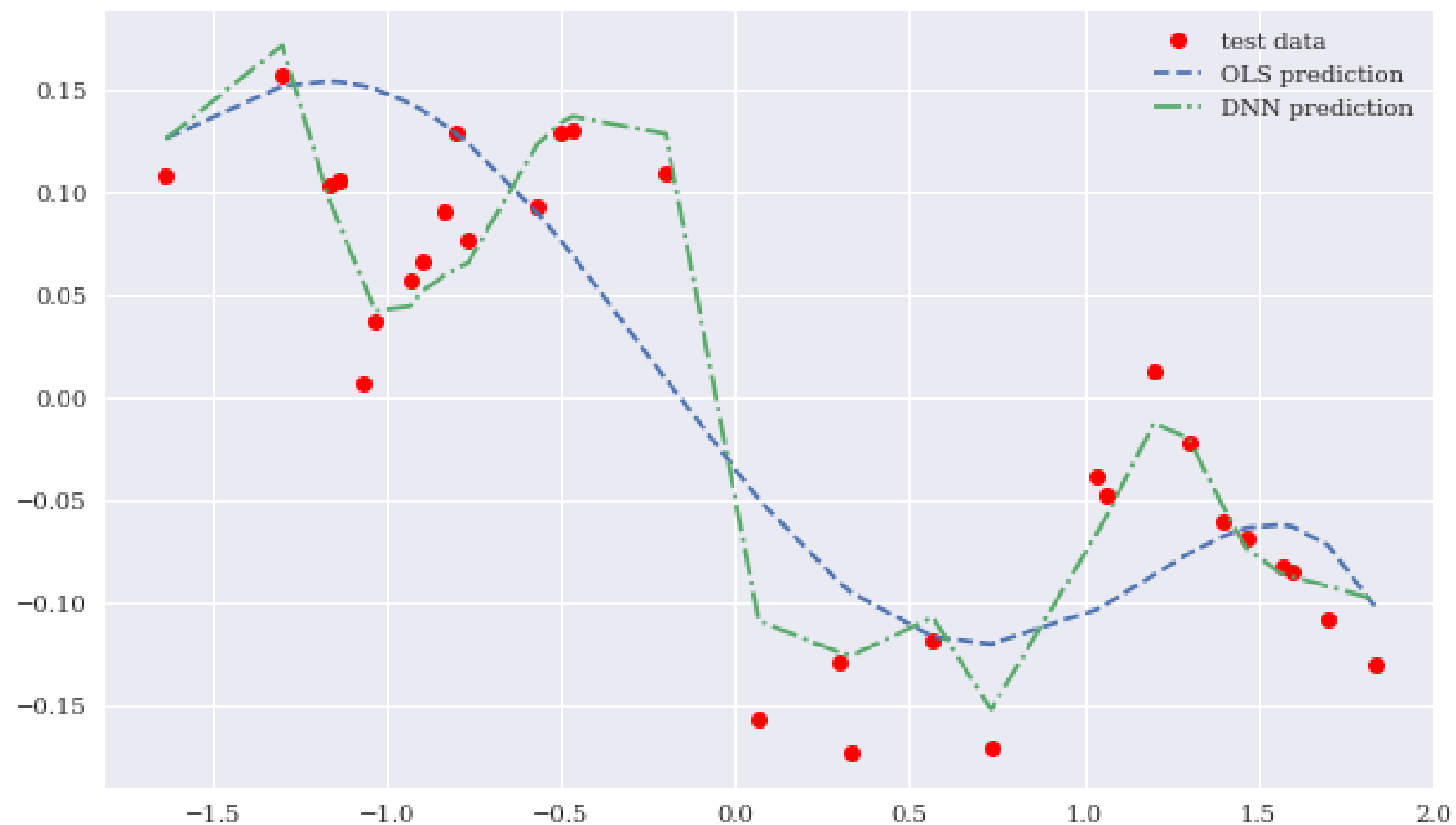
Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

MSE values for DNN model on the training and validation data sets



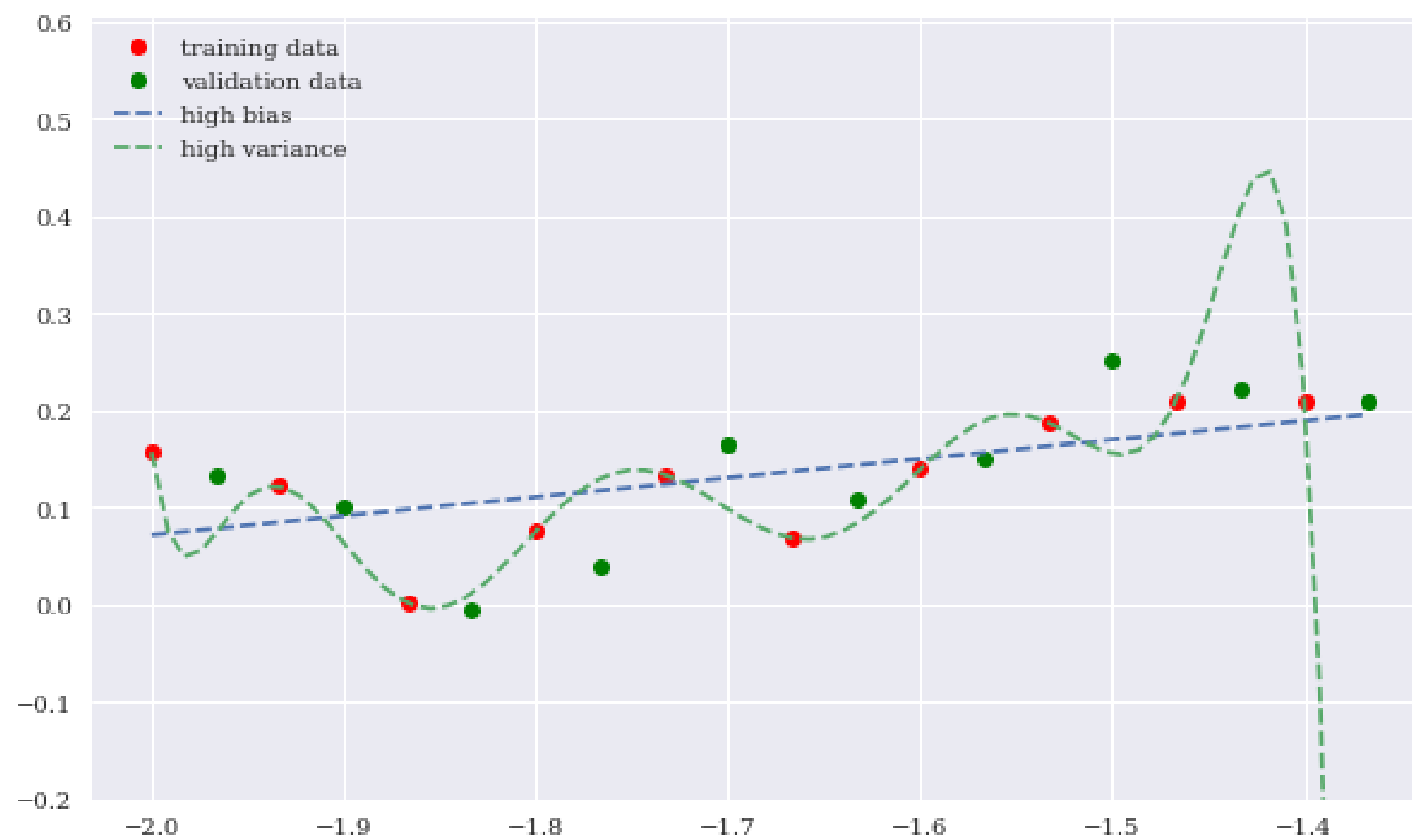
Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

Test data and predictions from OLS regression and the DNN model



Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

High bias and high variance OLS regression fits



Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

Trí tuệ nhân tạo
trong tài chính

Trí tuệ nhân tạo trong tài chính

- Thị trường hiệu quả
- Dự đoán thị trường dựa trên dữ liệu lợi nhuận
- Dự đoán thị trường với nhiều tính năng hơn
- Dự đoán thị trường trong ngày

Life 3.0: Being human in the age of artificial intelligence

Max Tegmark (2017)

Một phép tính lấy thông tin và biến đổi nó, thực hiện cái mà các nhà toán học gọi là hàm... Nếu bạn sở hữu một hàm nhập tất cả dữ liệu tài chính của thế giới và đưa ra những cổ phiếu tốt nhất để mua, bạn sẽ sớm trở nên cực kỳ giàu có.

Efficient Markets

- Giả thuyết thị trường hiệu quả (EMH)
 - Giả thuyết bước đi ngẫu nhiên (RWH)
- Dạng EMH yếu
 - Tập thông tin θ_t chỉ bao gồm lịch sử giá và lợi nhuận trong quá khứ của thị trường.
- Dạng bán mạnh của EMH
 - Tập thông tin θ_t được coi là tất cả thông tin có sẵn công khai, bao gồm không chỉ lịch sử giá và lợi nhuận trong quá khứ mà còn cả báo cáo tài chính, tin bài, dữ liệu thời tiết, v.v.
- Dạng EMH mạnh
 - Tập thông tin θ_t bao gồm tất cả thông tin có sẵn cho bất kỳ ai (nghĩa là ngay cả thông tin cá nhân).

```
import numpy as np
import pandas as pd
from pylab import plt, mpl
plt.style.use('seaborn')
mpl.rcParams['savefig.dpi'] = 300
mpl.rcParams['font.family'] = 'serif'
pd.set_option('precision', 4)
np.set_printoptions(suppress=True, precision=4)

url = 'http://hilpisch.com/aiif_eikon_eod_data.csv'

data = pd.read_csv(url, index_col=0, parse_dates=True).dropna()

(data / data.iloc[0]).plot(figsize=(10, 6), cmap='coolwarm')
```


Normalized time series data (end-of-day)



Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.


```
lags = 7

def add_lags(data, ric, lags):
    cols = []
    df = pd.DataFrame(data[ric])
    for lag in range(1, lags + 1):
        col = 'lag_{}'.format(lag)
        df[col] = df[ric].shift(lag)
        cols.append(col)
    df.dropna(inplace=True)
    return df, cols

dfs = {}
for sym in data.columns:
    df, cols = add_lags(data, sym, lags)
    dfs[sym] = df
    dfs[sym].head(7)
```

Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

lagged prices

	GLD	lag_1	lag_2	lag_3	lag_4	lag_5	lag_6	lag_7
Date								
2010-01-13	111.54	110.49	112.85	111.37	110.82	111.51	109.70	109.80
2010-01-14	112.03	111.54	110.49	112.85	111.37	110.82	111.51	109.70
2010-01-15	110.86	112.03	111.54	110.49	112.85	111.37	110.82	111.51
2010-01-19	111.52	110.86	112.03	111.54	110.49	112.85	111.37	110.82
2010-01-20	108.94	111.52	110.86	112.03	111.54	110.49	112.85	111.37
2010-01-21	107.37	108.94	111.52	110.86	112.03	111.54	110.49	112.85
2010-01-22	107.17	107.37	108.94	111.52	110.86	112.03	111.54	110.49

Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

```
regs = {}
for sym in data.columns:
    df = dfs[sym]
    reg = np.linalg.lstsq(df[cols], df[sym], rcond=-1)[0]
    #Return the least-squares solution to a linear matrix equation
    regs[sym] = reg

rega = np.stack(tuple(regs.values()))
regd = pd.DataFrame(rega, columns=cols, index=data.columns)
regd
```

regression analysis

```
reg = np.linalg.lstsq(df[cols], df[sym], rcond=-1)[0]
```

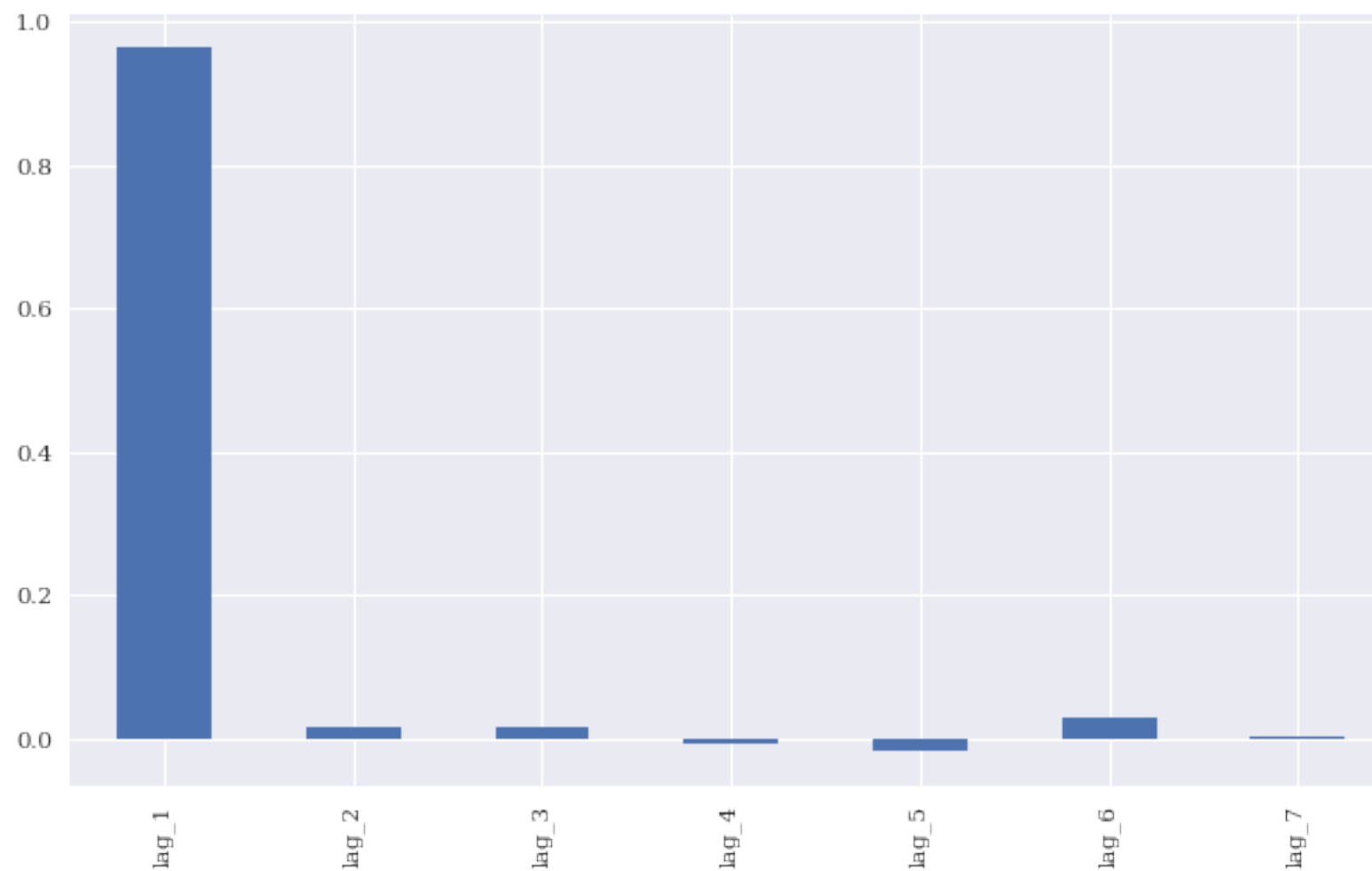
	lag_1	lag_2	lag_3	lag_4	lag_5	lag_6	lag_7
AAPL.O	1.0106	-0.0592	0.0258	0.0535	-0.0172	0.0060	-0.0184
MSFT.O	0.8928	0.0112	0.1175	-0.0832	-0.0258	0.0567	0.0323
INTC.O	0.9519	0.0579	0.0490	-0.0772	-0.0373	0.0449	0.0112
AMZN.O	0.9799	-0.0134	0.0206	0.0007	0.0525	-0.0452	0.0056
GS.N	0.9806	0.0342	-0.0172	0.0042	-0.0387	0.0585	-0.0215
SPY	0.9692	0.0067	0.0228	-0.0244	-0.0237	0.0379	0.0121
.SPX	0.9672	0.0106	0.0219	-0.0252	-0.0318	0.0515	0.0063
.VIX	0.8823	0.0591	-0.0289	0.0284	-0.0256	0.0511	0.0306
EUR=	0.9859	0.0239	-0.0484	0.0508	-0.0217	0.0149	-0.0055
XAU=	0.9864	0.0069	0.0166	-0.0215	0.0044	0.0198	-0.0125
GDX	0.9765	0.0096	-0.0039	0.0223	-0.0364	0.0379	-0.0065
GLD	0.9766	0.0246	0.0060	-0.0142	-0.0047	0.0223	-0.0106

Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

Average optimal regression parameters for the lagged prices

81

```
regd.mean().plot(kind='bar', figsize=(10, 6))
```



Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

Correlations between the lagged time series

```
dfs[sym].corr()
```

	GLD	lag_1	lag_2	lag_3	lag_4	lag_5	lag_6	lag_7
GLD	1.0000	0.9972	0.9946	0.9920	0.9893	0.9867	0.9841	0.9815
lag_1	0.9972	1.0000	0.9972	0.9946	0.9920	0.9893	0.9867	0.9842
lag_2	0.9946	0.9972	1.0000	0.9972	0.9946	0.9920	0.9893	0.9867
lag_3	0.9920	0.9946	0.9972	1.0000	0.9972	0.9946	0.9920	0.9893
lag_4	0.9893	0.9920	0.9946	0.9972	1.0000	0.9972	0.9946	0.9920
lag_5	0.9867	0.9893	0.9920	0.9946	0.9972	1.0000	0.9972	0.9946
lag_6	0.9841	0.9867	0.9893	0.9920	0.9946	0.9972	1.0000	0.9972
lag_7	0.9815	0.9842	0.9867	0.9893	0.9920	0.9946	0.9972	1.0000

Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

```
from statsmodels.tsa.stattools import adfuller
#Tests for stationarity using the Augmented Dickey-Fuller (ADF) test

adfuller(data[sym].dropna())

(-1.9488969577009954,
 0.3094193074034718,
 0,
 2515,
 {'1%': -3.4329527780962255,
  '10%': -2.567382133955709,
  '5%': -2.8626898965523724},
 8446.683102944744)
```


Dự đoán thị trường dựa trên dữ liệu lợi nhuận

- Sự thiếu hiệu quả về mặt thống kê
 - được đưa ra khi một mô hình có thể dự đoán hướng biến động giá trong tương lai với một lợi thế nhất định (giả sử dự đoán đó đúng trong 55% hoặc 60% trường hợp)
- Sự thiếu hiệu quả kinh tế
 - sẽ chỉ được đưa ra nếu sự thiếu hiệu quả về mặt thống kê có thể được khai thác một cách có lợi thông qua chiến lược giao dịch có tính đến chi phí giao dịch, chẳng hạn như.

Dự đoán thị trường dựa trên dữ liệu lợi nhuận

- Tạo tập dữ liệu với dữ liệu trả về nhật ký bị trễ
 - Dữ liệu trả về nhật ký bị trễ được chuẩn hóa cũng được kiểm tra tính dừng (đã cho)
- Các tính năng được kiểm tra độ tương quan (không tương quan)
- Dữ liệu liên quan đến chuỗi thời gian
- Hiệu quả thị trường dạng yếu

```
rets = np.log(data / data.shift(1))
rets.dropna(inplace=True)
rets
```

log
returns

	AAPL.O	MSFT.O	INTC.O	AMZN.O	GS.N	SPY	.SPX	.VIX	EUR=	XAU=	GDX	GLD
Date												
2010-01-05	0.0017	0.0003	-0.0005	0.0059	0.0175	2.6436e-03	3.1108e-03	-0.0350	-2.9883e-03	-0.0012	0.0096	-0.0009
2010-01-06	-0.0160	-0.0062	-0.0034	-0.0183	-0.0107	7.0379e-04	5.4538e-04	-0.0099	3.0577e-03	0.0176	0.0240	0.0164
2010-01-07	-0.0019	-0.0104	-0.0097	-0.0172	0.0194	4.2124e-03	3.9933e-03	-0.0052	-6.5437e-03	-0.0058	-0.0049	-0.0062
2010-01-08	0.0066	0.0068	0.0111	0.0267	-0.0191	3.3223e-03	2.8775e-03	-0.0500	6.5437e-03	0.0037	0.0150	0.0050
2010-01-11	-0.0089	-0.0128	0.0057	-0.0244	-0.0159	1.3956e-03	1.7452e-03	-0.0325	6.9836e-03	0.0144	0.0066	0.0132
...	...	...	...	...	...	...	...	...	...	...	...	...
2019-12-24	0.0010	-0.0002	0.0030	-0.0021	0.0036	3.1131e-05	-1.9543e-04	0.0047	9.0200e-05	0.0091	0.0315	0.0094
2019-12-26	0.0196	0.0082	0.0069	0.0435	0.0056	5.3092e-03	5.1151e-03	-0.0016	8.1143e-04	0.0083	0.0145	0.0078
2019-12-27	-0.0004	0.0018	0.0043	0.0006	-0.0024	-2.4775e-04	3.3951e-05	0.0598	7.0945e-03	-0.0006	-0.0072	-0.0004
2019-12-30	0.0059	-0.0087	-0.0077	-0.0123	-0.0037	-5.5285e-03	-5.7976e-03	0.0985	1.9667e-03	0.0031	0.0212	0.0021
2019-12-31	0.0073	0.0007	0.0039	0.0005	0.0006	2.4264e-03	2.9417e-03	-0.0728	1.1604e-03	0.0012	-0.0071	0.0019

2515 rows × 12 columns

Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

```
dfs = {}
for sym in data:
    df, cols = add_lags(rets, sym, lags)
    mu, std = df[cols].mean(), df[cols].std()
    df[cols] = (df[cols] - mu) / std
    dfs[sym] = df
    dfs[sym].head()
```

	GLD	lag_1	lag_2	lag_3	lag_4	lag_5	lag_6	lag_7
Date								
2010-01-14	0.0044	0.9570	-2.1692	1.3386	0.4959	-0.6434	1.6613	-0.1028
2010-01-15	-0.0105	0.4379	0.9571	-2.1689	1.3388	0.4966	-0.6436	1.6614
2010-01-19	0.0059	-1.0842	0.4385	0.9562	-2.1690	1.3395	0.4958	-0.6435
2010-01-20	-0.0234	0.5967	-1.0823	0.4378	0.9564	-2.1686	1.3383	0.4958
2010-01-21	-0.0145	-2.4045	0.5971	-1.0825	0.4379	0.9571	-2.1680	1.3384

Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

Augmented Dickey-Fuller (ADF)

Tests for stationarity of the time series data

```
adfuller(dfs[sym] ['lag_1'])
```

```
(-51.568251505825536,  
 0.0,  
 0,  
 2507,  
 {'1%': -3.4329610922579095,  
  '10%': -2.567384088736619,  
  '5%': -2.8626935681060375},  
 7017.165474260225)
```

Shows the correlation data for the features

```
dfs[sym].corr()
```

	GLD	lag_1	lag_2	lag_3	lag_4	lag_5	lag_6	lag_7
GLD	1.0000	-0.0297	0.0003	1.2635e-02	-0.0026	-5.9392e-03	0.0099	-0.0013
lag_1	-0.0297	1.0000	-0.0305	8.1418e-04	0.0128	-2.8765e-03	-0.0053	0.0098
lag_2	0.0003	-0.0305	1.0000	-3.1617e-02	0.0003	1.3234e-02	-0.0043	-0.0052
lag_3	0.0126	0.0008	-0.0316	1.0000e+00	-0.0313	-6.8542e-06	0.0141	-0.0044
lag_4	-0.0026	0.0128	0.0003	-3.1329e-02	1.0000	-3.1761e-02	0.0002	0.0141
lag_5	-0.0059	-0.0029	0.0132	-6.8542e-06	-0.0318	1.0000e+00	-0.0323	0.0002
lag_6	0.0099	-0.0053	-0.0043	1.4115e-02	0.0002	-3.2289e-02	1.0000	-0.0324
lag_7	-0.0013	0.0098	-0.0052	-4.3869e-03	0.0141	2.1707e-04	-0.0324	1.0000

Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

OLS Regression

```
from sklearn.metrics import accuracy_score

%%time
for sym in data:
    df = dfs[sym]
    reg = np.linalg.lstsq(df[cols], df[sym], rcond=-1)[0]
    pred = np.dot(df[cols], reg)
    acc = accuracy_score(np.sign(df[sym]), np.sign(pred))
    print(f'OLS | {sym:10s} | acc={acc:.4f}')
```

OLS Regression Accuracy

OLS		AAPL.O		acc=0.5056
OLS		MSFT.O		acc=0.5088
OLS		INTC.O		acc=0.5040
OLS		AMZN.O		acc=0.5048
OLS		GS.N		acc=0.5080
OLS		SPY		acc=0.5080
OLS		.SPX		acc=0.5167
OLS		.VIX		acc=0.5291
OLS		EUR=		acc=0.4984
OLS		XAU=		acc=0.5207
OLS		GDX		acc=0.5307
OLS		GLD		acc=0.5072

Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

```
from sklearn.neural_network import MLPRegressor
```

```
%%time
for sym in data.columns:
    df = dfs[sym]
    model = MLPRegressor(hidden_layer_sizes=[512],
        random_state=100,
        max_iter=1000,
        early_stopping=True,
        validation_fraction=0.15,
        shuffle=False)
    model.fit(df[cols].values, df[sym].values)
    pred = model.predict(df[cols].values)
    acc = accuracy_score(np.sign(df[sym].values),
        np.sign(pred))
    print(f'MLP | {sym:10s} | acc={acc:.4f}')
```


Scikit-learn MLPRegressor Accuracy

MLP	AAPL.O	acc=0.6005
MLP	MSFT.O	acc=0.5853
MLP	INTC.O	acc=0.5766
MLP	AMZN.O	acc=0.5510
MLP	GS.N	acc=0.6527
MLP	SPY	acc=0.5419
MLP	.SPX	acc=0.5399
MLP	.VIX	acc=0.6579
MLP	EUR=	acc=0.5642
MLP	XAU=	acc=0.5522
MLP	GDX	acc=0.6029
MLP	GLD	acc=0.5259

Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

```
import tensorflow as tf
from keras.layers import Dense
from keras.models import Sequential

np.random.seed(100)
tf.random.set_seed(100)

def create_model(problem='regression'):
    model = Sequential()
    model.add(Dense(512, input_dim=len(cols), activation='relu'))
    if problem == 'regression':
        model.add(Dense(1, activation='linear'))
        model.compile(loss='mse', optimizer='adam')
    else:
        model.add(Dense(1, activation='sigmoid'))
        model.compile(loss='binary_crossentropy', optimizer='adam')
    return model
```

```
%%time
for sym in data.columns[:]:
    df = dfs[sym]
    model = create_model()
    model.fit(df[cols], df[sym], epochs=25, verbose=False)
    pred = model.predict(df[cols])
    acc = accuracy_score(np.sign(df[sym]), np.sign(pred))
    print(f'DNN | {sym:10s} | acc={acc:.4f}')
```

TF Keras DNN Accuracy

DNN		AAPL.O		acc=0.6069
DNN		MSFT.O		acc=0.6260
DNN		INTC.O		acc=0.6344
DNN		AMZN.O		acc=0.6316
DNN		GS.N		acc=0.6045
DNN		SPY		acc=0.5610
DNN		.SPX		acc=0.5435
DNN		.VIX		acc=0.6096
DNN		EUR=		acc=0.5817
DNN		XAU=		acc=0.6017
DNN		GDX		acc=0.6164
DNN		GLD		acc=0.5973

Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

Train Data (0.8): In-Sample Test Data (0.2): Out-of-Sample

```
split = int(len(dfs[sym]) * 0.8)
```

```
%%time
for sym in data.columns:
    df = dfs[sym]
    train = df.iloc[:split]
    reg = np.linalg.lstsq(train[cols], train[sym], rcond=-1)[0]
    test = df.iloc[split:]
    pred = np.dot(test[cols], reg)
    acc = accuracy_score(np.sign(test[sym]), np.sign(pred))
    print(f'OLS | {sym:10s} | acc={acc:.4f}')
```

OLS Out-of-Sample Accuracy

OLS		AAPL.O		acc=0.5219
OLS		MSFT.O		acc=0.4960
OLS		INTC.O		acc=0.5418
OLS		AMZN.O		acc=0.4841
OLS		GS.N		acc=0.4980
OLS		SPY		acc=0.5020
OLS		.SPX		acc=0.5120
OLS		.VIX		acc=0.5458
OLS		EUR=		acc=0.4482
OLS		XAU=		acc=0.5299
OLS		GDX		acc=0.5159
OLS		GLD		acc=0.5100

Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

```
%%time
for sym in data.columns:
    df = dfs[sym]
    train = df.iloc[:split]
    model = MLPRegressor(hidden_layer_sizes=[512],
        random_state=100,
        max_iter=1000,
        early_stopping=True,
        validation_fraction=0.15,
        shuffle=False)
    model.fit(train[cols].values, train[sym].values)
    test = df.iloc[split:]
    pred = model.predict(test[cols].values)
    acc = accuracy_score(np.sign(test[sym].values), np.sign(pred))
    print(f'MLP | {sym:10s} | acc={acc:.4f}')
```


MLP Out-of-Sample Accuracy

MLP		AAPL.O		acc=0.4920
MLP		MSFT.O		acc=0.5279
MLP		INTC.O		acc=0.5279
MLP		AMZN.O		acc=0.4641
MLP		GS.N		acc=0.5040
MLP		SPY		acc=0.5259
MLP		.SPX		acc=0.5478
MLP		.VIX		acc=0.5279
MLP		EUR=		acc=0.4980
MLP		XAU=		acc=0.5239
MLP		GDX		acc=0.4880
MLP		GLD		acc=0.5000

Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

```
%%time
for sym in data.columns:
    df = dfs[sym]
    train = df.iloc[:split]
    model = create_model()
    model.fit(train[cols], train[sym], epochs=50, verbose=False)
    test = df.iloc[split:]
    pred = model.predict(test[cols])
    acc = accuracy_score(np.sign(test[sym]), np.sign(pred))
    print(f'DNN | {sym:10s} | acc={acc:.4f}')
```

DNN Out-of-Sample Accuracy

DNN	AAPL.O	acc=0.4701
DNN	MSFT.O	acc=0.4960
DNN	INTC.O	acc=0.5040
DNN	AMZN.O	acc=0.4920
DNN	GS.N	acc=0.5538
DNN	SPY	acc=0.5299
DNN	.SPX	acc=0.5458
DNN	.VIX	acc=0.5020
DNN	EUR=	acc=0.5100
DNN	XAU=	acc=0.4940
DNN	GDX	acc=0.4661
DNN	GLD	acc=0.4880

Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

Dự đoán thị trường với nhiều tính năng hơn

- Trong giao dịch, có một truyền thống lâu đời là sử dụng các chỉ báo kỹ thuật để tạo ra các tín hiệu mua hoặc bán dựa trên các mô hình quan sát được.
- Các chỉ báo kỹ thuật như vậy, về cơ bản thuộc bất kỳ loại nào, cũng có thể được sử dụng làm tính năng cho việc đào tạo mạng lưới thần kinh.
- SMA, các giá trị tối thiểu và tối đa, động lượng và độ biến động lặn là các đặc điểm

```
url = 'http://hilpisch.com/aiif_eikon_eod_data.csv'
```

```
data = pd.read_csv(url, index_col=0, parse_dates=True).dropna()  
data
```

	AAPL.O	MSFT.O	INTC.O	AMZN.O	GS.N	SPY	.SPX	.VIX	EUR=	XAU=	GDX	GLD
Date												
2010-01-04	30.5728	30.950	20.88	133.90	173.08	113.33	1132.99	20.04	1.4411	1120.0000	47.71	109.80
2010-01-05	30.6257	30.960	20.87	134.69	176.14	113.63	1136.52	19.35	1.4368	1118.6500	48.17	109.70
2010-01-06	30.1385	30.770	20.80	132.25	174.26	113.71	1137.14	19.16	1.4412	1138.5000	49.34	111.51
2010-01-07	30.0828	30.452	20.60	130.00	177.67	114.19	1141.69	19.06	1.4318	1131.9000	49.10	110.82
2010-01-08	30.2828	30.660	20.83	133.52	174.31	114.57	1144.98	18.13	1.4412	1136.1000	49.84	111.37
...	...	...	...	...	...	...	...	...	...	...	...	...
2019-12-24	284.2700	157.380	59.41	1789.21	229.91	321.23	3223.38	12.67	1.1087	1498.8100	28.66	141.27
2019-12-26	289.9100	158.670	59.82	1868.77	231.21	322.94	3239.91	12.65	1.1096	1511.2979	29.08	142.38
2019-12-27	289.8000	158.960	60.08	1869.80	230.66	322.86	3240.02	13.43	1.1175	1510.4167	28.87	142.33
2019-12-30	291.5200	157.590	59.62	1846.89	229.80	321.08	3221.29	14.82	1.1197	1515.1230	29.49	142.63
2019-12-31	293.6500	157.700	59.85	1847.84	229.93	321.86	3230.78	13.78	1.1210	1517.0100	29.28	142.90

2516 rows × 12 columns

Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

```
def add_lags(data, ric, lags, window=50):
    cols = []
    df = pd.DataFrame(data[ric])
    df.dropna(inplace=True)
    df['r'] = np.log(df / df.shift())
    df['sma'] = df[ric].rolling(window).mean()
    df['min'] = df[ric].rolling(window).min()
    df['max'] = df[ric].rolling(window).max()
    df['mom'] = df['r'].rolling(window).mean()
    df['vol'] = df['r'].rolling(window).std()
    df.dropna(inplace=True)
    df['d'] = np.where(df['r'] > 0, 1, 0)
    features = [ric, 'r', 'd', 'sma', 'min', 'max', 'mom', 'vol']
    for f in features:
        for lag in range(1, lags + 1):
            col = f'{f}_lag_{lag}'
            df[col] = df[f].shift(lag)
            cols.append(col)
    df.dropna(inplace=True)
    return df, cols
```

```
lags = 5

dfs = {}
for ric in data:
    df, cols = add_lags(data, ric, lags)
    dfs[ric] = df.dropna(), cols

len(cols)
```

40

```
from sklearn.neural_network import MLPClassifier
```

```
%%time
for ric in data:
    model = MLPClassifier(hidden_layer_sizes=[512],
                           random_state=100,
                           max_iter=1000,
                           early_stopping=True,
                           validation_fraction=0.15,
                           shuffle=False)
    df, cols = dfs[ric]
    df[cols] = (df[cols] - df[cols].mean()) / df[cols].std()
    model.fit(df[cols].values, df['d'].values)
    pred = model.predict(df[cols].values)
    acc = accuracy_score(df['d'].values, pred)
    print(f'IN-SAMPLE | {ric:7s} | acc={acc:.4f}')
```


MLP In-Sample Accuracy

IN-SAMPLE	AAPL.O	acc=0.5510
IN-SAMPLE	MSFT.O	acc=0.5376
IN-SAMPLE	INTC.O	acc=0.5607
IN-SAMPLE	AMZN.O	acc=0.5559
IN-SAMPLE	GS.N	acc=0.5794
IN-SAMPLE	SPY	acc=0.5729
IN-SAMPLE	.SPX	acc=0.5941
IN-SAMPLE	.VIX	acc=0.6940
IN-SAMPLE	EUR=	acc=0.5766
IN-SAMPLE	XAU=	acc=0.5672
IN-SAMPLE	GDX	acc=0.5847
IN-SAMPLE	GLD	acc=0.5567

Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

```
%%time
for ric in data:
    model = create_model('classification')
    df, cols = dfs[ric]
    df[cols] = (df[cols] - df[cols].mean()) / df[cols].std()
    model.fit(df[cols], df['d'], epochs=50, verbose=False)
    pred = np.where(model.predict(df[cols]) > 0.5, 1, 0)
    acc = accuracy_score(df['d'], pred)
    print(f'IN-SAMPLE | {ric:7s} | acc={acc:.4f}')
```

TF Keras DNN In-Sample Accuracy

IN-SAMPLE	AAPL.O	acc=0.7042
IN-SAMPLE	MSFT.O	acc=0.6928
IN-SAMPLE	INTC.O	acc=0.6969
IN-SAMPLE	AMZN.O	acc=0.6713
IN-SAMPLE	GS.N	acc=0.6924
IN-SAMPLE	SPY	acc=0.6806
IN-SAMPLE	.SPX	acc=0.6920
IN-SAMPLE	.VIX	acc=0.7347
IN-SAMPLE	EUR=	acc=0.6766
IN-SAMPLE	XAU=	acc=0.7038
IN-SAMPLE	GDX	acc=0.6806
IN-SAMPLE	GLD	acc=0.6936

Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

```
def train_test_model(model):  
    for ric in data:  
        df, cols = dfs[ric]  
        split = int(len(df) * 0.85)  
        train = df.iloc[:split].copy()  
        mu, std = train[cols].mean(), train[cols].std()  
        train[cols] = (train[cols] - mu) / std  
        model.fit(train[cols].values, train['d'].values)  
        test = df.iloc[split:].copy()  
        test[cols] = (test[cols] - mu) / std  
        pred = model.predict(test[cols].values)  
        acc = accuracy_score(test['d'].values, pred)  
        print(f'OUT-OF-SAMPLE | {ric:7s} | acc={acc:.4f}')
```

```
model_mlp = MLPClassifier(hidden_layer_sizes=[512],  
random_state=100,  
max_iter=1000,  
early_stopping=True,  
validation_fraction=0.15,  
shuffle=False)
```

```
train_test_model(model_mlp)
```

`train_test_model(model_mlp)`

OUT-OF-SAMPLE	AAPL.O	acc=0.4432
OUT-OF-SAMPLE	MSFT.O	acc=0.4595
OUT-OF-SAMPLE	INTC.O	acc=0.5000
OUT-OF-SAMPLE	AMZN.O	acc=0.5270
OUT-OF-SAMPLE	GS.N	acc=0.4838
OUT-OF-SAMPLE	SPY	acc=0.4811
OUT-OF-SAMPLE	.SPX	acc=0.5027
OUT-OF-SAMPLE	.VIX	acc=0.5676
OUT-OF-SAMPLE	EUR=	acc=0.4649
OUT-OF-SAMPLE	XAU=	acc=0.5514
OUT-OF-SAMPLE	GDX	acc=0.5162
OUT-OF-SAMPLE	GLD	acc=0.4946

```
from sklearn.ensemble import BaggingClassifier

base_estimator = MLPClassifier(hidden_layer_sizes=[256],
                                random_state=100,
                                max_iter=1000,
                                early_stopping=True,
                                validation_fraction=0.15,
                                shuffle=False)

model_bag = BaggingClassifier(base_estimator=base_estimator,
                               n_estimators=35,
                               max_samples=0.25,
                               max_features=0.5,
                               bootstrap=False,
                               bootstrap_features=True,
                               n_jobs=8,
                               random_state=100
                               )
```

Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

`train_test_model(model_bag)`

OUT-OF-SAMPLE	AAPL.O	acc=0.5000
OUT-OF-SAMPLE	MSFT.O	acc=0.5703
OUT-OF-SAMPLE	INTC.O	acc=0.5054
OUT-OF-SAMPLE	AMZN.O	acc=0.5270
OUT-OF-SAMPLE	GS.N	acc=0.5135
OUT-OF-SAMPLE	SPY	acc=0.5568
OUT-OF-SAMPLE	.SPX	acc=0.5514
OUT-OF-SAMPLE	.VIX	acc=0.5432
OUT-OF-SAMPLE	EUR=	acc=0.5054
OUT-OF-SAMPLE	XAU=	acc=0.5351
OUT-OF-SAMPLE	GDX	acc=0.5054
OUT-OF-SAMPLE	GLD	acc=0.5189

Dự đoán thị trường trong ngày

- Hiệu quả yếu vào cuối ngày
- Ít hiệu quả trong ngày
 - Dữ liệu trong ngày
 - Dữ liệu hàng giờ

Dữ liệu trong ngày

```
url = 'http://hilpisch.com/aiif_eikon_id_data.csv'
data = pd.read_csv(url, index_col=0, parse_dates=True) # .dropna()
data.tail()
```

	AAPL.O	MSFT.O	INTC.O	AMZN.O	GS.N	SPY	.SPX	.VIX	EUR=	XAU=	GDX	GLD
Date												
2019-12-31 20:00:00	292.36	157.2845	59.575	1845.22	228.92	320.94	3219.75	14.16	1.1215	1519.6451	29.40	143.12
2019-12-31 21:00:00	293.37	157.4900	59.820	1846.95	229.89	321.89	3230.56	13.92	1.1216	1517.3600	29.29	142.93
2019-12-31 22:00:00	293.82	157.9000	59.990	1850.20	229.93	322.39	3230.78	13.78	1.1210	1517.0100	29.30	142.90
2019-12-31 23:00:00	293.75	157.8300	59.910	1851.00	NaN	322.22	NaN	NaN	1.1211	1516.8900	29.40	142.88
2020-01-01 00:00:00	293.81	157.8800	59.870	1850.10	NaN	322.32	NaN	NaN	1.1211	NaN	29.34	143.00

Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

```
lags = 5

dfs = {}
for ric in data:
    df, cols = add_lags(data, ric, lags)
    dfs[ric] = df, cols
```

```
train_test_model(model_mlp)
```

OUT-OF-SAMPLE	 AAPL.O	 acc=0.5420
OUT-OF-SAMPLE	 MSFT.O	 acc=0.4930
OUT-OF-SAMPLE	 INTC.O	 acc=0.5549
OUT-OF-SAMPLE	 AMZN.O	 acc=0.4709
OUT-OF-SAMPLE	 GS.N	 acc=0.5184
OUT-OF-SAMPLE	 SPY	 acc=0.4860
OUT-OF-SAMPLE	 .SPX	 acc=0.5019
OUT-OF-SAMPLE	 .VIX	 acc=0.4885
OUT-OF-SAMPLE	 EUR=	 acc=0.5130
OUT-OF-SAMPLE	 XAU=	 acc=0.4824
OUT-OF-SAMPLE	 GDX	 acc=0.4765
OUT-OF-SAMPLE	 GLD	 acc=0.5455

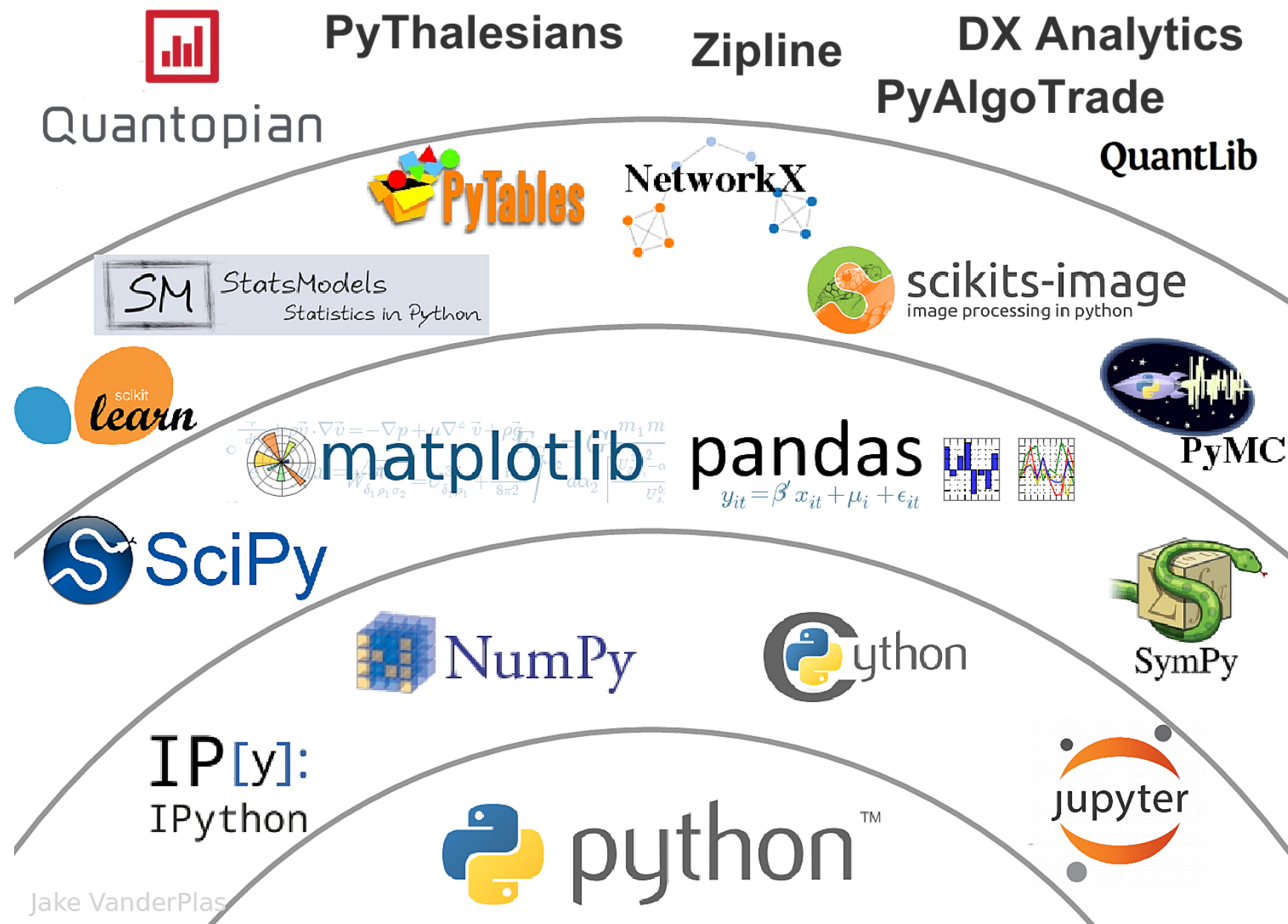
Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

```
train_test_model(model_bag)
```

OUT-OF-SAMPLE	AAPL.O	acc=0.5660
OUT-OF-SAMPLE	MSFT.O	acc=0.5551
OUT-OF-SAMPLE	INTC.O	acc=0.5072
OUT-OF-SAMPLE	AMZN.O	acc=0.4830
OUT-OF-SAMPLE	GS.N	acc=0.5020
OUT-OF-SAMPLE	SPY	acc=0.4680
OUT-OF-SAMPLE	.SPX	acc=0.4677
OUT-OF-SAMPLE	.VIX	acc=0.5161
OUT-OF-SAMPLE	EUR=	acc=0.5242
OUT-OF-SAMPLE	XAU=	acc=0.5229
OUT-OF-SAMPLE	GDX	acc=0.5107
OUT-OF-SAMPLE	GLD	acc=0.5475

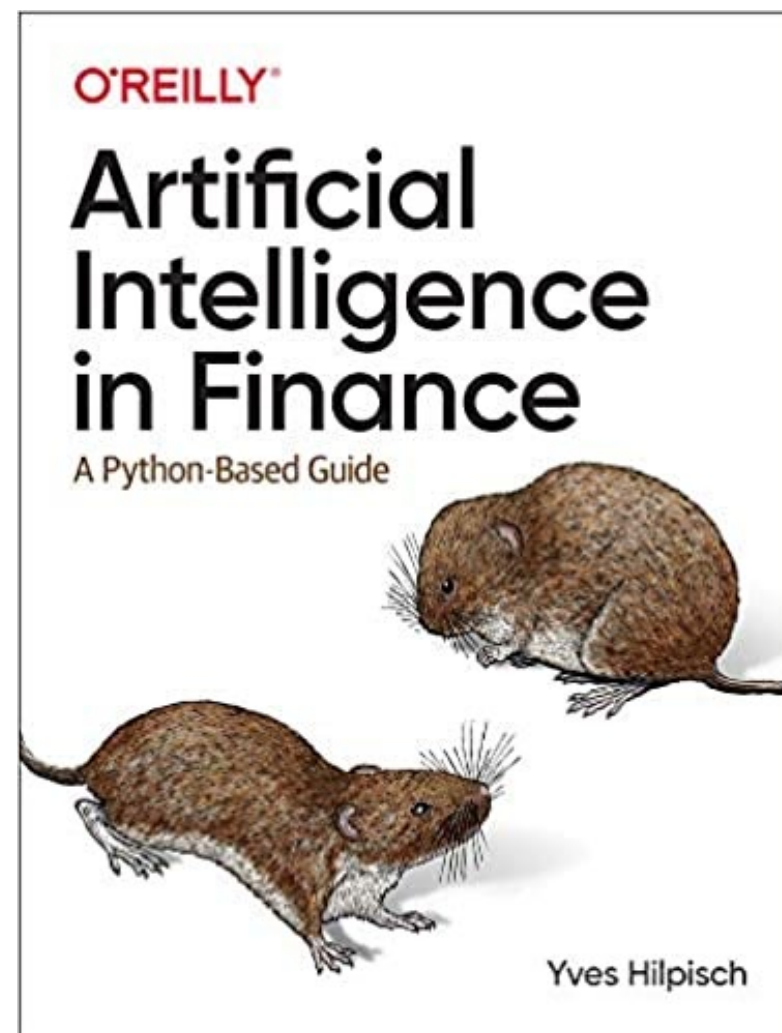
Source: Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media.

The Quant Finance PyData Stack



Source: http://nbviewer.jupyter.org/format/slides/github/quantopian/pyfolio/blob/master/pyfolio/examples/overview_slides.ipynb#5

Yves Hilpisch (2020),
Artificial Intelligence in Finance:
A Python-Based Guide,
O'Reilly



Source: <https://www.amazon.com/Artificial-Intelligence-Finance-Python-Based-Guide/dp/1492055433>

Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly


[yhilpisch](#) / [aiif](#) Public

<https://github.com/yhilpisch/aiif>
Notifications
Star 98
Fork 77

[Code](#)
[Issues](#)
[Pull requests](#)
[Actions](#)
[Projects](#)
[Wiki](#)
[Security](#)
[Insights](#)

main
1 branch
0 tags
Go to file
Code

yves Code updates for TF 2.3. e334251 on Dec 8, 2020 4 commits

code	Code updates for TF 2.3.	11 months ago
.gitignore	Code updates for TF 2.3.	11 months ago
LICENSE.txt	Code updates.	11 months ago
README.md	Code updates.	11 months ago

README.md

Artificial Intelligence in Finance

About this Repository

This repository provides Python code and Jupyter Notebooks accompanying the **Artificial Intelligence in Finance** book published by [O'Reilly](#).



About

Jupyter Notebooks and code for the book **Artificial Intelligence in Finance** (O'Reilly) by Yves Hilpisch.

home.tpq.io/books/aiif

[Readme](#)

[View license](#)

Releases

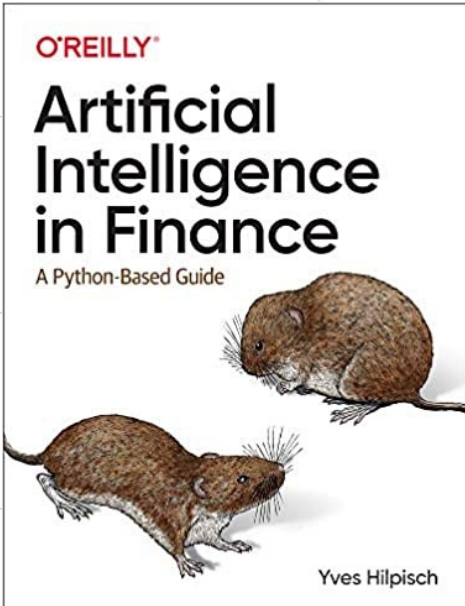
No releases published

Packages

No packages published

Languages

Jupyter Notebook 97.4% Python 2.6%



Source: <https://github.com/yhilpisch/aiif>

Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly


[yhilpisch / aiif](#)
Public

Notifications
Star 98
Fork 77

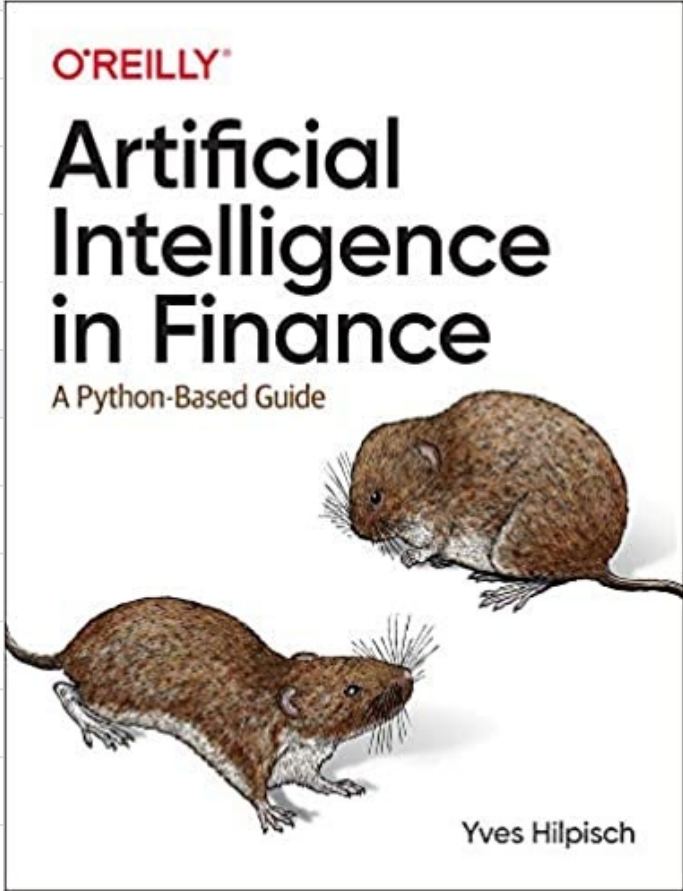
[Code](#)
[Issues](#)
[Pull requests](#)
[Actions](#)
[Projects](#)
[Wiki](#)
[Security](#)
[Insights](#)

main
aiif / code /
<https://github.com/yhilpisch/aiif/tree/main/code>
Go to file

yves Code updates for TF 2.3.

e334251 on Dec 8, 2020 History

..	
oanda	Code updates for TF 2.3.
01_artificial_intelligence.ipynb	Code updates for TF 2.3.
02_superintelligence.ipynb	Code updates for TF 2.3.
03_normative_finance.ipynb	Code updates for TF 2.3.
04_data_driven_finance_a.ipynb	Initial commit.
04_data_driven_finance_b.ipynb	Initial commit.
05_machine_learning.ipynb	Code updates for TF 2.3.
06_ai_first_finance.ipynb	Code updates for TF 2.3.
07_dense_networks.ipynb	Code updates for TF 2.3.
08_recurrent_networks.ipynb	Code updates for TF 2.3.
09_reinforcement_learning_a.ipynb	Code updates.
09_reinforcement_learning_b.ipynb	Code updates for TF 2.3.



Source: <https://github.com/yhilpisch/aiif/tree/main/code>

Python in Google Colab (Python101)

<https://colab.research.google.com/drive/1FEG6DnGvwfUbeo4zJ1zTunjMqf2RkCrT>

The screenshot shows a Google Colab notebook interface. The browser address bar displays the URL: <https://colab.research.google.com/drive/1FEG6DnGvwfUbeo4zJ1zTunjMqf2RkCrT?authuser=2#scrollTo=wsh36fLxDKC3>. The notebook title is 'python101.ipynb'. The interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help) and a toolbar with options for CODE, TEXT, CELL, and a status bar showing 'CONNECTED' and 'EDITING'.

The notebook contains four code cells, each with a play button icon on the left and a console output below the code:

```
1 # Future Value
2 pv = 100
3 r = 0.1
4 n = 7
5 fv = pv * ((1 + (r)) ** n)
6 print(round(fv, 2))
```

194.87

```
[11] 1 amount = 100
      2 interest = 10 #10% = 0.01 * 10
      3 years = 7
      4
      5 future_value = amount * ((1 + (0.01 * interest)) ** years)
      6 print(round(future_value, 2))
```

194.87

```
[12] 1 # Python Function def
      2 def getfv(pv, r, n):
      3     fv = pv * ((1 + (r)) ** n)
      4     return fv
      5 fv = getfv(100, 0.1, 7)
      6 print(round(fv, 2))
```

194.87

```
[13] 1 # Python if else
      2 score = 80
      3 if score >=60 :
      4     print("Pass")
      5 else:
      6     print("Fail").
```

Pass

Python in Google Colab (Python101)

The screenshot shows a Google Colab notebook interface. At the top, the notebook is titled 'python101.ipynb' with a star icon. The menu bar includes 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', 'Help', and 'All changes saved'. On the right, there are icons for 'Comment', 'Share', 'Settings', and a user profile 'A'. Below the menu bar, the interface is divided into three main sections: a 'Table of contents' sidebar on the left, a central area with a dropdown menu, and a code editor on the right.

The 'Table of contents' sidebar lists the following topics:

- AI in Finance
 - Normative Finance and Financial Theories
 - Uncertainty and Risk**
 - Expected Utility Theory (EUT)
 - Mean-Variance Portfolio Theory (MVPT)
 - Capital Asset Pricing Model (CAPM)
 - Arbitrage Pricing Theory (APT)
 - Deep Learning for Financial Time Series Forecasting
 - Portfolio Optimization and Algorithmic Trading
 - Investment Portfolio Optimisation with Python
 - Efficient Frontier Portfolio Optimisation in Python
 - Investment Portfolio Optimization

The central dropdown menu is currently set to 'AI in Finance'. Below it, the code editor displays the following Python code:

```
1 import numpy as np
2
3 #The prices of the stock and bond today.
4 S0 = 10
5 B0 = 10
6 print('S0', S0)
7 print('B0', B0)
8
9 #The uncertain payoff of the stock and bond tomorrow.
10 S1 = np.array((20, 5))
11 B1 = np.array((11, 11))
12 print('S1', S1)
13 print('B1', B1)
14
15 #The market price vector
16 M0 = np.array((S0, B0))
```

On the right side of the code editor, there is a toolbar with icons for undo, redo, insert, comment, settings, print, and delete. Above the code editor, there is a status bar showing 'RAM' and 'Disk' usage, and a 'Editing' mode indicator.

Overlaid on the right side of the notebook is the text 'AI in Finance' in a large, bold, red font.

Python in Google Colab (Python101)

 python101.ipynb ☆
File Edit View Insert Runtime Tools Help [All changes saved](#)

Comment Share ⚙️ A

Table of contents

- Data Driven Finance
 - Financial Econometrics and Regression**
 - Data Availability
 - Normative Theories Revisited
 - Mean-Variance Portfolio Theory
 - Capital Asset Pricing Model
 - Arbitrage-Pricing Theory
 - Debunking Central Assumptions
 - Normality
 - Sample Data Sets
 - Real Financial Returns
 - Linear Relationships
 - Deep Learning for Financial Time Series Forecasting
 - Portfolio Optimization and Algorithmic Trading
 - Investment Portfolio Optimisation with Python
 - Efficient Frontier Portfolio Optimisation in Python
 - Investment Portfolio Optimization

+ Code + Text

✓ RAM Disk

Editing

Data Driven Finance

Financial Econometrics and Regression

```
[18] 1 import numpy as np
      2
      3 def f(x):
      4     return 2 + 1 / 2 * x
      5
      6 x = np.arange(-4, 5)
      7 x
```

```
array([-4, -3, -2, -1,  0,  1,  2,  3,  4])
```

```
1 y = f(x)
2 y
```

```
array([ 0.00,  0.50,  1.00,  1.50,  2.00,  2.50,  3.00,  3.50,  4.00])
```

```
1 print('x', x)
2
3 print('y', y)
4
5 beta = np.cov(x, y, ddof=0)[0, 1] / x.var()
6 print('beta', beta)
```

Python in Google Colab (Python101)

 python101.ipynb ☆

File Edit View Insert Runtime Tools Help All changes saved

Comment Share ⚙️ A

RAM  Disk  Editing ^

Table of contents X

- Financial Econometrics and Regression
- Data Availability
- Normative Theories Revisited
 - Mean-Variance Portfolio Theory
 - Capital Asset Pricing Model
 - Arbitrage-Pricing Theory
- Debunking Central Assumptions
- Normality
 - Sample Data Sets
 - Real Financial Returns
- Linear Relationships
- Financial Econometrics and Machine Learning
 - Machine Learning**
 - Data
 - Success
 - Capacity
 - Evaluation
 - Bias & Variance
 - Cross-Validation

Machine Learning

▼ Data

```
1 import numpy as np
2 import pandas as pd
3 from pylab import plt, mpl
4 np.random.seed(100)
5 plt.style.use('seaborn')
6 mpl.rcParams['savefig.dpi'] = 300
7 mpl.rcParams['font.family'] = 'serif'
8
9 url = 'http://hilpisch.com/aiif_eikon_eod_data.csv'
10
11 raw = pd.read_csv(url, index_col=0, parse_dates=True)['EUR=']
12 raw.head()
```

Date
2010-01-01 1.4323
2010-01-04 1.4411
2010-01-05 1.4368
2010-01-06 1.4412
2010-01-07 1.4318
Name: EUR=, dtype: float64

[2] 1 raw.tail()

Machine Learning

Python in Google Colab (Python101)


python101.ipynb
☆

File Edit View Insert Runtime Tools Help Saving...

Comment Share Settings A

RAM Disk Editing

Table of contents

- Financial Econometrics and Regression
- Data Availability
- Normative Theories Revisited
 - Mean-Variance Portfolio Theory
 - Capital Asset Pricing Model
 - Arbitrage-Pricing Theory
- Debunking Central Assumptions
- Normality
- Sample Data Sets
- Real Financial Returns
- Linear Relationships
- Financial Econometrics and Machine Learning
 - Machine Learning
 - Data
 - Success**
 - Capacity
 - Evaluation
 - Bias & Variance
 - Cross-Validation

+ Code + Text

Success

```

1 def MSE(l, p):
2     return np.mean((l - p) ** 2)

```

```

[9] 1 reg = np.polyfit(f, l, deg=5)
    2 reg

array([-0.01910626, -0.0147182 ,  0.10990388,  0.06007211, -0.20833598,
        -0.03275423])

```

```

[10] 1 p = np.polyval(reg, f)
     2 p

array([ 0.12088427,  0.11526131,  0.11080193,  0.10739461,  0.10493286,
         0.10331514,  0.10244475,  0.10222973,  0.10258281,  0.10342126,
         0.10466683,  0.10624564,  0.1080881 ,  0.1101288 ,  0.11230643,
         0.11456366,  0.11684709,  0.11910711,  0.12129784,  0.123377 ,
         0.12530587,  0.12704913,  0.12857481,  0.1298542 ,  0.1308617 ,
         0.1315748 ,  0.13197395,  0.13204243,  0.13176634,  0.13113443,
         0.13013803,  0.12877097,  0.12702948,  0.12491207,  0.12241947,
         0.11955452,  0.11632208,  0.11272891,  0.10878364,  0.1044966 ,
         0.09987977,  0.09494668,  0.0897123 ,  0.08419296,  0.07840627,
         0.07237098,  0.06610693,  0.05963494,  0.05297671,  0.04615473,
         0.03919218,  0.03211286,  0.02494106,  0.01770149,  0.01041918,
         0.00311939, -0.00417251, -0.0114311 , -0.01863101, -0.02574704,
        -0.03275423, -0.03962796, -0.04634406, -0.05287887, -0.05920936,
        -0.06531322, -0.07116897, -0.07675602, -0.08205478, -0.08704677,
         0.08171117,  0.08601251,  0.09001567,  0.09362002,  0.09681671,
         0.09959671,  0.10196101,  0.10390061,  0.10541551,  0.10650671,
         0.10717421,  0.10741791,  0.10723791,  0.10663331,  0.10560451,
         0.10415151,  0.10228431,  0.10000291,  0.09731731,  0.09422751,
         0.09074351,  0.08687531,  0.08263291,  0.07802731,  0.07306851,
         0.06776651,  0.06213231,  0.05617691,  0.05,

```

<https://tinyurl.com/aintpupython101>

Python in Google Colab (Python101)

 python101.ipynb ☆

File Edit View Insert Runtime Tools Help Saving...

Comment Share Settings A

RAM Disk Editing ^

Table of contents

Financial Econometrics and Regression

Data Availability

Normative Theories Revisited

Mean-Variance Portfolio Theory

Capital Asset Pricing Model

Arbitrage-Pricing Theory

Debunking Central Assumptions

Normality

Sample Data Sets

Real Financial Returns

Linear Relationships

Financial Econometrics and Machine Learning

Machine Learning

Data

Success

Capacity

Evaluation

Bias & Variance

Cross-Validation

+ Code + Text

✓ [11] 1 MSE(l, p)

0s

0.003416642295737103

✓ 1s

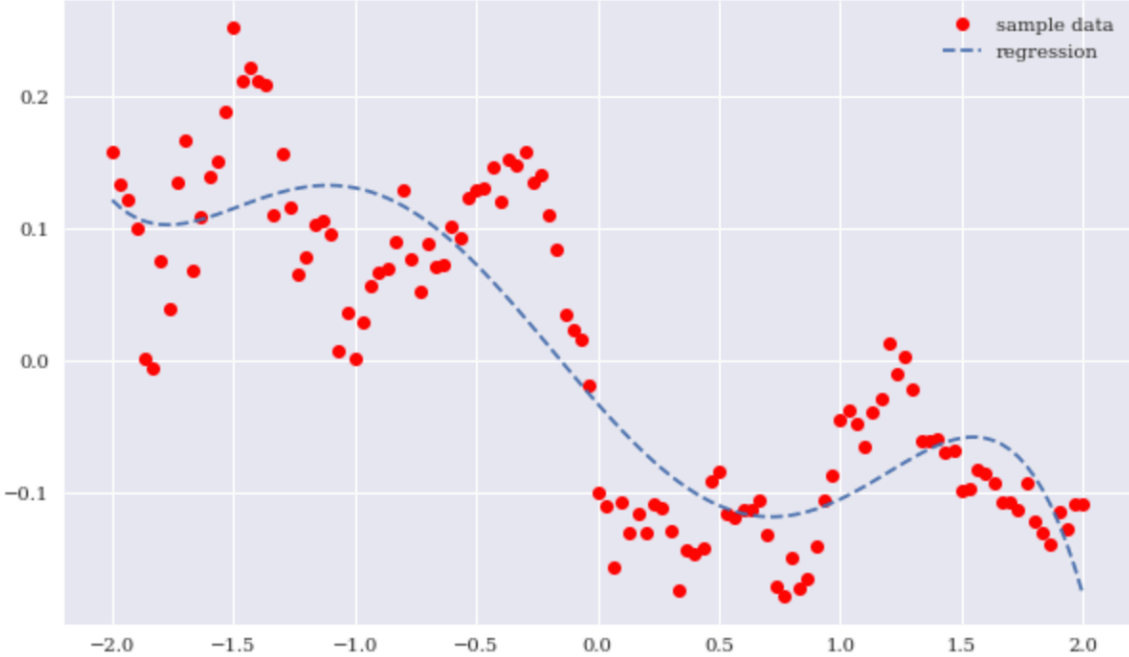
1 plt.figure(figsize=(10, 6))

2 plt.plot(f, l, 'ro', label='sample data')

3 plt.plot(f, p, '--', label='regression')

4 plt.legend()

<matplotlib.legend.Legend at 0x7f66a41d5750>



Python in Google Colab (Python101)



python101.ipynb ☆

File Edit View Insert Runtime Tools Help All changes saved

Comment

Share



A

Table of contents

- Financial Econometrics and Regression
- Data Availability
- Normative Theories Revisited
 - Mean-Variance Portfolio Theory
 - Capital Asset Pricing Model
 - Arbitrage-Pricing Theory
- Debunking Central Assumptions
- Normality
 - Sample Data Sets
 - Real Financial Returns
- Linear Relationships
- Financial Econometrics and Machine Learning
 - Machine Learning
 - Data
 - Success
 - Capacity**
 - Evaluation
 - Bias & Variance
 - Cross-Validation

+ Code + Text

✓ RAM
Disk

Editing

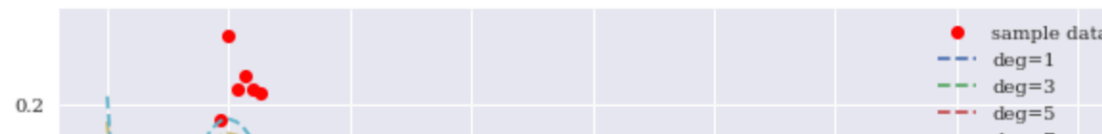
Capacity

```
[21] 1 reg = {}
      2 for d in range(1, 12, 2):
      3     reg[d] = np.polyfit(f, l, deg=d)
      4     p = np.polyval(reg[d], f)
      5     mse = MSE(l, p)
      6     print(f'{d:2d} | MSE={mse}')
```

```
1 | MSE=0.005322474034260403
3 | MSE=0.004353110724143185
5 | MSE=0.003416642295737103
7 | MSE=0.002738950177235401
9 | MSE=0.0014119616263308346
11 | MSE=0.0012651237868752398
```

```
1 plt.figure(figsize=(10, 6))
2 plt.plot(f, l, 'ro', label='sample data')
3 for d in reg:
4     p = np.polyval(reg[d], f)
5     plt.plot(f, p, '--', label=f'deg={d}')
6 plt.legend()
```

<matplotlib.legend.Legend at 0x7f6630300310>



<https://tinyurl.com/aintpupython101>

Python in Google Colab (Python101)



python101.ipynb ☆

File Edit View Insert Runtime Tools Help Saving...

Comment

Share



A

Table of contents

- Financial Econometrics and Regression
- Data Availability
- Normative Theories Revisited
 - Mean-Variance Portfolio Theory
 - Capital Asset Pricing Model
 - Arbitrage-Pricing Theory
- Debunking Central Assumptions
- Normality
 - Sample Data Sets
 - Real Financial Returns
- Linear Relationships
- Financial Econometrics and Machine Learning
 - Machine Learning
 - Data
 - Success
 - Capacity**
 - Evaluation
 - Bias & Variance
 - Cross-Validation

+ Code + Text

✓ RAM
Disk

Editing

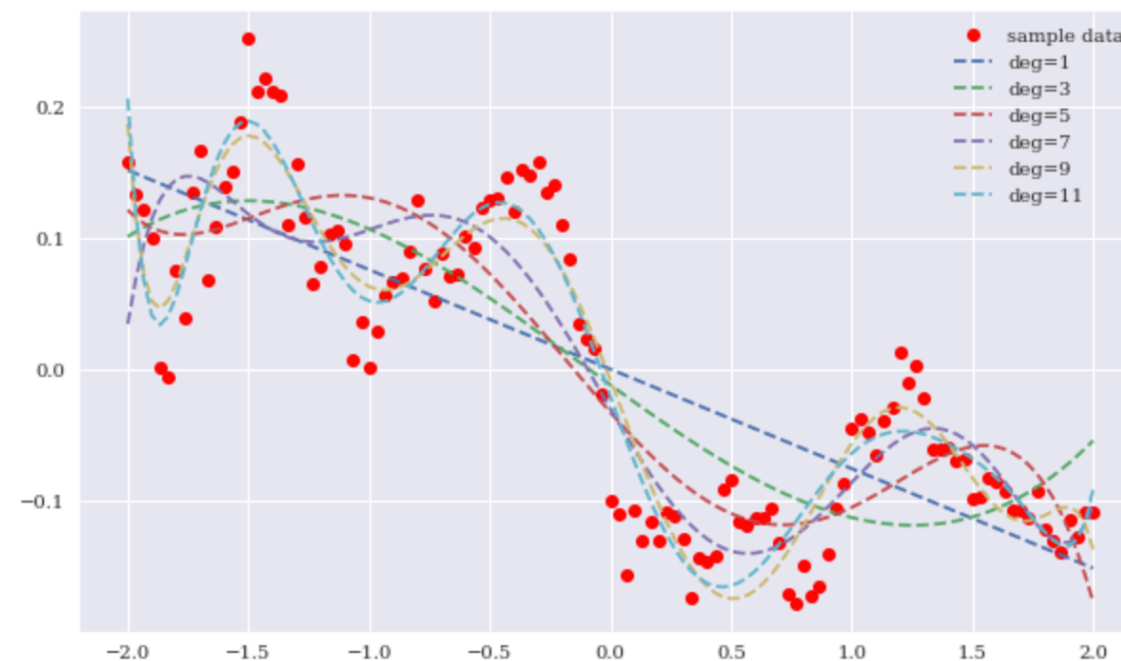


```
9 | MSE=0.0014119616263308346
11 | MSE=0.0012651237868752398
```



```
1 plt.figure(figsize=(10, 6))
2 plt.plot(f, l, 'ro', label='sample data')
3 for d in reg:
4     p = np.polyval(reg[d], f)
5     plt.plot(f, p, '--', label=f'deg={d}')
6 plt.legend()
```

```
<matplotlib.legend.Legend at 0x7f6630300310>
```



Python in Google Colab (Python101)

```
def create_dnn_model(hl=1, hu=256):
    ''' Function to create Keras DNN model.
    Parameters
    =====
    hl: int
    number of hidden layers
    hu: int
    number of hidden units (per layer)
    '''
    model = Sequential()
    for _ in range(hl):
        model.add(Dense(hu, activation='relu', input_dim=1))
    model.add(Dense(1, activation='linear'))
    model.compile(loss='mse', optimizer='rmsprop')
    return model

model = create_dnn_model(3)

model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_2 (Dense)	(None, 256)	512
dense_3 (Dense)	(None, 256)	65792
dense_4 (Dense)	(None, 256)	65792
dense_5 (Dense)	(None, 1)	257

```
=====
Total params: 132,353
Trainable params: 132,353
Non-trainable params: 0
```

Python in Google Colab (Python101)



python101.ipynb ☆

File Edit View Insert Runtime Tools Help [All changes saved](#)

Comment

Share



A

Table of contents

- Financial Econometrics and Regression
- Data Availability
- Normative Theories Revisited
 - Mean-Variance Portfolio Theory
 - Capital Asset Pricing Model
 - Arbitrage-Pricing Theory
- Debunking Central Assumptions
- Normality
 - Sample Data Sets
 - Real Financial Returns
- Linear Relationships
- Financial Econometrics and Machine Learning
 - Machine Learning
 - Data
 - Success
 - Capacity**
 - Evaluation
 - Bias & Variance
 - Cross-Validation

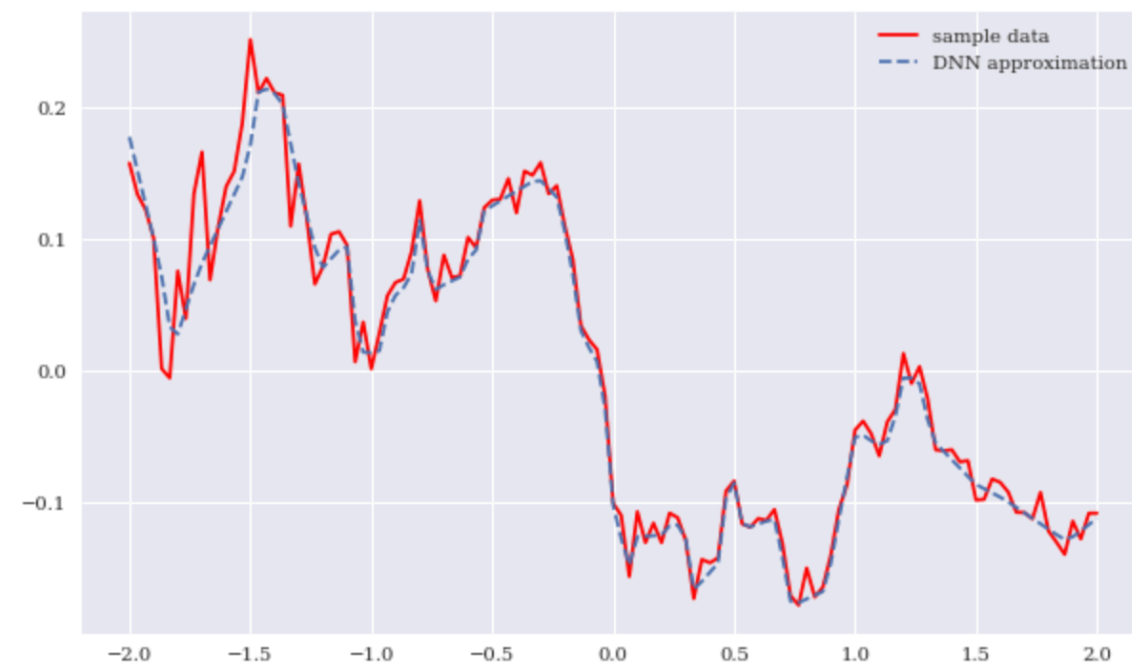
+ Code + Text

✓ RAM
Disk

Editing



```
1 p = model.predict(f).flatten()
2
3 MSE(1, p)
4
5 plt.figure(figsize=(10, 6))
6 plt.plot(f, 1, 'r', label='sample data')
7 plt.plot(f, p, '--', label='DNN approximation')
8 plt.legend();
```



Python in Google Colab (Python101)

 python101.ipynb ☆
File Edit View Insert Runtime Tools Help [All changes saved](#)

Comment

Share

⚙️

A

Table of contents

Financial Econometrics and Regression

Data Availability

Normative Theories Revisited

Mean-Variance Portfolio Theory

Capital Asset Pricing Model

Arbitrage-Pricing Theory

Debunking Central Assumptions

Normality

Sample Data Sets

Real Financial Returns

Linear Relationships

Financial Econometrics and Machine Learning

Machine Learning

Data

Success

Capacity

Evaluation

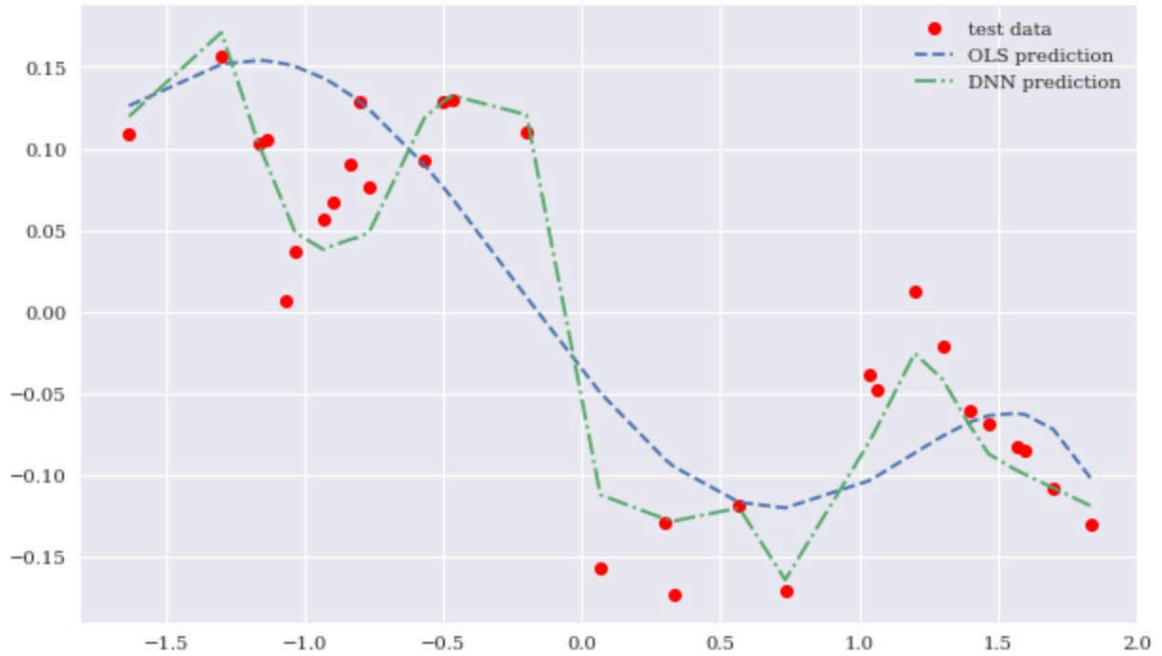
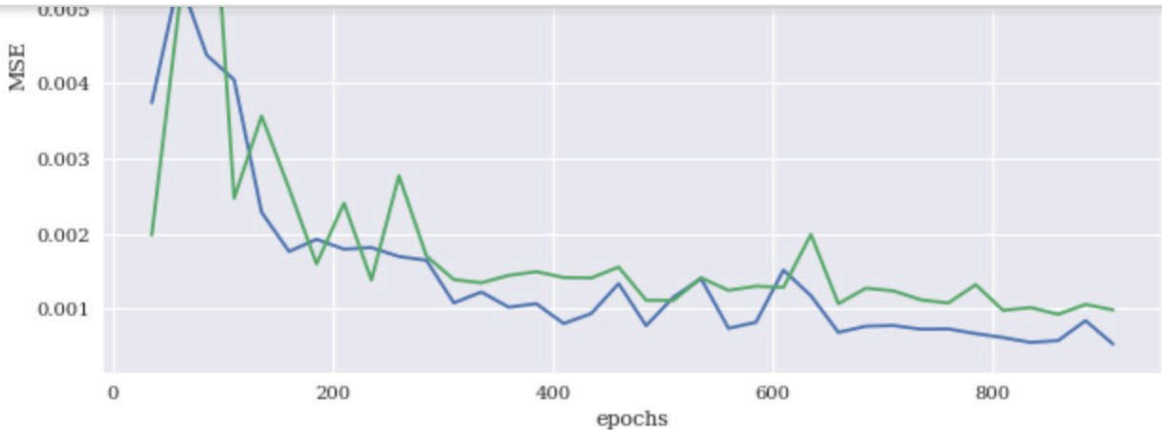
Bias & Variance

Cross-Validation

+ Code + Text

✓ RAM Disk

Editing



Python in Google Colab (Python101)


python101.ipynb
☆

File Edit View Insert Runtime Tools Help All changes saved

Comment Share Settings A

RAM Disk Editing

Table of contents

- Mean-Variance Portfolio Theory
- Capital Asset Pricing Model
- Arbitrage-Pricing Theory
- Debunking Central Assumptions
- Normality
- Sample Data Sets
- Real Financial Returns
- Linear Relationships
- Financial Econometrics and Machine Learning
- Machine Learning
- Data
- Success
- Capacity
- Evaluation
- Bias & Variance
- Cross-Validation
- AI-First Finance
- Efficient Markets**
- Market Prediction Based on Returns Data
- Market Prediction With More Features
- Market Prediction Intraday

Efficient Markets

```

1 import numpy as np
2 import pandas as pd
3 from pylab import plt, mpl
4 plt.style.use('seaborn')
5 mpl.rcParams['savefig.dpi'] = 300
6 mpl.rcParams['font.family'] = 'serif'
7 pd.set_option('precision', 4)
8 np.set_printoptions(suppress=True, precision=4)
9
10 url = 'http://hilpisch.com/aiif_eikon_eod_data.csv'
11 data = pd.read_csv(url, index_col=0, parse_dates=True).dropna()
12 (data / data.iloc[0]).plot(figsize=(10, 6), cmap='coolwarm')

```

<matplotlib.axes._subplots.AxesSubplot at 0x7f29f972f210>



The plot displays the performance of several financial instruments over a ten-year period from 2010 to 2020. The y-axis represents the value of the index or commodity, ranging from 0 to 14. The x-axis represents the year. The plot shows that most indices and commodities have increased significantly over the decade, with AAPL.O and MSFT.O showing the highest growth. Other indices like INTC.O, AMZN.O, GS.N, SPY, and SPX also show upward trends. Commodities like EUR, XAU, GDX, and GLD show more modest growth.

Python in Google Colab (Python101)

python101.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Table of contents

Mean-Variance Portfolio Theory

Capital Asset Pricing Model

Arbitrage-Pricing Theory

Debunking Central Assumptions

Normality

Sample Data Sets

Real Financial Returns

Linear Relationships

Financial Econometrics and Machine Learning

Machine Learning

Data

Success

Capacity

Evaluation

Bias & Variance

Cross-Validation

AI-First Finance

Efficient Markets

Market Prediction Based on Returns Data

Market Prediction With More Features

Market Prediction Intraday

+ Code + Text

RAM Disk

Editing

Market Prediction Based on Returns Data

1 rets = np.log(data / data.shift(1))

2

3 rets.dropna(inplace=True)

4 rets

	AAPL.O	MSFT.O	INTC.O	AMZN.O	GS.N	SPY	.SPX	.VIX	EUR=	XAU=	GDX	GLD
Date												
2010-01-05	0.0017	0.0003	-0.0005	0.0059	0.0175	2.6436e-03	3.1108e-03	-0.0350	-2.9883e-03	-0.0012	0.0096	-0.0009
2010-01-06	-0.0160	-0.0062	-0.0034	-0.0183	-0.0107	7.0379e-04	5.4538e-04	-0.0099	3.0577e-03	0.0176	0.0240	0.0164
2010-01-07	-0.0019	-0.0104	-0.0097	-0.0172	0.0194	4.2124e-03	3.9933e-03	-0.0052	-6.5437e-03	-0.0058	-0.0049	-0.0062
2010-01-08	0.0066	0.0068	0.0111	0.0267	-0.0191	3.3223e-03	2.8775e-03	-0.0500	6.5437e-03	0.0037	0.0150	0.0050
2010-01-11	-0.0089	-0.0128	0.0057	-0.0244	-0.0159	1.3956e-03	1.7452e-03	-0.0325	6.9836e-03	0.0144	0.0066	0.0132
...	...	...	...	...	...	...	...	...	...	...	...	...
2019-12-24	0.0010	-0.0002	0.0030	-0.0021	0.0036	3.1131e-05	-1.9543e-04	0.0047	9.0200e-05	0.0091	0.0315	0.0094
2019-12-26	0.0196	0.0082	0.0069	0.0435	0.0056	5.3092e-03	5.1151e-03	-0.0016	8.1143e-04	0.0083	0.0145	0.0078
2019-12-27	-0.0004	0.0018	0.0043	0.0006	-0.0024	-2.4775e-04	3.3951e-05	0.0598	7.0945e-03	-0.0006	-0.0072	-0.0004

Python in Google Colab (Python101)

python101.ipynb ☆

File Edit View Insert Runtime Tools Help [All changes saved](#)

Comment Share Settings A

RAM Disk Editing

Table of contents

- Mean-Variance Portfolio Theory
- Capital Asset Pricing Model
- Arbitrage-Pricing Theory
- Debunking Central Assumptions
- Normality
- Sample Data Sets
- Real Financial Returns
- Linear Relationships
- Financial Econometrics and Machine Learning
- Machine Learning
- Data
- Success
- Capacity
- Evaluation
- Bias & Variance
- Cross-Validation
- AI-First Finance
- Efficient Markets
- Market Prediction Based on Returns Data
- Market Prediction With More Features**
- Market Prediction Intraday

Market Prediction With More Features

```
1 url = 'http://hilpisch.com/aiif_eikon_eod_data.csv'
2
3 data = pd.read_csv(url, index_col=0, parse_dates=True).dropna()
4 data
```

	AAPL.O	MSFT.O	INTC.O	AMZN.O	GS.N	SPY	.SPX	.VIX	EUR=	XAU=	GDX	GLD
Date												
2010-01-04	30.5728	30.950	20.88	133.90	173.08	113.33	1132.99	20.04	1.4411	1120.0000	47.71	109.80
2010-01-05	30.6257	30.960	20.87	134.69	176.14	113.63	1136.52	19.35	1.4368	1118.6500	48.17	109.70
2010-01-06	30.1385	30.770	20.80	132.25	174.26	113.71	1137.14	19.16	1.4412	1138.5000	49.34	111.51
2010-01-07	30.0828	30.452	20.60	130.00	177.67	114.19	1141.69	19.06	1.4318	1131.9000	49.10	110.82
2010-01-08	30.2828	30.660	20.83	133.52	174.31	114.57	1144.98	18.13	1.4412	1136.1000	49.84	111.37
...	...	...	...	...	...	...	...	...	...	...	...	...
2019-12-24	284.2700	157.380	59.41	1789.21	229.91	321.23	3223.38	12.67	1.1087	1498.8100	28.66	141.27
2019-12-26	289.9100	158.670	59.82	1868.77	231.21	322.94	3239.91	12.65	1.1096	1511.2979	29.08	142.38
2019-12-27	289.8000	158.960	60.08	1869.80	230.66	322.86	3240.02	13.43	1.1175	1510.4167	28.87	142.33
2019-12-30	291.5200	157.590	59.62	1846.89	229.80	321.08	3221.29	14.82	1.1197	1515.1230	29.49	142.63
2019-12-31	293.6500	157.700	59.85	1847.84	229.93	321.86	3230.78	13.78	1.1210	1517.0100	29.28	142.90

2516 rows x 12 columns

<https://tinyurl.com/aintpupython101>

Python in Google Colab (Python101)


python101.ipynb ☆

File Edit View Insert Runtime Tools Help Saving...

Comment Share

RAM Disk

Editing

Market Prediction Intraday

```

1 url = 'http://hilpisch.com/aiif_eikoh_data/data.csv'
2 data = pd.read_csv(url, index_col=0, parse_dates=True) # dropna()
3 data.tail()

```

	AAPL.O	MSFT.O	INTC.O	AMZN.O	GS.N	SPY	.SPX	.VIX	EUR=	XAU=	GDX	GLD
2019-12-31 20:00:00	292.36	157.2845	59.575	1845.22	228.92	320.94	3219.75	14.16	1.1215	1519.6451	29.40	143.12
2019-12-31 21:00:00	293.37	157.4900	59.820	1846.95	229.89	321.89	3230.56	13.92	1.1216	1517.3600	29.29	142.93
2019-12-31 22:00:00	293.82	157.9000	59.990	1850.20	229.93	322.39	3230.78	13.78	1.1210	1517.0100	29.30	142.90
2019-12-31 23:00:00	293.75	157.8300	59.910	1851.00	NaN	322.22	NaN	NaN	1.1211	1516.8900	29.40	142.88
2020-01-01 00:00:00	293.81	157.8800	59.870	1850.10	NaN	322.32	NaN	NaN	1.1211	NaN	29.34	143.00

```

1 data.info()

```

```

<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 5529 entries, 2019-03-01 00:00:00 to 2020-01-01 00:00:00
Data columns (total 12 columns):
#   Column   Non-Null Count  Dtype
---  -
0    AAPL.O   3384 non-null   float64
1    MSFT.O   3378 non-null   float64

```

Table of contents

- Mean-Variance Portfolio Theory
- Capital Asset Pricing Model
- Arbitrage-Pricing Theory
- Debunking Central Assumptions
- Normality
- Sample Data Sets
- Real Financial Returns
- Linear Relationships
- Financial Econometrics and Machine Learning
- Machine Learning
- Data
- Success
- Capacity
- Evaluation
- Bias & Variance
- Cross-Validation
- AI-First Finance
- Efficient Markets
- Market Prediction Based on Returns Data
- Market Prediction With More Features
- Market Prediction Intraday**

Summary

- **Kinh tế lượng tài chính**
 - Lý thuyết tài chính, Phân tích hồi quy dùng OLS
- **Máy học**
 - Quá trình học, Đánh giá, Bias and variance
 - Cross-validation
- **Trí tuệ nhân tạo trong tài chính**
 - Thị trường hiệu quả
 - Dự đoán thị trường dựa trên dữ liệu lợi nhuận
 - Dự đoán thị trường với nhiều tính năng hơn

References

- Yves Hilpisch (2020), Artificial Intelligence in Finance: A Python-Based Guide, O'Reilly Media, <https://github.com/yhilpisch/aiif> .
- Chris Brooks (2019), Introductory Econometrics for Finance, 4th Edition, Cambridge University Press
- Oliver Linton (2019), Financial Econometrics: Models and Methods, Cambridge University Press
- Tom Mitchell (1997), Machine Learning, McGraw-Hill.
- Max Tegmark (2017), Life 3.0: Being human in the age of artificial intelligence. Vintage.
- Ajay Agrawal, Joshua Gans, and Avi Goldfarb (2018). Prediction machines: the simple economics of artificial intelligence. Harvard Business Press.
- Eugene F. Fama (1995), "Random walks in stock market prices." Financial Analysts Journal 51, no. 1, 75-80.
- Ruey S. Tsay (2005), Analysis of financial time series, Wiley.

**THANKS FOR YOUR
ATTENTION!**